

Namespace ASE

Classes

[Array](#)

The class for storing arrays of numbers

[BOOSEException](#)

Generic BOOSE Language exception Extends Exception

[Canvas](#)

The drawing canvas to be used with this implementation of BOOSE

[CanvasCommand](#)

Abstract class to be extended by CommandXParameter

[Circle](#)

Command for drawing circles around current cursor position

[Command](#)

Base class for all drawing commands

[CommandException](#)

Exception thrown by Command class, message depends on the type of Command that throws the exception

[CommandFactory](#)

Factory class for creating commands

[CommandOneParameter](#)

Abstract class for Commands with one parameter

[CommandThreeParameters](#)

Abstract class for Commands with three parameters

[CommandTwoParameters](#)

Abstract class for Commands with two parameters

[Else](#)

This class doesn't handle what happens when an if condition is false, rather it skips ahead to "end if" when the else keyword is encountered after completing the contents of an "if"

[End](#)

handles the end of "if", "while" and "for"

[Evaluation](#)

An evaluation encompasses all things that can have a value, such as variables, expressions and conditions used in loops and ifs.

[FactoryException](#)

Exception thrown by CommandFactory

[For](#)

A for loop, implemented as an if, while and reassign

[If](#)

The If statement

[Int](#)

Stores an integer

[MoveTo](#)

Command for moving pen to specified position on canvas

[Parser](#)

The class used for reading a BOOSE program

[ParserException](#)

Exception thrown by Parser class

[Peek](#)

Used to access values stored in an array

[PenColour](#)

Command for altering the pen colour using rgb values

[Poke](#)

Used to input data into an array

[Program](#)

Default blazor template code

[Real](#)

Stores a floating point number

[Reassign](#)

Assigns a new value to a variable

[Rect](#)

Command for drawing rectangles at current cursor position

[StoredProgram](#)

Class for storing command objects

[StoredProgramException](#)

Exception thrown by StoredProgram class

[While](#)

The while loop

[Write](#)

Command for writing to the canvas

Interfaces

[ICanvas](#)

The interface to be implemented by the Canvas class

[ICommand](#)

The interface to be implemented by the Command class

[ICommandFactory](#)

The interface to be implemented by the CommandFactory class

[IParser](#)

The interface to be implemented by the Parser class

[IStoredProgram](#)

Class Array

Namespace: [ASE](#)

Assembly: ASE.dll

The class for storing arrays of numbers

```
public class Array : Evaluation, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [Evaluation](#) ← Array

Implements

[ICommand](#)

Inherited Members

[Evaluation.evaluatedExpression](#) , [Evaluation.expression](#) , [Evaluation.value](#) , [Evaluation.varName](#) , [Evaluation.CheckParameters\(\)](#) , [Evaluation.Compile\(\)](#) , [Evaluation.Evaluate\(string, StoredProgram\)](#) , [Evaluation.Reassign\(string, StoredProgram\)](#) , [Evaluation.Expression](#) , [Evaluation.Value](#) , [Evaluation.VarName](#) , [Evaluation.scopeID](#) , [Command.ProcessParameters\(string\)](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Program](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Array()

Constructor assigns the array command a name so it can be identified as such

```
public Array()
```

Fields

contents

The actual array that stores the contents of the BOOSE array

```
public double[] contents
```

Field Value

[double](#)[↗][]

Methods

Execute()

Gives the array a name, type and a starting size, 1 if unspecified

```
public override Task Execute()
```

Returns

[Task](#)[↗]

Exceptions

[CommandException](#)

Thrown if an incorrect number of arguments is given

Class BOOSEException

Namespace: [ASE](#)

Assembly: ASE.dll

Generic BOOSE Language exception Extends Exception

```
public class BOOSEException : Exception, ISerializable
```

Inheritance

[object](#)  ← [Exception](#)  ← BOOSEException

Implements

[ISerializable](#) 

Derived

[CommandException](#), [FactoryException](#), [ParserException](#), [StoredProgramException](#)

Inherited Members

[Exception.GetBaseException\(\)](#)  , [Exception.ToString\(\)](#)  , [Exception.GetType\(\)](#)  ,
[Exception.TargetSite](#)  , [Exception.Message](#)  , [Exception.Data](#)  , [Exception.InnerException](#)  ,
[Exception.HelpLink](#)  , [Exception.Source](#)  , [Exception.HResult](#)  , [Exception.StackTrace](#)  ,
[Exception.SerializeObjectState](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

BOOSEException(string)

```
public BOOSEException(string msg)
```

Parameters

msg [string](#) 

Class Canvas

Namespace: [ASE](#)

Assembly: ASE.dll

The drawing canvas to be used with this implementation of BOOSE

```
public class Canvas : ICanvas
```

Inheritance

[object](#)  ← Canvas

Implements

[ICanvas](#)

Inherited Members

[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Canvas(BECanvasComponent, Canvas2DContext)

The constructor takes both the component and context as arguments as generating a context is done asynchronously which should be avoided in constructors

```
public Canvas(BECanvasComponent component, Canvas2DContext context)
```

Parameters

component [BECanvasComponent](#)

The html canvas element to draw to

context [Canvas2DContext](#)

The context of the drawing canvas

Properties

Component

The reference to the html5 element

```
public BECanvasComponent Component { get; set; }
```

Property Value

[BECanvasComponent](#)

Context

Handle for drawing to the html canvas

```
public Canvas2DContext Context { get; set; }
```

Property Value

[Canvas2DContext](#)

XPos

The x axis position of the pen on the canvas relative to the top left corner

```
public double XPos { get; set; }
```

Property Value

[double](#)

YPos

The y axis position of the pen on the canvas relative to the top left corner

```
public double YPos { get; set; }
```

Property Value

[double](#)[↗]

Methods

Circle(double, bool)

Draws a circle to the canvas

```
public Task Circle(double radius, bool filled)
```

Parameters

radius [double](#)[↗]

The radius of the circle in pixels

filled [bool](#)[↗]

The Circle will be filled if set to true

Returns

[Task](#)[↗]

Clear()

Clears the canvas

```
public Task Clear()
```

Returns

[Task](#)[↗]

DrawTo(double, double)

Draws line from current position to the one specified

```
public Task DrawTo(double x, double y)
```

Parameters

x [double](#)

x coordinate to draw to

y [double](#)

y coordinate to draw to

Returns

[Task](#)

GetBitmap()

Temporary - returns the context of the canvas

```
public object GetBitmap()
```

Returns

[object](#)

The Canvas2DContext of the canvas

MoveTo(double, double)

Moves pen to specified position

```
public Task MoveTo(double x, double y)
```

Parameters

x [double](#)

x coordinate to move to

y [double](#)

y coordinate to move to

Returns

[Task](#)

Rect(double, double, bool)

Draws rectangle on canvas

```
public Task Rect(double width, double height, bool filled)
```

Parameters

width [double](#)

The width of the rectangle

height [double](#)

The height of the rectangle

filled [bool](#)

Rectangle will be filled if set to true

Returns

[Task](#)

Reset()

Clears canvas and returns pen to top left corner

```
public Task Reset()
```

Returns

[Task](#)

Set(double, double)

Resizes canvas

```
public Task Set(double x, double y)
```

Parameters

x [double](#)

Width of canvas

y [double](#)

Height of canvas

Returns

[Task](#)

Returns completed task for use with async methods

SetColour(double, double, double)

Sets pen colour to rgb value

```
public Task SetColour(double red, double green, double blue)
```

Parameters

red [double](#)

r value 0-255

green [double](#)

g value 0-255

blue [double](#)

b value 0-255

Returns

[Task](#)

Tri(double, double)

Draws triangle to canvas

```
public Task Tri(double width, double height)
```

Parameters

width [double](#)

The width of the triangle

height [double](#)

The height of the triangle

Returns

[Task](#)

WriteText(string)

Writes a message to the canvas

```
public Task WriteText(string text)
```

Parameters

text [string](#) 

Text to be written

Returns

[Task](#) 

Class CanvasCommand

Namespace: [ASE](#)

Assembly: ASE.dll

Abstract class to be extended by CommandXParameter

```
public abstract class CanvasCommand : Command, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← CanvasCommand

Implements

[ICommand](#)

Derived

[CommandOneParameter](#)

Inherited Members

[Command.CheckParameters\(\)](#), [Command.Compile\(\)](#), [Command.Execute\(\)](#), [Command.ProcessParameters\(string\)](#), [Command.Set\(StoredProgram, string\)](#), [Command.Name](#), [Command.ParameterList](#), [Command.Parameters](#), [Command.Program](#), [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ToString\(\)](#) , [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

CanvasCommand()

Generic constructor

```
public CanvasCommand()
```

CanvasCommand(ICanvas)

Constructor that sets all properties to their default values

```
public CanvasCommand(ICanvas c)
```

Parameters

c [ICanvas](#)

The canvas to be drawn to

Fields

canvas

The canvas to be drawn to

```
protected ICanvas canvas
```

Field Value

[ICanvas](#)

xPos

Redundant - current x coordinate of pen on canvas

```
protected int xPos
```

Field Value

[int](#)

yPos

Redundant - current y coordinate of pen on canvas

```
protected int yPos
```

Field Value

[int](#)

Properties

Canvas

Redundant? - The canvas to be drawn to

```
public ICanvas Canvas { get; set; }
```

Property Value

[ICanvas](#)

Class Circle

Namespace: [ASE](#)

Assembly: ASE.dll

Command for drawing circles around current cursor position

```
public class Circle : CommandOneParameter, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [CanvasCommand](#) ← [CommandOneParameter](#) ← Circle

Implements

[ICommand](#)

Inherited Members

[CommandOneParameter.param1](#) , [CommandOneParameter.param1unprocessed](#) , [CommandOneParameter.CheckParameters\(\)](#) , [CanvasCommand.canvas](#) , [CanvasCommand.xPos](#) , [CanvasCommand.yPos](#) , [CanvasCommand.Canvas](#) , [Command.Compile\(\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.Set\(StoredProgram,string\)](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Program](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object,object\)](#)  , [object.ReferenceEquals\(object,object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Circle()

Constructor for factory instantiation

```
public Circle()
```

Circle(Canvas, int)

redundant? - Creates Circle object with properties initialised

```
public Circle(Canvas c, int radius)
```

Parameters

c [Canvas](#)

Canvas to draw to

radius [int](#)

The radius of the circle

Methods

Execute()

Draws the circle

```
public override Task Execute()
```

Returns

[Task](#)

Class Command

Namespace: [ASE](#)

Assembly: ASE.dll

Base class for all drawing commands

```
public abstract class Command : ICommand
```

Inheritance

[object](#)  ← Command

Implements

[ICommand](#)

Derived

[CanvasCommand](#), [Else](#), [Evaluation](#)

Inherited Members

[object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ToString\(\)](#) , [object.Equals\(object\)](#) ,
[object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

Command()

```
protected Command()
```

Properties

Name

The name of the command for identification when stored in StoredProgram object

```
public string Name { get; set; }
```


Property Value

[string](#)

ParameterList

The raw list of parameters

```
public string ParameterList { get; set; }
```

Property Value

[string](#)

Parameters

Array of split parameters

```
public string[] Parameters { get; set; }
```

Property Value

[string](#)[]

Program

The StoredProgram object a Command is stored in, currently used for accessing Canvas

```
public StoredProgram Program { get; set; }
```

Property Value

[StoredProgram](#)

Methods

CheckParameters()

Abstract method for bound checking Command parameters

```
public abstract Task CheckParameters()
```

Returns

[Task](#)

Returns task for use with async methods

Compile()

Currently a call to ProcessParameters

```
public virtual Task Compile()
```

Returns

[Task](#)

Execute()

Unimplemented base method for executing commands

```
public virtual Task Execute()
```

Returns

[Task](#)

Return Task for use with async Methods

Exceptions

[NotImplementedException](#)

ProcessParameters(string)

Splits raw parameter list into individual parameters

```
public Task<int> ProcessParameters(string ParameterList)
```

Parameters

ParameterList [string](#)

Redundant? - raw list of parameters

Returns

[Task](#)<[int](#)>

The number of parameters

Set(StoredProgram, string)

Assigns associated StoredProgram and raw parameter list. To be used after factory instantiation.

```
public virtual Task Set(StoredProgram Program, string Params)
```

Parameters

Program [StoredProgram](#)

Params [string](#)

Returns

[Task](#)

Class CommandException

Namespace: [ASE](#)

Assembly: ASE.dll

Exception thrown by Command class, message depends on the type of Command that throws the exception

```
public class CommandException : BOOSEException, ISerializable
```

Inheritance

[object](#)  ← [Exception](#)  ← [BOOSEException](#) ← CommandException

Implements

[ISerializable](#) 

Inherited Members

[Exception.GetBaseException\(\)](#)  , [Exception.ToString\(\)](#)  , [Exception.GetType\(\)](#)  ,
[Exception.TargetSite](#)  , [Exception.Message](#)  , [Exception.Data](#)  , [Exception.InnerException](#)  ,
[Exception.HelpLink](#)  , [Exception.Source](#)  , [Exception.HResult](#)  , [Exception.StackTrace](#)  ,
[Exception.SerializeObjectState](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

CommandException(string)

```
public CommandException(string commandtype)
```

Parameters

commandtype [string](#) 

Class CommandFactory

Namespace: [ASE](#)

Assembly: ASE.dll

Factory class for creating commands

```
public class CommandFactory : ICommandFactory
```

Inheritance

[object](#)  ← CommandFactory

Implements

[ICommandFactory](#)

Inherited Members

[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

CommandFactory(Canvas)

Constructor initialises _canvas property

```
public CommandFactory(Canvas c)
```

Parameters

c [Canvas](#)

The Canvas to draw to

Methods

MakeCommand(string, StoredProgram)

Creates blank object of the Command specified

```
public virtual Task<ICommand> MakeCommand(string commandType, StoredProgram Program)
```

Parameters

commandType [string](#)

The type of command to create

Program [StoredProgram](#)

Returns

[Task](#) <[ICommand](#)>

The newly created command

Exceptions

[FactoryException](#)

Thrown if invalid command type is passed

Class CommandOneParameter

Namespace: [ASE](#)

Assembly: ASE.dll

Abstract class for Commands with one parameter

```
public abstract class CommandOneParameter : CanvasCommand, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [CanvasCommand](#) ← CommandOneParameter

Implements

[ICommand](#)

Derived

[Circle](#), [CommandTwoParameters](#), [End](#), [Reassign](#), [Write](#)

Inherited Members

[CanvasCommand.canvas](#) , [CanvasCommand.xPos](#) , [CanvasCommand.yPos](#) ,
[CanvasCommand.Canvas](#) , [Command.Compile\(\)](#) , [Command.Execute\(\)](#) ,
[Command.ProcessParameters\(string\)](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.Name](#) ,
[Command.ParameterList](#) , [Command.Parameters](#) , [Command.Program](#) , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

CommandOneParameter()

Generic constructor

```
public CommandOneParameter()
```

CommandOneParameter(Canvas)

Uses inherited constructor to assign a Canvas to draw to

```
public CommandOneParameter(Canvas c)
```

Parameters

c [Canvas](#)

The Canvas to draw to

Fields

param1

The first parameter of the Command

```
protected double param1
```

Field Value

[double](#)

param1unprocessed

Redundant? - The unprocessed first parameter

```
protected string param1unprocessed
```

Field Value

[string](#)

Methods

CheckParameters()

Extracts and evaluates the first parameter


```
public override Task CheckParameters()
```

Returns

[Task](#)

Exceptions

[CommandException](#)

Thrown if parameter is not an number

Class CommandThreeParameters

Namespace: [ASE](#)

Assembly: ASE.dll

Abstract class for Commands with three parameters

```
public class CommandThreeParameters : CommandTwoParameters, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [CanvasCommand](#) ← [CommandOneParameter](#) ← [CommandTwoParameters](#) ← CommandThreeParameters

Implements

[ICommand](#)

Derived

[For](#), [If](#), [Peek](#), [PenColour](#), [Poke](#)

Inherited Members

[CommandTwoParameters.param2](#) , [CommandTwoParameters.param2unprocessed](#) , [CommandOneParameter.param1](#) , [CommandOneParameter.param1unprocessed](#) , [CanvasCommand.canvas](#) , [CanvasCommand.xPos](#) , [CanvasCommand.yPos](#) , [CanvasCommand.Canvas](#) , [Command.Compile\(\)](#) , [Command.Execute\(\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Program](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

CommandThreeParameters()

Generic constructor

```
public CommandThreeParameters()
```

Fields

param3

The third parameter of the Command

```
protected double param3
```

Field Value

[double](#)

param3unprocessed

Redundant? - The unprocessed third parameter

```
protected string param3unprocessed
```

Field Value

[string](#)

Methods

CheckParameters()

Uses inherited version of itself to extract the first and second parameter from list of parameters then extracts the third

```
public override Task CheckParameters()
```

Returns

[Task](#)

Class CommandTwoParameters

Namespace: [ASE](#)

Assembly: ASE.dll

Abstract class for Commands with two parameters

```
public abstract class CommandTwoParameters : CommandOneParameter, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [CanvasCommand](#) ← [CommandOneParameter](#) ← CommandTwoParameters

Implements

[ICommand](#)

Derived

[CommandThreeParameters](#), [MoveTo](#), [Rect](#)

Inherited Members

[CommandOneParameter.param1](#), [CommandOneParameter.param1unprocessed](#),
[CanvasCommand.canvas](#), [CanvasCommand.xPos](#), [CanvasCommand.yPos](#),
[CanvasCommand.Canvas](#), [Command.Compile\(\)](#), [Command.Execute\(\)](#),
[Command.ProcessParameters\(string\)](#), [Command.Set\(StoredProgram, string\)](#), [Command.Name](#),
[Command.ParameterList](#), [Command.Parameters](#), [Command.Program](#), [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ToString\(\)](#) , [object.Equals\(object\)](#) ,
[object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

CommandTwoParameters()

Generic constructor

```
public CommandTwoParameters()
```

CommandTwoParameters(Canvas)

Redundant - Constructor that sets canvas property

```
public CommandTwoParameters(Canvas c)
```

Parameters

c [Canvas](#)

The canvas to draw to

Fields

param2

The second parameter of the Command

```
protected double param2
```

Field Value

[double](#)

param2unprocessed

Redundant? - The unprocessed third parameter

```
protected string param2unprocessed
```

Field Value

[string](#)

Methods

CheckParameters()

Uses inherited version of itself to extract the first argument from array then extracts the second one

```
public override Task CheckParameters()
```

Returns

[Task](#)

Class Else

Namespace: [ASE](#)

Assembly: ASE.dll

This class doesn't handle what happens when an if condition is false, rather it skips ahead to "end if" when the else keyword is encountered after completing the contents of an "if"

```
public class Else : Command, ICommand
```

Inheritance

[object](#)  ← [Command](#)  ← Else

Implements

[ICommand](#)

Inherited Members

[Command.ProcessParameters\(string\)](#), [Command.Set\(StoredProgram, string\)](#), [Command.Name](#), [Command.ParameterList](#), [Command.Parameters](#), [Command.Program](#), [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ToString\(\)](#) , [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

Else()

Constructor assigns name for identification in StoredProgram

```
public Else()
```

Properties

ID

ID is used to match the else with it's corresponding if and end

```
public int ID { get; set; }
```

Property Value

[int](#)

Methods

CheckParameters()

Does nothing

```
public override Task CheckParameters()
```

Returns

[Task](#)

Compile()

Assigns ID and removes it from the stack

```
public override Task Compile()
```

Returns

[Task](#)

Execute()

skips program execution to the appropriate end statement

```
public override Task Execute()
```

Returns

[Task](#) 

Exceptions

[CommandException](#)

thrown if end not found

Class End

Namespace: [ASE](#)

Assembly: ASE.dll

handles the end of "if", "while" and "for"

```
public class End : CommandOneParameter, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [CanvasCommand](#) ← [CommandOneParameter](#) ← End

Implements

[ICommand](#)

Inherited Members

[CommandOneParameter.param1](#) , [CommandOneParameter.param1unprocessed](#) ,
[CommandOneParameter.CheckParameters\(\)](#) , [CanvasCommand.canvas](#) , [CanvasCommand.xPos](#) ,
[CanvasCommand.yPos](#) , [CanvasCommand.Canvas](#) , [Command.ProcessParameters\(string\)](#) ,
[Command.Set\(StoredProgram, string\)](#) , [Command.Name](#) , [Command.ParameterList](#) ,
[Command.Parameters](#) , [Command.Program](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  ,
[object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  ,
[object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

End()

Constructor assigns name for identification

```
public End()
```

Properties

ID

ID used for matching to corresponding control flow command

```
public int ID { get; set; }
```

Property Value

[int](#)

Methods

Compile()

assigns ID and removes it from the stack

```
public override Task Compile()
```

Returns

[Task](#)

Execute()

Dependant on corresponding command, does nothing for "if", returns to the start of the loop for loops

```
public override Task Execute()
```

Returns

[Task](#)

Exceptions

[CommandException](#)

thrown when syntax errors are encountered

Class Evaluation

Namespace: [ASE](#)

Assembly: ASE.dll

An evaluation encompasses all things that can have a value, such as variables, expressions and conditions used in loops and ifs.

```
public class Evaluation : Command, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← Evaluation

Implements

[ICommand](#)

Derived

[Array](#), [Int](#), [Real](#)

Inherited Members

[Command.ProcessParameters\(string\)](#), [Command.Set\(StoredProgram, string\)](#), [Command.Name](#), [Command.ParameterList](#), [Command.Parameters](#), [Command.Program](#), [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ToString\(\)](#) , [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

Evaluation()

Constructor assigns name

```
public Evaluation()
```

Fields

evaluatedExpression

`protected string` evaluatedExpression

Field Value

[string](#)

expression

`protected string` expression

Field Value

[string](#)

value

`protected int` value

Field Value

[int](#)

varName

`protected string` varName

Field Value

[string](#)

Properties

Expression

The right hand side of an expression in a variable initialisation or variable change of value, or a boolean in a conditional command

```
public string Expression { get; set; }
```

Property Value

[string](#)

Value

Value of variable once the expression has been calculated. Virtual so that variables of other types can override it.

```
public double Value { get; set; }
```

Property Value

[double](#)

VarName

Name of variable as a string (as opposed to Name which is the name of the class, such as Int, Real or Boolean).

```
public string VarName { get; set; }
```

Property Value

[string](#)

scopeID

ID used for identifying the scope in which the variable exists

```
public int scopeID { get; set; }
```

Property Value

[int](#)

Methods

CheckParameters()

Does nothing in this case.

```
public override Task CheckParameters()
```

Returns

[Task](#)

Compile()

assigns name and scopeID and adds to variable list in StoredProgram

```
public override Task Compile()
```

Returns

[Task](#)

Evaluate(string, StoredProgram)

Evaluates a mathematical expression, static for use in other unrelated commands

```
public static Task<double> Evaluate(string expression, StoredProgram storedProgram)
```

Parameters

expression [string](#)

The expression to be evaluated

storedProgram [StoredProgram](#)

The program in which to look for variables

Returns

[Task](#) <[double](#)>

Exceptions

[CommandException](#)

Thrown if a variable isn't found

Execute()

Calculates the right hand side of the expression and assigns it to Value

```
public override Task Execute()
```

Returns

[Task](#)

Exceptions

[CommandException](#)

Thrown if a variable in the expression isn't found

Reassign(string, StoredProgram)

Assigns a new value to a variable

```
public static Task Reassign(string expression, StoredProgram storedProgram)
```

Parameters

expression [string](#)

The expression containing the new value

storedProgram [StoredProgram](#)

Where to look for variables

Returns

[Task](#) 

Exceptions

[CommandException](#)

Thrown if variable is not found

Class FactoryException

Namespace: [ASE](#)

Assembly: ASE.dll

Exception thrown by CommandFactory

```
public class FactoryException : BOOSEException, ISerializable
```

Inheritance

[object](#)  ← [Exception](#)  ← [BOOSEException](#) ← FactoryException

Implements

[ISerializable](#) 

Inherited Members

[Exception.GetBaseException\(\)](#)  , [Exception.ToString\(\)](#)  , [Exception.GetType\(\)](#)  ,
[Exception.TargetSite](#)  , [Exception.Message](#)  , [Exception.Data](#)  , [Exception.InnerException](#)  ,
[Exception.HelpLink](#)  , [Exception.Source](#)  , [Exception.HResult](#)  , [Exception.StackTrace](#)  ,
[Exception.SerializeObjectState](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

FactoryException()

Exception thrown when an invalid command type is passed

```
public FactoryException()
```

Class For

Namespace: [ASE](#)

Assembly: ASE.dll

A for loop, implemented as an if, while and reassign

```
public class For : CommandThreeParameters, ICommand
```

Inheritance

[object](#)  < [Command](#) < [CanvasCommand](#) < [CommandOneParameter](#) < [CommandTwoParameters](#) < [CommandThreeParameters](#) < For

Implements

[ICommand](#)

Inherited Members

[CommandThreeParameters.param3](#) , [CommandThreeParameters.param3unprocessed](#) , [CommandThreeParameters.CheckParameters\(\)](#) , [CommandTwoParameters.param2](#) , [CommandTwoParameters.param2unprocessed](#) , [CommandOneParameter.param1](#) , [CommandOneParameter.param1unprocessed](#) , [CanvasCommand.canvas](#) , [CanvasCommand.xPos](#) , [CanvasCommand.yPos](#) , [CanvasCommand.Canvas](#) , [Command.ProcessParameters\(string\)](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Program](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

For()

Constructor assigns name

```
public For()
```

Fields

ID

Matches to corresponding end statement

```
public int ID
```

Field Value

[int](#)

condition

The condition for stopping

```
public While condition
```

Field Value

[While](#)

counter

Counts the number of iterations

```
public Int counter
```

Field Value

[Int](#)

firstRun

Is this the first iteration?

```
public bool firstRun
```

Field Value

[bool](#)

increment

increments the counter

```
public Reassign increment
```

Field Value

[Reassign](#)

Methods

Compile()

Identifies all parameters then compiles "if", "when" and "reassign"

```
public override Task Compile()
```

Returns

[Task](#)

Execute()

Executes the 3 component commands

```
public override Task Execute()
```

Returns

[Task](#)

Interface ICanvas

Namespace: [ASE](#)

Assembly: ASE.dll

The interface to be implemented by the Canvas class

```
public interface ICanvas
```

Properties

Component

```
BECanvasComponent Component { get; set; }
```

Property Value

[BECanvasComponent](#)

Context

```
Canvas2DContext Context { get; set; }
```

Property Value

[Canvas2DContext](#)

XPos

```
double XPos { get; set; }
```

Property Value

[double](#)

YPos

```
double YPos { get; set; }
```

Property Value

[double](#)

Methods

Circle(double, bool)

```
Task Circle(double radius, bool filled)
```

Parameters

radius [double](#)

filled [bool](#)

Returns

[Task](#)

Clear()

```
Task Clear()
```

Returns

[Task](#)

DrawTo(double, double)

Task `DrawTo(double x, double y)`

Parameters

x [double](#)

y [double](#)

Returns

[Task](#)

GetBitmap()

`object GetBitmap()`

Returns

[object](#)

MoveTo(double, double)

Task `MoveTo(double x, double y)`

Parameters

x [double](#)

y [double](#)

Returns

[Task](#)

Rect(double, double, bool)

Task `Rect(double width, double height, bool filled)`

Parameters

`width` [double](#)

`height` [double](#)

`filled` [bool](#)

Returns

[Task](#)

Reset()

Task `Reset()`

Returns

[Task](#)

Set(double, double)

Task `Set(double width, double height)`

Parameters

`width` [double](#)

`height` [double](#)

Returns

[Task](#)

SetColour(double, double, double)

Task `SetColour(double red, double green, double blue)`

Parameters

`red` [double](#)

`green` [double](#)

`blue` [double](#)

Returns

[Task](#)

Tri(double, double)

Task `Tri(double width, double height)`

Parameters

`width` [double](#)

`height` [double](#)

Returns

[Task](#)

WriteText(string)

Task `WriteText(string text)`

Parameters

text [string](#)

Returns

[Task](#)

Interface ICommand

Namespace: [ASE](#)

Assembly: ASE.dll

The interface to be implemented by the Command class

```
public interface ICommand
```

Methods

CheckParameters()

Task `CheckParameters()`

Returns

[Task](#)

Compile()

Task `Compile()`

Returns

[Task](#)

Execute()

Task `Execute()`

Returns

[Task](#)

Set(StoredProgram, string)

Task **Set**(StoredProgram Program, [string](#) Params)

Parameters

Program [StoredProgram](#)

Params [string](#)

Returns

[Task](#)

Interface ICommandFactory

Namespace: [ASE](#)

Assembly: ASE.dll

The interface to be implemented by the CommandFactory class

```
public interface ICommandFactory
```

Methods

MakeCommand(string, StoredProgram)

```
Task<ICommand> MakeCommand(string commandType, StoredProgram Program)
```

Parameters

commandType [string](#) 

Program [StoredProgram](#)

Returns

[Task](#)  [ICommand](#)

Interface IParser

Namespace: [ASE](#)

Assembly: ASE.dll

The interface to be implemented by the Parser class

```
public interface IParser
```

Methods

ParseCommand(string)

```
Task<ICommand> ParseCommand(string Line)
```

Parameters

Line [string](#) 

Returns

[Task](#)  <[ICommand](#)>

ParseProgram(string)

```
Task ParseProgram(string Program)
```

Parameters

Program [string](#) 

Returns

[Task](#) 

Interface IStoredProgram

Namespace: [ASE](#)

Assembly: ASE.dll

```
public interface IStoredProgram
```

Properties

PC

```
int PC { get; set; }
```

Property Value

[int](#)

Methods

AddCommand(Command)

```
Task<int> AddCommand(Command c)
```

Parameters

c [Command](#)

Returns

[Task](#) <[int](#) >

AddVariable(Evaluation)

Task `AddVariable`(Evaluation Variable)

Parameters

Variable [Evaluation](#)

Returns

[Task](#)

Run()

Task `Run()`

Returns

[Task](#)

Class If

Namespace: [ASE](#)

Assembly: ASE.dll

The If statement

```
public class If : CommandThreeParameters, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [CanvasCommand](#) ← [CommandOneParameter](#) ← [CommandTwoParameters](#) ← [CommandThreeParameters](#) ← If

Implements

[ICommand](#)

Derived

[While](#)

Inherited Members

[CommandThreeParameters.param3](#) , [CommandThreeParameters.param3unprocessed](#) , [CommandThreeParameters.CheckParameters\(\)](#) , [CommandTwoParameters.param2](#) , [CommandTwoParameters.param2unprocessed](#) , [CommandOneParameter.param1](#) , [CommandOneParameter.param1unprocessed](#) , [CanvasCommand.canvas](#) , [CanvasCommand.xPos](#) , [CanvasCommand.yPos](#) , [CanvasCommand.Canvas](#) , [Command.ProcessParameters\(string\)](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Program](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

If()

Constructor assigns name

```
public If()
```

Properties

ID

Matches to corresponding else and end

```
public int ID { get; set; }
```

Property Value

[int](#)

Methods

Compile()

assigns ID and adds IDs to the else and end stack

```
public override Task Compile()
```

Returns

[Task](#)

Condition()

Evaluated if condition is true or not

```
public Task<bool> Condition()
```

Returns

[Task](#)<[bool](#)>

true or false

Exceptions

[CommandException](#)

Thorwn if an invalid boolean operator is used

Execute()

Depending on condition will either continue execution or skip to the else or end if an else isn't found

```
public override Task Execute()
```

Returns

[Task](#)

Exceptions

[CommandException](#)

Thrown if no else or end is found

Class Int

Namespace: [ASE](#)

Assembly: ASE.dll

Stores an integer

```
public class Int : Evaluation, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [Evaluation](#) ← Int

Implements

[ICommand](#)

Inherited Members

[Evaluation.evaluatedExpression](#) , [Evaluation.expression](#) , [Evaluation.value](#) , [Evaluation.varName](#) , [Evaluation.CheckParameters\(\)](#) , [Evaluation.Compile\(\)](#) , [Evaluation.Evaluate\(string, StoredProgram\)](#) , [Evaluation.Reassign\(string, StoredProgram\)](#) , [Evaluation.Expression](#) , [Evaluation.Value](#) , [Evaluation.VarName](#) , [Evaluation.scopeID](#) , [Command.ProcessParameters\(string\)](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Program](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Int()

constructor assigns name

```
public Int()
```

Methods

Execute()

Uses inherited execute then casts its own value to an integer

```
public override Task Execute()
```

Returns

[Task](#)

Class MoveTo

Namespace: [ASE](#)

Assembly: ASE.dll

Command for moving pen to specified position on canvas

```
public class MoveTo : CommandTwoParameters, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [CanvasCommand](#) ← [CommandOneParameter](#) ← [CommandTwoParameters](#) ← MoveTo

Implements

[ICommand](#)

Inherited Members

[CommandTwoParameters.param2](#) , [CommandTwoParameters.param2unprocessed](#) , [CommandTwoParameters.CheckParameters\(\)](#) , [CommandOneParameter.param1](#) , [CommandOneParameter.param1unprocessed](#) , [CanvasCommand.canvas](#) , [CanvasCommand.xPos](#) , [CanvasCommand.yPos](#) , [CanvasCommand.Canvas](#) , [Command.Compile\(\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Program](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

MoveTo()

Constructor for factory instantiation

```
public MoveTo()
```

Methods

Execute()

Moves the pen

```
public override Task Execute()
```

Returns

[Task](#)

Class Parser

Namespace: [ASE](#)

Assembly: ASE.dll

The class used for reading a BOOSE program

```
public class Parser : IParser
```

Inheritance

[object](#)  ← Parser

Implements

[IParser](#)

Inherited Members

[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Parser(CommandFactory, StoredProgram)

```
public Parser(CommandFactory Factory, StoredProgram Program)
```

Parameters

Factory [CommandFactory](#)

Program [StoredProgram](#)

Fields

lines

Constructor that sets properties to the provided instances

```
public string[] lines
```

Field Value

[string](#)  []

Methods

ParseCommand(string)

Identifies the type of command, and passes it to the factory for instantiation

```
public virtual Task<ICommand> ParseCommand(string Line)
```

Parameters

Line [string](#) 

The line containing the command and it's parameters

Returns

[Task](#)  [ICommand](#) 

The new command that has been created

ParseProgram(string)

Splits the program into lines and passes them to ParseCommand, adds result into StoredProgram instance

```
public virtual Task ParseProgram(string program)
```

Parameters

program [string](#) 

The string containing the program to be run

Returns

[Task](#)

Class ParseException

Namespace: [ASE](#)

Assembly: ASE.dll

Exception thrown by Parser class

```
public class ParseException : BOOSEException, ISerializable
```

Inheritance

[object](#)  ← [Exception](#)  ← [BOOSEException](#) ← ParseException

Implements

[ISerializable](#) 

Inherited Members

[Exception.GetBaseException\(\)](#)  , [Exception.ToString\(\)](#)  , [Exception.GetType\(\)](#)  ,
[Exception.TargetSite](#)  , [Exception.Message](#)  , [Exception.Data](#)  , [Exception.InnerException](#)  ,
[Exception.HelpLink](#)  , [Exception.Source](#)  , [Exception.HResult](#)  , [Exception.StackTrace](#)  ,
[Exception.SerializeObjectState](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

ParseException()

Exception thrown when incorrect syntax is found

```
public ParseException()
```

Class Peek

Namespace: [ASE](#)

Assembly: ASE.dll

Used to access values stored in an array

```
public class Peek : CommandThreeParameters, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [CanvasCommand](#) ← [CommandOneParameter](#) ← [CommandTwoParameters](#) ← [CommandThreeParameters](#) ← Peek

Implements

[ICommand](#)

Inherited Members

[CommandThreeParameters.param3](#) , [CommandThreeParameters.param3unprocessed](#) , [CommandThreeParameters.CheckParameters\(\)](#) , [CommandTwoParameters.param2](#) , [CommandTwoParameters.param2unprocessed](#) , [CommandOneParameter.param1](#) , [CommandOneParameter.param1unprocessed](#) , [CanvasCommand.canvas](#) , [CanvasCommand.xPos](#) , [CanvasCommand.yPos](#) , [CanvasCommand.Canvas](#) , [Command.ProcessParameters\(string\)](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Program](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Peek()

constructor assigns name

```
public Peek()
```

Methods

Compile()

Just a call to ProcessParameters

```
public override Task Compile()
```

Returns

[Task](#)

Execute()

Finds the value of the specified index of an array and assigns it to a variable

```
public override Task Execute()
```

Returns

[Task](#)

Exceptions

[CommandException](#)

Thrown by various syntax errors

ProcessParameters()

Seperates all the parameters

```
public Task<int> ProcessParameters()
```

Returns

[Task](#) <[int](#)>

The numebr of parameters

Class PenColour

Namespace: [ASE](#)

Assembly: ASE.dll

Command for altering the pen colour using rgb values

```
public class PenColour : CommandThreeParameters, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [CanvasCommand](#) ← [CommandOneParameter](#) ← [CommandTwoParameters](#) ← [CommandThreeParameters](#) ← PenColour

Implements

[ICommand](#)

Inherited Members

[CommandThreeParameters.param3](#) , [CommandThreeParameters.param3unprocessed](#) , [CommandThreeParameters.CheckParameters\(\)](#) , [CommandTwoParameters.param2](#) , [CommandTwoParameters.param2unprocessed](#) , [CommandOneParameter.param1](#) , [CommandOneParameter.param1unprocessed](#) , [CanvasCommand.canvas](#) , [CanvasCommand.xPos](#) , [CanvasCommand.yPos](#) , [CanvasCommand.Canvas](#) , [Command.Compile\(\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Program](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

PenColour()

Constructor for factory instantiation

```
public PenColour()
```

Methods

Execute()

Changes the pen colour

```
public override Task Execute()
```

Returns

[Task](#)

Class Poke

Namespace: [ASE](#)

Assembly: ASE.dll

Used to input data into an array

```
public class Poke : CommandThreeParameters, ICommand
```

Inheritance

[object](#)  < [Command](#) < [CanvasCommand](#) < [CommandOneParameter](#) < [CommandTwoParameters](#) < [CommandThreeParameters](#) < Poke

Implements

[ICommand](#)

Inherited Members

[CommandThreeParameters.param3](#) , [CommandThreeParameters.param3unprocessed](#) , [CommandThreeParameters.CheckParameters\(\)](#) , [CommandTwoParameters.param2](#) , [CommandTwoParameters.param2unprocessed](#) , [CommandOneParameter.param1](#) , [CommandOneParameter.param1unprocessed](#) , [CanvasCommand.canvas](#) , [CanvasCommand.xPos](#) , [CanvasCommand.yPos](#) , [CanvasCommand.Canvas](#) , [Command.ProcessParameters\(string\)](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Program](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Poke()

Constructor assigns name

```
public Poke()
```

Methods

Compile()

Just a call to ProcessParameters

```
public override Task Compile()
```

Returns

[Task](#)

Execute()

Inputs the specified value into an array

```
public override Task Execute()
```

Returns

[Task](#)

Exceptions

[CommandException](#)

Thrown due to various syntax errors

ProcessParameters()

Seperates all the parameters

```
public Task<int> ProcessParameters()
```

Returns

[Task](#) <[int](#)>

The numebr of parameters

Class Program

Namespace: [ASE](#)

Assembly: ASE.dll

Default blazor template code

```
public class Program
```

Inheritance

[object](#)  ← Program

Inherited Members

[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Program()

```
public Program()
```

Methods

Main(string[])

```
public static void Main(string[] args)
```

Parameters

args [string](#)  []

Class Real

Namespace: [ASE](#)

Assembly: ASE.dll

Stores a floating point number

```
public class Real : Evaluation, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [Evaluation](#) ← Real

Implements

[ICommand](#)

Inherited Members

[Evaluation.evaluatedExpression](#) , [Evaluation.expression](#) , [Evaluation.value](#) , [Evaluation.varName](#) , [Evaluation.CheckParameters\(\)](#) , [Evaluation.Compile\(\)](#) , [Evaluation.Execute\(\)](#) , [Evaluation.Evaluate\(string, StoredProgram\)](#) , [Evaluation.Reassign\(string, StoredProgram\)](#) , [Evaluation.Expression](#) , [Evaluation.Value](#) , [Evaluation.VarName](#) , [Evaluation.scopeID](#) , [Command.ProcessParameters\(string\)](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Program](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Real()

Constructor assigns name

```
public Real()
```

Class Reassign

Namespace: [ASE](#)

Assembly: ASE.dll

Assigns a new value to a variable

```
public class Reassign : CommandOneParameter, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [CanvasCommand](#) ← [CommandOneParameter](#) ← Reassign

Implements

[ICommand](#)

Inherited Members

[CommandOneParameter.param1](#) , [CommandOneParameter.param1unprocessed](#) , [CommandOneParameter.CheckParameters\(\)](#) , [CanvasCommand.canvas](#) , [CanvasCommand.xPos](#) , [CanvasCommand.yPos](#) , [CanvasCommand.Canvas](#) , [Command.Compile\(\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Program](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Reassign(string)

Constructor assigns a name and the name of the target variable to reassign

```
public Reassign(string target)
```

Parameters

target [string](#) 

The name of the target variable

Fields

targetName

The name of the variable to assign to

```
public string targetName
```

Field Value

[string](#)

Methods

Execute()

Assigns new value to specified variable

```
public override Task Execute()
```

Returns

[Task](#)

Exceptions

[CommandException](#)

Thrown if a variable isn't found

Class Rect

Namespace: [ASE](#)

Assembly: ASE.dll

Command for drawing rectangles at current cursor position

```
public class Rect : CommandTwoParameters, ICommand
```

Inheritance

[object](#)  $\leftarrow$ [Command](#) $\leftarrow$ [CanvasCommand](#) $\leftarrow$ [CommandOneParameter](#) $\leftarrow$ [CommandTwoParameters](#) $\leftarrow$ Rect

Implements

[ICommand](#)

Inherited Members

[CommandTwoParameters.param2](#) , [CommandTwoParameters.param2unprocessed](#) , [CommandTwoParameters.CheckParameters\(\)](#) , [CommandOneParameter.param1](#) , [CommandOneParameter.param1unprocessed](#) , [CanvasCommand.canvas](#) , [CanvasCommand.xPos](#) , [CanvasCommand.yPos](#) , [CanvasCommand.Canvas](#) , [Command.Compile\(\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Program](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Rect()

Constructor for factory instantiation

```
public Rect()
```

Methods

Execute()

Draws rectangle

```
public override Task Execute()
```

Returns

[Task](#)

Class StoredProgram

Namespace: [ASE](#)

Assembly: ASE.dll

Class for storing command objects

```
public class StoredProgram : ArrayList, IList, ICollection, IEnumerable,  
    ICloneable, IStoredProgram
```

Inheritance

[object](#) ← [ArrayList](#) ← StoredProgram

Implements

[IList](#), [ICollection](#), [IEnumerable](#), [ICloneable](#), [IStoredProgram](#)

Inherited Members

[ArrayList.Adapter\(IList\)](#), [ArrayList.Add\(object\)](#), [ArrayList.AddRange\(ICollection\)](#), [ArrayList.BinarySearch\(int, int, object, IComparer\)](#), [ArrayList.BinarySearch\(object\)](#), [ArrayList.BinarySearch\(object, IComparer\)](#), [ArrayList.Clear\(\)](#), [ArrayList.Clone\(\)](#), [ArrayList.Contains\(object\)](#), [ArrayList.CopyTo\(Array\)](#), [ArrayList.CopyTo\(Array, int\)](#), [ArrayList.CopyTo\(int, Array, int, int\)](#), [ArrayList.FixedSize\(IList\)](#), [ArrayList.FixedSize\(ArrayList\)](#), [ArrayList.GetEnumerator\(\)](#), [ArrayList.GetEnumerator\(int, int\)](#), [ArrayList.IndexOf\(object\)](#), [ArrayList.IndexOf\(object, int\)](#), [ArrayList.IndexOf\(object, int, int\)](#), [ArrayList.Insert\(int, object\)](#), [ArrayList.InsertRange\(int, ICollection\)](#), [ArrayList.LastIndexOf\(object\)](#), [ArrayList.LastIndexOf\(object, int\)](#), [ArrayList.LastIndexOf\(object, int, int\)](#), [ArrayList.ReadOnly\(IList\)](#), [ArrayList.ReadOnly\(ArrayList\)](#), [ArrayList.Remove\(object\)](#), [ArrayList.RemoveAt\(int\)](#), [ArrayList.RemoveRange\(int, int\)](#), [ArrayList.Repeat\(object, int\)](#), [ArrayList.Reverse\(\)](#), [ArrayList.Reverse\(int, int\)](#), [ArrayList.SetRange\(int, ICollection\)](#), [ArrayList.GetRange\(int, int\)](#), [ArrayList.Sort\(\)](#), [ArrayList.Sort\(IComparer\)](#), [ArrayList.Sort\(int, int, IComparer\)](#), [ArrayList.Synchronized\(IList\)](#), [ArrayList.Synchronized\(ArrayList\)](#), [ArrayList.ToArray\(\)](#), [ArrayList.ToArray\(Type\)](#), [ArrayList.TrimToSize\(\)](#), [ArrayList.Capacity](#), [ArrayList.Count](#), [ArrayList.IsFixedSize](#), [ArrayList.IsReadOnly](#), [ArrayList.IsSynchronized](#), [ArrayList.SyncRoot](#), [ArrayList.this\[int\]](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ToString\(\)](#), [object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.ReferenceEquals\(object, object\)](#), [object.GetHashCode\(\)](#)

Constructors

StoredProgram(ICanvas)

Constructor initialises PC and canvas

```
public StoredProgram(ICanvas canvas)
```

Parameters

canvas [ICanvas](#)

The canvas to draw to

Fields

IDCounter

The next assignable ID

```
public int IDCounter
```

Field Value

[int](#)

canvas

The canvas that the commands should draw to

```
public ICanvas canvas
```

Field Value

[ICanvas](#)

elseID

Stack of ID's for else statements

```
public ArrayList elseID
```

Field Value

[ArrayList](#)

endID

Stack of ID's for end statements

```
public ArrayList endID
```

Field Value

[ArrayList](#)

Properties

PC

The counter for the currently run command

```
public int PC { get; set; }
```

Property Value

[int](#)

Variables

Array list for storing variables in a program

```
public ArrayList Variables { get; set; }
```

Property Value

[ArrayList](#)

scopeID

The ID of the current scope

```
public ArrayList scopeID { get; set; }
```

Property Value

[ArrayList](#)

Methods

AddCommand(Command)

Adds command to collection

```
public Task<int> AddCommand(Command c)
```

Parameters

c [Command](#)

The command to be added

Returns

[Task](#) <[int](#)>

The current number of commands

AddVariable(Evaluation)

Adds variable to the program

```
public virtual Task AddVariable(Evaluation Variable)
```

Parameters

Variable [Evaluation](#)

The variable to be added

Returns

[Task](#)

DeleteVariable(string)

Removes a variable from the program

```
public virtual Task DeleteVariable(string varName)
```

Parameters

varName [string](#)

The name of the variable to be deleted

Returns

[Task](#)

FindVariable(int)

Finds variable by index

```
public Task<Evaluation> FindVariable(int varIndex)
```

Parameters

varIndex [int](#)

The index of the variable

Returns

[Task](#) <[Evaluation](#)>

The found variable

FindVariable(string)

Finds the index of a variable by name

```
public virtual Task<int> FindVariable(string varName)
```

Parameters

varName [string](#)

Name of variable to find

Returns

[Task](#) <[int](#)>

The index of the variable if found, -1 if not

Run()

Runs all the commands stored in the program

```
public virtual Task Run()
```

Returns

[Task](#)

Class StoredProgramException

Namespace: [ASE](#)

Assembly: ASE.dll

Exception thrown by StoredProgram class

```
public class StoredProgramException : BOOSEException, ISerializable
```

Inheritance

[object](#)  ← [Exception](#)  ← [BOOSEException](#)  ← StoredProgramException

Implements

[ISerializable](#) 

Inherited Members

[Exception.GetBaseException\(\)](#)  , [Exception.ToString\(\)](#)  , [Exception.GetType\(\)](#)  ,
[Exception.TargetSite](#)  , [Exception.Message](#)  , [Exception.Data](#)  , [Exception.InnerException](#)  ,
[Exception.HelpLink](#)  , [Exception.Source](#)  , [Exception.HResult](#)  , [Exception.StackTrace](#)  ,
[Exception.SerializeObjectState](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

StoredProgramException()

Currently unused

```
public StoredProgramException()
```

Class While

Namespace: [ASE](#)

Assembly: ASE.dll

The while loop

```
public class While : If, ICommand
```

Inheritance

[object](#)  < [Command](#) < [CanvasCommand](#) < [CommandOneParameter](#) < [CommandTwoParameters](#) < [CommandThreeParameters](#) < [If](#) < While

Implements

[ICommand](#)

Inherited Members

[If.Condition\(\)](#), [If.ID](#), [CommandThreeParameters.param3](#), [CommandThreeParameters.param3unprocessed](#), [CommandThreeParameters.CheckParameters\(\)](#), [CommandTwoParameters.param2](#), [CommandTwoParameters.param2unprocessed](#), [CommandOneParameter.param1](#), [CommandOneParameter.param1unprocessed](#), [CanvasCommand.canvas](#), [CanvasCommand.xPos](#), [CanvasCommand.yPos](#), [CanvasCommand.Canvas](#), [Command.ProcessParameters\(string\)](#), [Command.Set\(StoredProgram, string\)](#), [Command.Name](#), [Command.ParameterList](#), [Command.Parameters](#), [Command.Program](#), [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ToString\(\)](#) , [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

While()

Constructor assigns name

```
public While()
```


Methods

Compile()

Assigns ID and adds it to the endID stack

```
public override Task Compile()
```

Returns

[Task](#)

Execute()

Evaluates the condition, if false skips to end, otherwise continues execution as normal

```
public override Task Execute()
```

Returns

[Task](#)

Exceptions

[CommandException](#)

Thrown if end not found

Class Write

Namespace: [ASE](#)

Assembly: ASE.dll

Command for writing to the canvas

```
public class Write : CommandOneParameter, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [CanvasCommand](#) ← [CommandOneParameter](#) ← Write

Implements

[ICommand](#)

Inherited Members

[CommandOneParameter.param1](#) , [CommandOneParameter.param1unprocessed](#) ,
[CanvasCommand.canvas](#) , [CanvasCommand.xPos](#) , [CanvasCommand.yPos](#) ,
[CanvasCommand.Canvas](#) , [Command.Compile\(\)](#) , [Command.ProcessParameters\(string\)](#) ,
[Command.Set\(StoredProgram, string\)](#) , [Command.Name](#) , [Command.ParameterList](#) ,
[Command.Parameters](#) , [Command.Program](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  ,
[object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  ,
[object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Write()

Constructor for factory instantiation

```
public Write()
```

Methods

CheckParameters()

Special version of check parameters that converts them to strings

```
public override Task CheckParameters()
```

Returns

[Task](#)

Execute()

writes message to canvas

```
public override Task Execute()
```

Returns

[Task](#)

Namespace ASE.Components

Classes

[App](#)

[Routes](#)

[_Imports](#)

Class App

Namespace: [ASE.Components](#)

Assembly: ASE.dll

```
public class App : ComponentBase, IComponent, IHandleEvent, IHandleAfterRender
```

Inheritance

[object](#)  ← [ComponentBase](#)  ← App

Implements

[IComponent](#) , [IHandleEvent](#) , [IHandleAfterRender](#) 

Inherited Members

[ComponentBase.OnInitialized\(\)](#) , [ComponentBase.OnInitializedAsync\(\)](#) ,
[ComponentBase.OnParametersSet\(\)](#) , [ComponentBase.OnParametersSetAsync\(\)](#) ,
[ComponentBase.StateHasChanged\(\)](#) , [ComponentBase.ShouldRender\(\)](#) ,
[ComponentBase.OnAfterRender\(bool\)](#) , [ComponentBase.OnAfterRenderAsync\(bool\)](#) ,
[ComponentBase.InvokeAsync\(Action\)](#) , [ComponentBase.InvokeAsync\(Func<Task>\)](#) ,
[ComponentBase.DispatchExceptionAsync\(Exception\)](#) ,
[ComponentBase.SetParametersAsync\(ParameterView\)](#) , [ComponentBase.RendererInfo](#) ,
[ComponentBase.Assets](#) , [ComponentBase.AssignedRenderMode](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ToString\(\)](#) , [object.Equals\(object\)](#) ,
[object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

App()

```
public App()
```

Methods

BuildRenderTree(RenderTreeBuilder)

```
protected override void BuildRenderTree(RenderTreeBuilder __builder)
```

Parameters

`__builder` [RenderTreeBuilder](#) 

Class Routes

Namespace: [ASE.Components](#)

Assembly: ASE.dll

```
public class Routes : ComponentBase, IComponent, IHandleEvent, IHandleAfterRender
```

Inheritance

[object](#) ← [ComponentBase](#) ← Routes

Implements

[IComponent](#), [IHandleEvent](#), [IHandleAfterRender](#)

Inherited Members

[ComponentBase.OnInitialized\(\)](#), [ComponentBase.OnInitializedAsync\(\)](#),
[ComponentBase.OnParametersSet\(\)](#), [ComponentBase.OnParametersSetAsync\(\)](#),
[ComponentBase.StateHasChanged\(\)](#), [ComponentBase.ShouldRender\(\)](#),
[ComponentBase.OnAfterRender\(bool\)](#), [ComponentBase.OnAfterRenderAsync\(bool\)](#),
[ComponentBase.InvokeAsync\(Action\)](#), [ComponentBase.InvokeAsync\(Func<Task>\)](#),
[ComponentBase.DispatchExceptionAsync\(Exception\)](#),
[ComponentBase.SetParametersAsync\(ParameterView\)](#), [ComponentBase.RendererInfo](#),
[ComponentBase.Assets](#), [ComponentBase.AssignedRenderMode](#), [object.GetType\(\)](#),
[object.MemberwiseClone\(\)](#), [object.ToString\(\)](#), [object.Equals\(object\)](#),
[object.Equals\(object, object\)](#), [object.ReferenceEquals\(object, object\)](#), [object.GetHashCode\(\)](#)

Constructors

Routes()

```
public Routes()
```

Methods

BuildRenderTree(RenderTreeBuilder)

```
protected override void BuildRenderTree(RenderTreeBuilder __builder)
```

Parameters

`__builder` [RenderTreeBuilder](#)

Class _Imports

Namespace: [ASE.Components](#)

Assembly: ASE.dll

```
public class _Imports : ComponentBase, IComponent, IHandleEvent, IHandleAfterRender
```

Inheritance

[object](#)  ← [ComponentBase](#)  ← [_Imports](#)

Implements

[IComponent](#) , [IHandleEvent](#) , [IHandleAfterRender](#) 

Inherited Members

[ComponentBase.OnInitialized\(\)](#) , [ComponentBase.OnInitializedAsync\(\)](#) ,
[ComponentBase.OnParametersSet\(\)](#) , [ComponentBase.OnParametersSetAsync\(\)](#) ,
[ComponentBase.StateHasChanged\(\)](#) , [ComponentBase.ShouldRender\(\)](#) ,
[ComponentBase.OnAfterRender\(bool\)](#) , [ComponentBase.OnAfterRenderAsync\(bool\)](#) ,
[ComponentBase.InvokeAsync\(Action\)](#) , [ComponentBase.InvokeAsync\(Func<Task>\)](#) ,
[ComponentBase.DispatchExceptionAsync\(Exception\)](#) ,
[ComponentBase.SetParametersAsync\(ParameterView\)](#) , [ComponentBase.RendererInfo](#) ,
[ComponentBase.Assets](#) , [ComponentBase.AssignedRenderMode](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ToString\(\)](#) , [object.Equals\(object\)](#) ,
[object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

_Imports()

```
public _Imports()
```

Methods

BuildRenderTree(RenderTreeBuilder)

```
protected override void BuildRenderTree(RenderTreeBuilder __builder)
```

Parameters

`__builder` [RenderTreeBuilder](#)

Namespace ASE.Components.Layout

Classes

[MainLayout](#)

[NavMenu](#)

Class MainLayout

Namespace: [ASE.Components.Layout](#)

Assembly: ASE.dll

```
public class MainLayout : LayoutComponentBase, IComponent,
    IHandleEvent, IHandleAfterRender
```

Inheritance

[object](#)  ← [ComponentBase](#)  ← [LayoutComponentBase](#)  ← MainLayout

Implements

[IComponent](#) , [IHandleEvent](#) , [IHandleAfterRender](#) 

Inherited Members

[LayoutComponentBase.SetParametersAsync\(ParameterView\)](#) , [LayoutComponentBase.Body](#) , [ComponentBase.OnInitialized\(\)](#) , [ComponentBase.OnInitializedAsync\(\)](#) , [ComponentBase.OnParametersSet\(\)](#) , [ComponentBase.OnParametersSetAsync\(\)](#) , [ComponentBase.StateHasChanged\(\)](#) , [ComponentBase.ShouldRender\(\)](#) , [ComponentBase.OnAfterRender\(bool\)](#) , [ComponentBase.OnAfterRenderAsync\(bool\)](#) , [ComponentBase.InvokeAsync\(Action\)](#) , [ComponentBase.InvokeAsync\(Func<Task>\)](#) , [ComponentBase.DispatchExceptionAsync\(Exception\)](#) , [ComponentBase.RendererInfo](#) , [ComponentBase.Assets](#) , [ComponentBase.AssignedRenderMode](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ToString\(\)](#) , [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

MainLayout()

```
public MainLayout()
```

Methods

BuildRenderTree(RenderTreeBuilder)

```
protected override void BuildRenderTree(RenderTreeBuilder __builder)
```

Parameters

`__builder` [RenderTreeBuilder](#)

Class NavMenu

Namespace: [ASE.Components.Layout](#)

Assembly: ASE.dll

```
public class NavMenu : ComponentBase, IComponent, IHandleEvent, IHandleAfterRender
```

Inheritance

[object](#)  ← [ComponentBase](#)  ← NavMenu

Implements

[IComponent](#) , [IHandleEvent](#) , [IHandleAfterRender](#) 

Inherited Members

[ComponentBase.OnInitialized\(\)](#) , [ComponentBase.OnInitializedAsync\(\)](#) ,
[ComponentBase.OnParametersSet\(\)](#) , [ComponentBase.OnParametersSetAsync\(\)](#) ,
[ComponentBase.StateHasChanged\(\)](#) , [ComponentBase.ShouldRender\(\)](#) ,
[ComponentBase.OnAfterRender\(bool\)](#) , [ComponentBase.OnAfterRenderAsync\(bool\)](#) ,
[ComponentBase.InvokeAsync\(Action\)](#) , [ComponentBase.InvokeAsync\(Func<Task>\)](#) ,
[ComponentBase.DispatchExceptionAsync\(Exception\)](#) ,
[ComponentBase.SetParametersAsync\(ParameterView\)](#) , [ComponentBase.RendererInfo](#) ,
[ComponentBase.Assets](#) , [ComponentBase.AssignedRenderMode](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ToString\(\)](#) , [object.Equals\(object\)](#) ,
[object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

NavMenu()

```
public NavMenu()
```

Methods

BuildRenderTree(RenderTreeBuilder)

```
protected override void BuildRenderTree(RenderTreeBuilder __builder)
```

Parameters

`__builder` [RenderTreeBuilder](#) 

Namespace ASE.Components.Pages

Classes

[Error](#)

[Home](#)

Class Error

Namespace: [ASE.Components.Pages](#)

Assembly: ASE.dll

```
[Route("/Error")]  
public class Error : ComponentBase, IComponent, IHandleEvent, IHandleAfterRender
```

Inheritance

[object](#) ← [ComponentBase](#) ← Error

Implements

[IComponent](#), [IHandleEvent](#), [IHandleAfterRender](#)

Inherited Members

[ComponentBase.OnInitializedAsync\(\)](#), [ComponentBase.OnParametersSet\(\)](#),
[ComponentBase.OnParametersSetAsync\(\)](#), [ComponentBase.StateHasChanged\(\)](#),
[ComponentBase.ShouldRender\(\)](#), [ComponentBase.OnAfterRender\(bool\)](#),
[ComponentBase.OnAfterRenderAsync\(bool\)](#), [ComponentBase.InvokeAsync\(Action\)](#),
[ComponentBase.InvokeAsync\(Func<Task>\)](#),
[ComponentBase.DispatchExceptionAsync\(Exception\)](#),
[ComponentBase.SetParametersAsync\(ParameterView\)](#), [ComponentBase.RendererInfo](#),
[ComponentBase.Assets](#), [ComponentBase.AssignedRenderMode](#), [object.GetType\(\)](#),
[object.MemberwiseClone\(\)](#), [object.ToString\(\)](#), [object.Equals\(object\)](#),
[object.Equals\(object, object\)](#), [object.ReferenceEquals\(object, object\)](#), [object.GetHashCode\(\)](#)

Constructors

Error()

```
public Error()
```

Methods

BuildRenderTree(RenderTreeBuilder)

```
protected override void BuildRenderTree(RenderTreeBuilder __builder)
```

Parameters

`__builder` [RenderTreeBuilder](#)

OnInitialized()

```
protected override void OnInitialized()
```

Class Home

Namespace: [ASE.Components.Pages](#)

Assembly: ASE.dll

```
[Route("/")]  
public class Home : ComponentBase, IComponent, IHandleEvent, IHandleAfterRender
```

Inheritance

[object](#)  ← [ComponentBase](#)  ← Home

Implements

[IComponent](#) , [IHandleEvent](#) , [IHandleAfterRender](#) 

Inherited Members

[ComponentBase.OnInitialized\(\)](#) , [ComponentBase.OnInitializedAsync\(\)](#) ,
[ComponentBase.OnParametersSet\(\)](#) , [ComponentBase.OnParametersSetAsync\(\)](#) ,
[ComponentBase.StateHasChanged\(\)](#) , [ComponentBase.ShouldRender\(\)](#) ,
[ComponentBase.OnAfterRender\(bool\)](#) , [ComponentBase.InvokeAsync\(Action\)](#) ,
[ComponentBase.InvokeAsync\(Func<Task>\)](#) ,
[ComponentBase.DispatchExceptionAsync\(Exception\)](#) ,
[ComponentBase.SetParametersAsync\(ParameterView\)](#) , [ComponentBase.RendererInfo](#) ,
[ComponentBase.Assets](#) , [ComponentBase.AssignedRenderMode](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ToString\(\)](#) , [object.Equals\(object\)](#) ,
[object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

Home()

```
public Home()
```

Fields

Input

`public string` Input

Field Value

[string](#)↗

_canvasReference

`public` BECanvasComponent _canvasReference

Field Value

[BECanvasComponent](#)

_context

`public` Canvas2DContext _context

Field Value

[Canvas2DContext](#)

about

`public string` about

Field Value

[string](#)↗

Methods

BuildRenderTree(RenderTreeBuilder)

```
protected override void BuildRenderTree(RenderTreeBuilder __builder)
```

Parameters

`__builder` [RenderTreeBuilder](#) 

OnAfterRenderAsync(bool)

```
protected override Task OnAfterRenderAsync(bool firstRender)
```

Parameters

`firstRender` [bool](#) 

Returns

[Task](#) 

Namespace BOOSE

Classes

[AboutBOOSE](#)

[Array](#)

[BOOSEException](#)

[Boolean](#)

[Call](#)

[Canvas](#)

[CanvasCommand](#)

[CanvasException](#)

[Cast](#)

[Circle](#)

[Command](#)

[CommandException](#)

[CommandFactory](#)

[CommandOneParameter](#)

[CommandThreeParameters](#)

[CommandTwoParameters](#)

[CompoundCommand](#)

[ConditionalCommand](#)

[DrawTo](#)

[Else](#)

[End](#)

[Evaluation](#)

[FactoryException](#)

[For](#)

[If](#)

[Int](#)

[Method](#)

[MoveTo](#)

[Parser](#)

[ParserException](#)

[Peek](#)

[PenColour](#)

[Poke](#)

[Real](#)

[Rect](#)

[RestrictionException](#)

[StoredProgram](#)

[StoredProgramException](#)

[VarException](#)

[While](#)

[Write](#)

Interfaces

[ICanvas](#)

[ICommand](#)

[ICommandFactory](#)

[IEvaluation](#)

[IParser](#)

[IStoredProgram](#)

Enums

[ConditionalCommand.conditionalTypes](#)

Class AboutBOOSE

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public static class AboutBOOSE
```

Inheritance

[object](#)  ← AboutBOOSE

Inherited Members

[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Fields

ARRAYLIMIT

```
public const int ARRAYLIMIT = 2
```

Field Value

[int](#) 

BODYLIMIT

```
public const int BODYLIMIT = 5
```

Field Value

[int](#) 

COMPOUNDCOMMANDLIMIT

```
public const int COMPOUNDCOMMANDLIMIT = 2
```

Field Value

[int](#)

MAXITERATIONS

```
public const int MAXITERATIONS = 50
```

Field Value

[int](#)

MAXPROGRAMSIZE

```
public const int MAXPROGRAMSIZE = 20
```

Field Value

[int](#)

METHODLIMIT

```
public const int METHODLIMIT = 1
```

Field Value

[int](#)

RESTRICTIONS

```
public const bool RESTRICTIONS = true
```

Field Value

[bool](#)

SIZELIMIT

```
public const int SIZELIMIT = 2000
```

Field Value

[int](#)

VARIABLELIMIT

```
public const int VARIABLELIMIT = 5
```

Field Value

[int](#)

Properties

Version

```
public static float Version { get; }
```

Property Value

[float](#)

Methods

about()

```
public static string about()
```

Returns

[string](#) 

Class Array

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class Array : Evaluation, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [Evaluation](#) ← Array

Implements

[ICommand](#)

Derived

[Peek](#), [Poke](#)

Inherited Members

[Evaluation.expression](#) , [Evaluation.evaluatedExpression](#) , [Evaluation.varName](#) , [Evaluation.value](#) , [Evaluation.ProcessExpression\(string\)](#) , [Evaluation.Expression](#) , [Evaluation.VarName](#) , [Evaluation.Value](#) , [Evaluation.Local](#) , [Command.program](#) , [Command.parameterList](#) , [Command.parameters](#) , [Command.paramsint](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.ToString\(\)](#) , [Command.Program](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Paramsint](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Array()

```
public Array()
```

Fields

PEEK

```
protected const bool PEEK = false
```

Field Value

[bool](#)

POKE

```
public const bool POKE = true
```

Field Value

[bool](#)

column

```
protected int column
```

Field Value

[int](#)

columnS

```
protected string columnS
```

Field Value

[string](#)

columns

```
protected int columns
```

Field Value

[int](#)↗

intArray

```
protected int[,] intArray
```

Field Value

[int](#)↗[,]

peekVar

```
protected string peekVar
```

Field Value

[string](#)↗

pokeValue

```
protected string pokeValue
```

Field Value

[string](#)↗

realArray

```
protected double[, ] realArray
```

Field Value

[double](#)[,]

row

```
protected int row
```

Field Value

[int](#)

rowS

```
protected string rowS
```

Field Value

[string](#)

rows

```
protected int rows
```

Field Value

[int](#)

type

`protected string` type

Field Value

[string](#)

valueInt

`protected int` valueInt

Field Value

[int](#)

valueReal

`protected double` valueReal

Field Value

[double](#)

Properties

Columns

`protected int` Columns { `get`; }

Property Value

[int](#)

Rows

```
protected int Rows { get; }
```

Property Value

[int](#)

Methods

ArrayRestrictions()

```
public void ArrayRestrictions()
```

CheckParameters(string[])

```
public override void CheckParameters(string[] parameterList)
```

Parameters

parameterList [string](#)[]

Compile()

```
public override void Compile()
```

Execute()

```
public override void Execute()
```

GetIntArray(int, int)

```
public virtual int GetIntArray(int row, int col)
```

Parameters

row [int](#)

col [int](#)

Returns

[int](#)

GetRealArray(int, int)

```
public virtual double GetRealArray(int row, int col)
```

Parameters

row [int](#)

col [int](#)

Returns

[double](#)

ProcessArrayParametersCompile(bool)

```
protected virtual void ProcessArrayParametersCompile(bool peekOrPoke)
```

Parameters

peekOrPoke [bool](#)

ProcessArrayParametersExecute(bool)

```
protected virtual void ProcessArrayParametersExecute(bool peekOrPoke)
```

Parameters

peekOrPoke [bool](#)

ReduceRestrictionCounter()

```
protected void ReduceRestrictionCounter()
```

SetIntArray(int, int, int)

```
public virtual void SetIntArray(int val, int row, int col)
```

Parameters

val [int](#)

row [int](#)

col [int](#)

SetRealArray(double, int, int)

```
public virtual void SetRealArray(double val, int row, int col)
```

Parameters

val [double](#)

row [int](#)

col [int](#)

Class BOOSEException

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class BOOSEException : Exception, ISerializable
```

Inheritance

[object](#)  ← [Exception](#)  ← BOOSEException

Implements

[ISerializable](#) 

Derived

[CanvasException](#), [CommandException](#), [FactoryException](#), [ParserException](#), [RestrictionException](#), [StoredProgramException](#), [VarException](#)

Inherited Members

[Exception.GetBaseException\(\)](#)  , [Exception.ToString\(\)](#)  , [Exception.GetType\(\)](#)  , [Exception.TargetSite](#)  , [Exception.Message](#)  , [Exception.Data](#)  , [Exception.InnerException](#)  , [Exception.HelpLink](#)  , [Exception.Source](#)  , [Exception.HResult](#)  , [Exception.StackTrace](#)  , [Exception.SerializeObjectState](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

BOOSEException(string)

```
public BOOSEException(string msg)
```

Parameters

msg [string](#) 

Class Boolean

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class Boolean : Evaluation, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [Evaluation](#) ← Boolean

Implements

[ICommand](#)

Derived

[ConditionalCommand](#)

Inherited Members

[Evaluation.expression](#) , [Evaluation.evaluatedExpression](#) , [Evaluation.varName](#) , [Evaluation.value](#) , [Evaluation.CheckParameters\(string\[\]\)](#) , [Evaluation.ProcessExpression\(string\)](#) , [Evaluation.Expression](#) , [Evaluation.VarName](#) , [Evaluation.Value](#) , [Evaluation.Local](#) , [Command.program](#) , [Command.parameterList](#) , [Command.parameters](#) , [Command.paramsint](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.ToString\(\)](#) , [Command.Program](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Paramsint](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Boolean()

```
public Boolean()
```

Properties

BoolValue

```
public bool BoolValue { get; set; }
```

Property Value

[bool](#) 

Methods

Compile()

```
public override void Compile()
```

Execute()

```
public override void Execute()
```

Restrictions()

```
public virtual void Restrictions()
```

Class Call

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class Call : CompoundCommand, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [Evaluation](#) ← [Boolean](#) ← [ConditionalCommand](#) ← [CompoundCommand](#) ←
Call

Implements

[ICommand](#)

Inherited Members

[CompoundCommand.ReduceRestrictions\(\)](#), [CompoundCommand.CheckParameters\(string\[\]\)](#), [CompoundCommand.CorrespondingCommand](#), [ConditionalCommand.endLineNumber](#), [ConditionalCommand.EndLineNumber](#), [ConditionalCommand.Condition](#), [ConditionalCommand.LineNumber](#), [ConditionalCommand.CondType](#), [ConditionalCommand.ReturnLineNumber](#), [Boolean.Restrictions\(\)](#), [Boolean.BoolValue](#), [Evaluation.expression](#), [Evaluation.evaluatedExpression](#), [Evaluation.varName](#), [Evaluation.value](#), [Evaluation.ProcessExpression\(string\)](#), [Evaluation.Expression](#), [Evaluation.VarName](#), [Evaluation.Value](#), [Evaluation.Local](#), [Command.program](#), [Command.parameterList](#), [Command.parameters](#), [Command.paramsint](#), [Command.Set\(StoredProgram, string\)](#), [Command.ProcessParameters\(string\)](#), [Command.ToString\(\)](#), [Command.Program](#), [Command.Name](#), [Command.ParameterList](#), [Command.Parameters](#), [Command.Paramsint](#), [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

Call()

```
public Call()
```

Fields

methodName

```
protected string methodName
```

Field Value

[string](#) 

Methods

Compile()

```
public override void Compile()
```

Execute()

```
public override void Execute()
```

Class Canvas

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class Canvas : ICanvas
```

Inheritance

[object](#)  ← Canvas

Implements

[ICanvas](#)

Inherited Members

[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Canvas()

```
public Canvas()
```

Fields

background_colour

```
protected Color background_colour
```

Field Value

[Color](#) 

Properties

PenColour

```
public virtual object PenColour { get; set; }
```

Property Value

[object](#)

Xpos

```
public virtual int Xpos { get; set; }
```

Property Value

[int](#)

Ypos

```
public virtual int Ypos { get; set; }
```

Property Value

[int](#)

Methods

Circle(int, bool)

```
public virtual void Circle(int radius, bool filled)
```

Parameters

radius [int](#)

filled [bool](#)

Clear()

```
public virtual void Clear()
```

DrawTo(int, int)

```
public virtual void DrawTo(int x, int y)
```

Parameters

x [int](#)

y [int](#)

MoveTo(int, int)

```
public virtual void MoveTo(int x, int y)
```

Parameters

x [int](#)

y [int](#)

Rect(int, int, bool)

```
public virtual void Rect(int width, int height, bool filled)
```

Parameters

width [int](#)

height [int](#)

filled [bool](#)

Rectangle(int, int, bool)

```
public virtual void Rectangle(int width, int height, bool filled)
```

Parameters

width [int](#)

height [int](#)

filled [bool](#)

Reset()

```
public virtual void Reset()
```

Set(int, int)

```
public virtual void Set(int width, int height)
```

Parameters

width [int](#)

height [int](#)

SetColour(int, int, int)

```
public virtual void SetColour(int red, int green, int blue)
```

Parameters

red [int](#)

green [int](#)

blue [int](#)

Tri(int, int)

```
public virtual void Tri(int width, int height)
```

Parameters

width [int](#)

height [int](#)

WriteText(string)

```
public virtual void WriteText(string text)
```

Parameters

text [string](#)

getBitmap()

```
public virtual object getBitmap()
```

Returns

[object](#)

Class CanvasCommand

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public abstract class CanvasCommand : Command, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← CanvasCommand

Implements

[ICommand](#)

Derived

[CommandOneParameter](#)

Inherited Members

[Command.program](#) , [Command.parameterList](#) , [Command.parameters](#) , [Command.paramsint](#) , [Command.CheckParameters\(string\[\]\)](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.Compile\(\)](#) , [Command.Execute\(\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.ToString\(\)](#) , [Command.Program](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Paramsint](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

CanvasCommand()

```
public CanvasCommand()
```

CanvasCommand(ICanvas)

```
public CanvasCommand(ICanvas c)
```

Parameters

c [ICanvas](#)

Fields

canvas

```
protected ICanvas canvas
```

Field Value

[ICanvas](#)

xPos

```
protected int xPos
```

Field Value

[int](#)

yPos

```
protected int yPos
```

Field Value

[int](#)

Properties

Canvas

```
public ICanvas Canvas { get; set; }
```


Property Value

[ICanvas](#)

Class CanvasException

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class CanvasException : BOOSEException, ISerializable
```

Inheritance

[object](#)  ← [Exception](#)  ← [BOOSEException](#) ← CanvasException

Implements

[ISerializable](#) 

Inherited Members

[Exception.GetBaseException\(\)](#)  , [Exception.ToString\(\)](#)  , [Exception.GetType\(\)](#)  ,
[Exception.TargetSite](#)  , [Exception.Message](#)  , [Exception.Data](#)  , [Exception.InnerException](#)  ,
[Exception.HelpLink](#)  , [Exception.Source](#)  , [Exception.HResult](#)  , [Exception.StackTrace](#)  ,
[Exception.SerializeObjectState](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

CanvasException(string)

```
public CanvasException(string msg)
```

Parameters

msg [string](#) 

Class Cast

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class Cast : Command, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← Cast

Implements

[ICommand](#)

Inherited Members

[Command.program](#) , [Command.parameterList](#) , [Command.parameters](#) , [Command.paramsint](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.ToString\(\)](#) , [Command.Program](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Paramsint](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Cast()

```
public Cast()
```

Methods

CheckParameters(string[])

```
public override void CheckParameters(string[] parameter)
```

Parameters

parameter [string](#)[]

Compile()

```
public override void Compile()
```

Execute()

```
public override void Execute()
```

Class Circle

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class Circle : CommandOneParameter, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [CanvasCommand](#) ← [CommandOneParameter](#) ← Circle

Implements

[ICommand](#)

Inherited Members

[CommandOneParameter.param1](#) , [CommandOneParameter.param1unprocessed](#) ,
[CanvasCommand.yPos](#) , [CanvasCommand.xPos](#) , [CanvasCommand.canvas](#) ,
[CanvasCommand.Canvas](#) , [Command.program](#) , [Command.parameterList](#) ,
[Command.parameters](#) , [Command.paramsint](#) , [Command.Set\(StoredProgram, string\)](#) ,
[Command.Compile\(\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.ToString\(\)](#) ,
[Command.Program](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) ,
[Command.Paramsint](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Circle()

```
public Circle()
```

Circle(Canvas, int)

```
public Circle(Canvas c, int radius)
```

Parameters

c [Canvas](#)

radius [int](#)

Methods

CheckParameters(string[])

```
public override void CheckParameters(string[] parameterList)
```

Parameters

parameterList [string](#)[]

Execute()

```
public override void Execute()
```

Class Command

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public abstract class Command : ICommand
```

Inheritance

[object](#)  ← Command

Implements

[ICommand](#)

Derived

[CanvasCommand](#), [Cast](#), [Evaluation](#)

Inherited Members

[object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.Equals\(object\)](#) ,
[object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

Command()

```
public Command()
```

Fields

parameterList

```
protected string parameterList
```

Field Value

[string](#) 

parameters

```
protected string[] parameters
```

Field Value

[string](#)  []

paramsint

```
protected int[] paramsint
```

Field Value

[int](#)  []

program

```
protected StoredProgram program
```

Field Value

[StoredProgram](#)

Properties

Name

```
public string Name { get; }
```

Property Value

[string](#) 

ParameterList

```
public string ParameterList { get; }
```

Property Value

[string](#)

Parameters

```
public string[] Parameters { get; set; }
```

Property Value

[string](#) []

Paramsint

```
public int[] Paramsint { get; set; }
```

Property Value

[int](#) []

Program

```
public StoredProgram Program { get; set; }
```

Property Value

[StoredProgram](#)

Methods

CheckParameters(string[])

```
public abstract void CheckParameters(string[] parameter)
```

Parameters

parameter [string](#)[]

Compile()

```
public virtual void Compile()
```

Execute()

```
public virtual void Execute()
```

ProcessParameters(string)

```
public int ProcessParameters(string ParameterList)
```

Parameters

ParameterList [string](#)

Returns

[int](#)

Set(StoredProgram, string)

```
public virtual void Set(StoredProgram Program, string Params)
```

Parameters

Program [StoredProgram](#)

Params [string](#)

ToString()

```
public override string ToString()
```

Returns

[string](#)

Class CommandException

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class CommandException : BOOSEException, ISerializable
```

Inheritance

[object](#)  ← [Exception](#)  ← [BOOSEException](#) ← CommandException

Implements

[ISerializable](#) 

Inherited Members

[Exception.GetBaseException\(\)](#)  , [Exception.ToString\(\)](#)  , [Exception.GetType\(\)](#)  ,
[Exception.TargetSite](#)  , [Exception.Message](#)  , [Exception.Data](#)  , [Exception.InnerException](#)  ,
[Exception.HelpLink](#)  , [Exception.Source](#)  , [Exception.HResult](#)  , [Exception.StackTrace](#)  ,
[Exception.SerializeObjectState](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

CommandException(string)

```
public CommandException(string msg)
```

Parameters

msg [string](#) 

Class CommandFactory

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class CommandFactory : ICommandFactory
```

Inheritance

[object](#)  ← CommandFactory

Implements

[ICommandFactory](#)

Inherited Members

[object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ToString\(\)](#) , [object.Equals\(object\)](#) ,
[object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

CommandFactory()

```
public CommandFactory()
```

Methods

MakeCommand(string)

```
public virtual ICommand MakeCommand(string commandType)
```

Parameters

commandType [string](#) 

Returns

[ICommand](#)

Class CommandOneParameter

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public abstract class CommandOneParameter : CanvasCommand, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [CanvasCommand](#) ← CommandOneParameter

Implements

[ICommand](#)

Derived

[Circle](#), [CommandTwoParameters](#), [Rect](#)

Inherited Members

[CanvasCommand.yPos](#), [CanvasCommand.xPos](#), [CanvasCommand.canvas](#),
[CanvasCommand.Canvas](#), [Command.program](#), [Command.parameterList](#),
[Command.parameters](#), [Command.paramsint](#), [Command.Set\(StoredProgram, string\)](#),
[Command.Compile\(\)](#), [Command.Execute\(\)](#), [Command.ProcessParameters\(string\)](#),
[Command.ToString\(\)](#), [Command.Program](#), [Command.Name](#), [Command.ParameterList](#),
[Command.Parameters](#), [Command.Paramsint](#), [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) ,
[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) ,
[object.GetHashCode\(\)](#) 

Constructors

CommandOneParameter()

```
public CommandOneParameter()
```

CommandOneParameter(Canvas)

```
public CommandOneParameter(Canvas c)
```

Parameters

c [Canvas](#)

Fields

param1

```
protected int param1
```

Field Value

[int](#)

param1unprocessed

```
protected string param1unprocessed
```

Field Value

[string](#)

Methods

CheckParameters(string[])

```
public override void CheckParameters(string[] parameterList)
```

Parameters

parameterList [string](#)[]

Class CommandThreeParameters

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public abstract class CommandThreeParameters : CommandTwoParameters, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [CanvasCommand](#) ← [CommandOneParameter](#) ← [CommandTwoParameters](#) ← CommandThreeParameters

Implements

[ICommand](#)

Derived

[PenColour](#)

Inherited Members

[CommandTwoParameters.param2](#) , [CommandTwoParameters.param2unprocessed](#) , [CommandOneParameter.param1](#) , [CommandOneParameter.param1unprocessed](#) , [CanvasCommand.yPos](#) , [CanvasCommand.xPos](#) , [CanvasCommand.canvas](#) , [CanvasCommand.Canvas](#) , [Command.program](#) , [Command.parameterList](#) , [Command.parameters](#) , [Command.paramsint](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.Compile\(\)](#) , [Command.Execute\(\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.ToString\(\)](#) , [Command.Program](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Paramsint](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

CommandThreeParameters()

```
public CommandThreeParameters()
```

CommandThreeParameters(Canvas)

```
public CommandThreeParameters(Canvas c)
```

Parameters

c [Canvas](#)

Fields

param3

```
protected int param3
```

Field Value

[int](#)

param3unprocessed

```
protected string param3unprocessed
```

Field Value

[string](#)

Methods

CheckParameters(string[])

```
public override void CheckParameters(string[] parameterList)
```

Parameters

parameterList [string](#) []

Class CommandTwoParameters

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public abstract class CommandTwoParameters : CommandOneParameter, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [CanvasCommand](#) ← [CommandOneParameter](#) ← CommandTwoParameters

Implements

[ICommand](#)

Derived

[CommandThreeParameters](#), [DrawTo](#), [MoveTo](#)

Inherited Members

[CommandOneParameter.param1](#) , [CommandOneParameter.param1unprocessed](#) ,
[CanvasCommand.yPos](#) , [CanvasCommand.xPos](#) , [CanvasCommand.canvas](#) ,
[CanvasCommand.Canvas](#) , [Command.program](#) , [Command.parameterList](#) ,
[Command.parameters](#) , [Command.paramsint](#) , [Command.Set\(StoredProgram, string\)](#) ,
[Command.Compile\(\)](#) , [Command.Execute\(\)](#) , [Command.ProcessParameters\(string\)](#) ,
[Command.ToString\(\)](#) , [Command.Program](#) , [Command.Name](#) , [Command.ParameterList](#) ,
[Command.Parameters](#) , [Command.Paramsint](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  ,
[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.GetHashCode\(\)](#) 

Constructors

CommandTwoParameters()

```
public CommandTwoParameters()
```

CommandTwoParameters(Canvas)

```
public CommandTwoParameters(Canvas c)
```

Parameters

c [Canvas](#)

Fields

param2

```
protected int param2
```

Field Value

[int](#)

param2unprocessed

```
protected string param2unprocessed
```

Field Value

[string](#)

Methods

CheckParameters(string[])

```
public override void CheckParameters(string[] parameterList)
```

Parameters

parameterList [string](#)[]

Class CompoundCommand

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class CompoundCommand : ConditionalCommand, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [Evaluation](#) ← [Boolean](#) ← [ConditionalCommand](#) ← CompoundCommand

Implements

[ICommand](#)

Derived

[Call](#), [Else](#), [End](#), [If](#), [Method](#), [While](#)

Inherited Members

[ConditionalCommand.endLineNumber](#) , [ConditionalCommand.Execute\(\)](#) ,
[ConditionalCommand.EndLineNumber](#) , [ConditionalCommand.Condition](#) ,
[ConditionalCommand.LineNumber](#) , [ConditionalCommand.CondType](#) ,
[ConditionalCommand.ReturnLineNumber](#) , [Boolean.Restrictions\(\)](#) , [Boolean.BoolValue](#) ,
[Evaluation.expression](#) , [Evaluation.evaluatedExpression](#) , [Evaluation.varName](#) , [Evaluation.value](#) ,
[Evaluation.ProcessExpression\(string\)](#) , [Evaluation.Expression](#) , [Evaluation.VarName](#) ,
[Evaluation.Value](#) , [Evaluation.Local](#) , [Command.program](#) , [Command.parameterList](#) ,
[Command.parameters](#) , [Command.paramsint](#) , [Command.Set\(StoredProgram, string\)](#) ,
[Command.ProcessParameters\(string\)](#) , [Command.ToString\(\)](#) , [Command.Program](#) ,
[Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Paramsint](#) ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

CompoundCommand()

```
public CompoundCommand()
```

Properties

CorrespondingCommand

```
public ConditionalCommand CorrespondingCommand { get; set; }
```

Property Value

[ConditionalCommand](#)

Methods

CheckParameters(string[])

```
public override void CheckParameters(string[] parameter)
```

Parameters

parameter [string](#)[↗][]

Compile()

```
public override void Compile()
```

ReduceRestrictions()

```
protected void ReduceRestrictions()
```

Class ConditionalCommand

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class ConditionalCommand : Boolean, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [Evaluation](#) ← [Boolean](#) ← ConditionalCommand

Implements

[ICommand](#)

Derived

[CompoundCommand](#), [For](#)

Inherited Members

[Boolean.Restrictions\(\)](#), [Boolean.BoolValue](#), [Evaluation.expression](#), [Evaluation.evaluatedExpression](#), [Evaluation.varName](#), [Evaluation.value](#), [Evaluation.CheckParameters\(string\[\]\)](#), [Evaluation.ProcessExpression\(string\)](#), [Evaluation.Expression](#), [Evaluation.VarName](#), [Evaluation.Value](#), [Evaluation.Local](#), [Command.program](#), [Command.parameterList](#), [Command.parameters](#), [Command.paramsint](#), [Command.Set\(StoredProgram, string\)](#), [Command.ProcessParameters\(string\)](#), [Command.ToString\(\)](#), [Command.Program](#), [Command.Name](#), [Command.ParameterList](#), [Command.Parameters](#), [Command.Paramsint](#), [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

ConditionalCommand()

```
public ConditionalCommand()
```

Fields

endLineNumber

```
protected int endLineNumber
```

Field Value

[int](#)

Properties

CondType

```
public ConditionalCommand.conditionalTypes CondType { get; set; }
```

Property Value

[ConditionalCommand.conditionalTypes](#)

Condition

```
public bool Condition { get; set; }
```

Property Value

[bool](#)

EndLineNumber

```
public int EndLineNumber { get; set; }
```

Property Value

[int](#)

LineNumber

```
public int LineNumber { get; set; }
```

Property Value

[int](#)

ReturnLineNumber

```
public int ReturnLineNumber { get; set; }
```

Property Value

[int](#)

Methods

Compile()

```
public override void Compile()
```

Execute()

```
public override void Execute()
```

Enum ConditionalCommand.conditionalTypes

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public enum ConditionalCommand.conditionalTypes
```

Fields

```
commFor = 2
```

```
commIF = 0
```

```
commWhile = 1
```

Class DrawTo

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class DrawTo : CommandTwoParameters, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [CanvasCommand](#) ← [CommandOneParameter](#) ← [CommandTwoParameters](#) ← DrawTo

Implements

[ICommand](#)

Inherited Members

[CommandTwoParameters.param2](#) , [CommandTwoParameters.param2unprocessed](#) , [CommandTwoParameters.CheckParameters\(string\[\]\)](#) , [CommandOneParameter.param1](#) , [CommandOneParameter.param1unprocessed](#) , [CanvasCommand.yPos](#) , [CanvasCommand.xPos](#) , [CanvasCommand.canvas](#) , [CanvasCommand.Canvas](#) , [Command.program](#) , [Command.parameterList](#) , [Command.parameters](#) , [Command.paramsint](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.Compile\(\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.ToString\(\)](#) , [Command.Program](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Paramsint](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

DrawTo()

```
public DrawTo()
```

DrawTo(Canvas, int, int)

```
public DrawTo(Canvas c, int x, int y)
```

Parameters

c [Canvas](#)

x [int](#)

y [int](#)

Methods

Execute()

```
public override void Execute()
```

Class Else

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class Else : CompoundCommand, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [Evaluation](#) ← [Boolean](#) ← [ConditionalCommand](#) ← [CompoundCommand](#) ← Else

Implements

[ICommand](#)

Inherited Members

[CompoundCommand.ReduceRestrictions\(\)](#), [CompoundCommand.CorrespondingCommand](#), [ConditionalCommand.endLineNumber](#), [ConditionalCommand.EndLineNumber](#), [ConditionalCommand.Condition](#), [ConditionalCommand.LineNumber](#), [ConditionalCommand.CondType](#), [ConditionalCommand.ReturnLineNumber](#), [Boolean.Restrictions\(\)](#), [Boolean.BoolValue](#), [Evaluation.expression](#), [Evaluation.evaluatedExpression](#), [Evaluation.varName](#), [Evaluation.value](#), [Evaluation.ProcessExpression\(string\)](#), [Evaluation.Expression](#), [Evaluation.VarName](#), [Evaluation.Value](#), [Evaluation.Local](#), [Command.program](#), [Command.parameterList](#), [Command.parameters](#), [Command.paramsint](#), [Command.Set\(StoredProgram, string\)](#), [Command.ProcessParameters\(string\)](#), [Command.ToString\(\)](#), [Command.Program](#), [Command.Name](#), [Command.ParameterList](#), [Command.Parameters](#), [Command.Paramsint](#), [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

Else()

```
public Else()
```

Properties

CorrespondingEnd

```
public End CorrespondingEnd { get; set; }
```

Property Value

[End](#)

Methods

CheckParameters(string[])

```
public override void CheckParameters(string[] parameter)
```

Parameters

parameter [string](#)

Compile()

```
public override void Compile()
```

Execute()

```
public override void Execute()
```

Class End

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class End : CompoundCommand, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [Evaluation](#) ← [Boolean](#) ← [ConditionalCommand](#) ← [CompoundCommand](#) ← End

Implements

[ICommand](#)

Inherited Members

[CompoundCommand.ReduceRestrictions\(\)](#), [CompoundCommand.CheckParameters\(string\[\]\)](#), [CompoundCommand.CorrespondingCommand](#), [ConditionalCommand.endLineNumber](#), [ConditionalCommand.EndLineNumber](#), [ConditionalCommand.Condition](#), [ConditionalCommand.LineNumber](#), [ConditionalCommand.CondType](#), [ConditionalCommand.ReturnLineNumber](#), [Boolean.Restrictions\(\)](#), [Boolean.BoolValue](#), [Evaluation.expression](#), [Evaluation.evaluatedExpression](#), [Evaluation.varName](#), [Evaluation.value](#), [Evaluation.ProcessExpression\(string\)](#), [Evaluation.Expression](#), [Evaluation.VarName](#), [Evaluation.Value](#), [Evaluation.Local](#), [Command.program](#), [Command.parameterList](#), [Command.parameters](#), [Command.paramsint](#), [Command.Set\(StoredProgram, string\)](#), [Command.ProcessParameters\(string\)](#), [Command.ToString\(\)](#), [Command.Program](#), [Command.Name](#), [Command.ParameterList](#), [Command.Parameters](#), [Command.Paramsint](#), [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

End()

```
public End()
```

Methods

Compile()

```
public override void Compile()
```

Execute()

```
public override void Execute()
```


Class Evaluation

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class Evaluation : Command, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← Evaluation

Implements

[ICommand](#)

Derived

[Array](#), [Boolean](#), [Int](#), [Real](#), [Write](#)

Inherited Members

[Command.program](#) , [Command.parameterList](#) , [Command.parameters](#) , [Command.paramsint](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.ToString\(\)](#) , [Command.Program](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Paramsint](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Evaluation()

```
public Evaluation()
```

Fields

evaluatedExpression

```
protected string evaluatedExpression
```

Field Value

[string](#)

expression

```
protected string expression
```

Field Value

[string](#)

value

```
protected int value
```

Field Value

[int](#)

varName

```
protected string varName
```

Field Value

[string](#)

Properties

Expression

```
public string Expression { get; set; }
```

Property Value

[string](#)

Local

```
public bool Local { get; set; }
```

Property Value

[bool](#)

Value

```
public virtual int Value { get; set; }
```

Property Value

[int](#)

VarName

```
public string VarName { get; set; }
```

Property Value

[string](#)

Methods

CheckParameters(string[])

```
public override void CheckParameters(string[] parameterList)
```

Parameters

parameterList [string](#)[]

Compile()

```
public override void Compile()
```

Execute()

```
public override void Execute()
```

ProcessExpression(string)

```
public virtual string ProcessExpression(string Expression)
```

Parameters

Expression [string](#)

Returns

[string](#)

Class FactoryException

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class FactoryException : BOOSEException, ISerializable
```

Inheritance

[object](#)  ← [Exception](#)  ← [BOOSEException](#) ← FactoryException

Implements

[ISerializable](#) 

Inherited Members

[Exception.GetBaseException\(\)](#)  , [Exception.ToString\(\)](#)  , [Exception.GetType\(\)](#)  ,
[Exception.TargetSite](#)  , [Exception.Message](#)  , [Exception.Data](#)  , [Exception.InnerException](#)  ,
[Exception.HelpLink](#)  , [Exception.Source](#)  , [Exception.HResult](#)  , [Exception.StackTrace](#)  ,
[Exception.SerializeObjectState](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

FactoryException(string)

```
public FactoryException(string msg)
```

Parameters

msg [string](#) 

Class For

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class For : ConditionalCommand, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [Evaluation](#) ← [Boolean](#) ← [ConditionalCommand](#) ← For

Implements

[ICommand](#)

Inherited Members

[ConditionalCommand.EndLineNumber](#) , [ConditionalCommand.EndLineNumber](#) ,
[ConditionalCommand.Condition](#) , [ConditionalCommand.LineNumber](#) ,
[ConditionalCommand.CondType](#) , [ConditionalCommand.ReturnLineNumber](#) ,
[Boolean.Restrictions\(\)](#) , [Boolean.BoolValue](#) , [Evaluation.expression](#) ,
[Evaluation.evaluatedExpression](#) , [Evaluation.varName](#) , [Evaluation.value](#) ,
[Evaluation.CheckParameters\(string\[\]\)](#) , [Evaluation.ProcessExpression\(string\)](#) ,
[Evaluation.Expression](#) , [Evaluation.VarName](#) , [Evaluation.Value](#) , [Evaluation.Local](#) ,
[Command.program](#) , [Command.parameterList](#) , [Command.parameters](#) , [Command.paramsint](#) ,
[Command.Set\(StoredProgram, string\)](#) , [Command.ProcessParameters\(string\)](#) ,
[Command.ToString\(\)](#) , [Command.Program](#) , [Command.Name](#) , [Command.ParameterList](#) ,
[Command.Parameters](#) , [Command.Paramsint](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  ,
[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.GetHashCode\(\)](#) 

Constructors

For()

```
public For()
```

Properties

From

```
public int From { get; set; }
```

Property Value

[int](#)

LoopControlV

```
public Evaluation LoopControlV { get; }
```

Property Value

[Evaluation](#)

Step

```
public int Step { get; set; }
```

Property Value

[int](#)

To

```
public int To { get; set; }
```

Property Value

[int](#)

Methods

Compile()

```
public override void Compile()
```

Execute()

```
public override void Execute()
```


Interface ICanvas

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public interface ICanvas
```

Properties

PenColour

```
object PenColour { get; set; }
```

Property Value

[object](#)

Xpos

```
int Xpos { get; set; }
```

Property Value

[int](#)

Ypos

```
int Ypos { get; set; }
```

Property Value

[int](#)

Methods

Circle(int, bool)

```
void Circle(int radius, bool filled)
```

Parameters

radius [int](#)

filled [bool](#)

Clear()

```
void Clear()
```

DrawTo(int, int)

```
void DrawTo(int x, int y)
```

Parameters

x [int](#)

y [int](#)

MoveTo(int, int)

```
void MoveTo(int x, int y)
```

Parameters

x [int](#)

y [int](#)

Rect(int, int, bool)

```
void Rect(int width, int height, bool filled)
```

Parameters

width [int](#)

height [int](#)

filled [bool](#)

Reset()

```
void Reset()
```

Set(int, int)

```
void Set(int width, int height)
```

Parameters

width [int](#)

height [int](#)

SetColour(int, int, int)

```
void SetColour(int red, int green, int blue)
```

Parameters

red [int](#)

green [int](#)

blue [int](#)

Tri(int, int)

```
void Tri(int width, int height)
```

Parameters

width [int](#)

height [int](#)

WriteText(string)

```
void WriteText(string text)
```

Parameters

text [string](#)

getBitmap()

```
object getBitmap()
```

Returns

[object](#)

Interface ICommand

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public interface ICommand
```

Methods

CheckParameters(string[])

```
void CheckParameters(string[] Parameters)
```

Parameters

Parameters [string](#)[↗][]

Compile()

```
void Compile()
```

Execute()

```
void Execute()
```

Set(StoredProgram, string)

```
void Set(StoredProgram Program, string Params)
```

Parameters

Program [StoredProgram](#)

Params [string](#)

Interface ICommandFactory

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public interface ICommandFactory
```

Methods

MakeCommand(string)

```
ICommand MakeCommand(string commandType)
```

Parameters

commandType [string](#) 

Returns

[ICommand](#)

Interface IEvaluation

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public interface IEvaluation : ICommand
```

Inherited Members

[ICommand.Set\(StoredProgram, string\)](#), [ICommand.Compile\(\)](#), [ICommand.Execute\(\)](#),
[ICommand.CheckParameters\(string\[\]\)](#)

Properties

Expression

```
string Expression { get; set; }
```

Property Value

[string](#) 

Value

```
object Value { get; set; }
```

Property Value

[object](#) 

VarName

```
string VarName { get; set; }
```


Property Value

[string](#) 

Interface IParser

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public interface IParser
```

Methods

ParseCommand(string)

```
ICommand ParseCommand(string Line)
```

Parameters

Line [string](#) 

Returns

[ICommand](#)

ParseProgram(string)

```
void ParseProgram(string program)
```

Parameters

program [string](#) 

Interface IStoredProgram

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public interface IStoredProgram
```

Properties

PC

```
int PC { get; set; }
```

Property Value

[int](#)

Methods

Add(Command)

```
int Add(Command C)
```

Parameters

C [Command](#)

Returns

[int](#)

AddVariable(Evaluation)

```
void AddVariable(Evaluation Variable)
```

Parameters

Variable [Evaluation](#)

Commandsleft()

```
bool Commandsleft()
```

Returns

[bool](#)

EvaluateExpression(string)

```
string EvaluateExpression(string Exp)
```

Parameters

Exp [string](#)

Returns

[string](#)

GetVarValue(string)

```
string GetVarValue(string varName)
```

Parameters

varName [string](#)

Returns

[string](#)

IsExpression(string)

```
bool IsExpression(string expression)
```

Parameters

expression [string](#)

Returns

[bool](#)

ResetProgram()

```
void ResetProgram()
```

Run()

```
void Run()
```

VariableExists(string)

```
bool VariableExists(string varName)
```

Parameters

varName [string](#)

Returns

Class If

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class If : CompoundCommand, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [Evaluation](#) ← [Boolean](#) ← [ConditionalCommand](#) ← [CompoundCommand](#) ← If

Implements

[ICommand](#)

Inherited Members

[CompoundCommand.ReduceRestrictions\(\)](#), [CompoundCommand.CheckParameters\(string\[\]\)](#), [CompoundCommand.Compile\(\)](#), [CompoundCommand.CorrespondingCommand](#), [ConditionalCommand.endLineNumber](#), [ConditionalCommand.Execute\(\)](#), [ConditionalCommand.EndLineNumber](#), [ConditionalCommand.Condition](#), [ConditionalCommand.LineNumber](#), [ConditionalCommand.CondType](#), [ConditionalCommand.ReturnLineNumber](#), [Boolean.Restrictions\(\)](#), [Boolean.BoolValue](#), [Evaluation.expression](#), [Evaluation.evaluatedExpression](#), [Evaluation.varName](#), [Evaluation.value](#), [Evaluation.ProcessExpression\(string\)](#), [Evaluation.Expression](#), [Evaluation.VarName](#), [Evaluation.Value](#), [Evaluation.Local](#), [Command.program](#), [Command.parameterList](#), [Command.parameters](#), [Command.paramsint](#), [Command.Set\(StoredProgram, string\)](#), [Command.ProcessParameters\(string\)](#), [Command.ToString\(\)](#), [Command.Program](#), [Command.Name](#), [Command.ParameterList](#), [Command.Parameters](#), [Command.Paramsint](#), [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

If()

```
public If()
```

Class Int

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class Int : Evaluation, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [Evaluation](#) ← Int

Implements

[ICommand](#)

Inherited Members

[Evaluation.expression](#) , [Evaluation.evaluatedExpression](#) , [Evaluation.varName](#) , [Evaluation.value](#) , [Evaluation.CheckParameters\(string\[\]\)](#) , [Evaluation.ProcessExpression\(string\)](#) , [Evaluation.Expression](#) , [Evaluation.VarName](#) , [Evaluation.Value](#) , [Evaluation.Local](#) , [Command.program](#) , [Command.parameterList](#) , [Command.parameters](#) , [Command.paramsint](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.ToString\(\)](#) , [Command.Program](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Paramsint](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Int()

```
public Int()
```

Methods

Compile()

```
public override void Compile()
```


Execute()

```
public override void Execute()
```

Restrictions()

```
public virtual void Restrictions()
```

Class Method

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class Method : CompoundCommand, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [Evaluation](#) ← [Boolean](#) ← [ConditionalCommand](#) ← [CompoundCommand](#) ← Method

Implements

[ICommand](#)

Inherited Members

[CompoundCommand.ReduceRestrictions\(\)](#), [CompoundCommand.CorrespondingCommand](#), [ConditionalCommand.endLineNumber](#), [ConditionalCommand.EndLineNumber](#), [ConditionalCommand.Condition](#), [ConditionalCommand.LineNumber](#), [ConditionalCommand.CondType](#), [ConditionalCommand.ReturnLineNumber](#), [Boolean.Restrictions\(\)](#), [Boolean.BoolValue](#), [Evaluation.expression](#), [Evaluation.evaluatedExpression](#), [Evaluation.varName](#), [Evaluation.value](#), [Evaluation.ProcessExpression\(string\)](#), [Evaluation.Expression](#), [Evaluation.VarName](#), [Evaluation.Value](#), [Evaluation.Local](#), [Command.program](#), [Command.parameterList](#), [Command.parameters](#), [Command.paramsint](#), [Command.Set\(StoredProgram, string\)](#), [Command.ProcessParameters\(string\)](#), [Command.ToString\(\)](#), [Command.Program](#), [Command.Name](#), [Command.ParameterList](#), [Command.Parameters](#), [Command.Paramsint](#), [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

Method()

```
public Method()
```

Properties

LocalVariables

```
public string[] LocalVariables { get; }
```

Property Value

[string](#)  []

MethodName

```
public string MethodName { get; }
```

Property Value

[string](#) 

Type

```
public string Type { get; }
```

Property Value

[string](#) 

Methods

CheckParameters(string[])

```
public override void CheckParameters(string[] parameter)
```

Parameters

parameter [string](#)  []

Compile()

```
public override void Compile()
```

Execute()

```
public override void Execute()
```

Class MoveTo

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class MoveTo : CommandTwoParameters, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [CanvasCommand](#) ← [CommandOneParameter](#) ← [CommandTwoParameters](#) ← MoveTo

Implements

[ICommand](#)

Inherited Members

[CommandTwoParameters.param2](#) , [CommandTwoParameters.param2unprocessed](#) , [CommandTwoParameters.CheckParameters\(string\[\]\)](#) , [CommandOneParameter.param1](#) , [CommandOneParameter.param1unprocessed](#) , [CanvasCommand.yPos](#) , [CanvasCommand.xPos](#) , [CanvasCommand.canvas](#) , [CanvasCommand.Canvas](#) , [Command.program](#) , [Command.parameterList](#) , [Command.parameters](#) , [Command.paramsint](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.Compile\(\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.ToString\(\)](#) , [Command.Program](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Paramsint](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

MoveTo()

```
public MoveTo()
```

Methods

Execute()

```
public override void Execute()
```

Class Parser

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class Parser : IParser
```

Inheritance

[object](#)  ← Parser

Implements

[IParser](#)

Inherited Members

[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Parser(CommandFactory, StoredProgram)

```
public Parser(CommandFactory Factory, StoredProgram Program)
```

Parameters

Factory [CommandFactory](#)

Program [StoredProgram](#)

Methods

ParseCommand(string)

```
public virtual ICommand ParseCommand(string Line)
```

Parameters

Line [string](#) 

Returns

[ICommand](#)

ParseProgram(string)

```
public virtual void ParseProgram(string program)
```

Parameters

program [string](#) 

Class ParserException

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class ParserException : BOOSEException, ISerializable
```

Inheritance

[object](#)  ← [Exception](#)  ← [BOOSEException](#) ← ParserException

Implements

[ISerializable](#) 

Inherited Members

[Exception.GetBaseException\(\)](#)  , [Exception.ToString\(\)](#)  , [Exception.GetType\(\)](#)  ,
[Exception.TargetSite](#)  , [Exception.Message](#)  , [Exception.Data](#)  , [Exception.InnerException](#)  ,
[Exception.HelpLink](#)  , [Exception.Source](#)  , [Exception.HResult](#)  , [Exception.StackTrace](#)  ,
[Exception.SerializeObjectState](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

ParserException(string)

```
public ParserException(string msg)
```

Parameters

msg [string](#) 

Class Peek

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class Peek : Array, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [Evaluation](#) ← [Array](#) ← Peek

Implements

[ICommand](#)

Inherited Members

[Array.PEEK](#) , [Array.POKE](#) , [Array.type](#) , [Array.rows](#) , [Array.columns](#) , [Array.valueInt](#) , [Array.valueReal](#) , [Array.intArray](#) , [Array.realArray](#) , [Array.pokeValue](#) , [Array.peekVar](#) , [Array.rowS](#) , [Array.columnS](#) , [Array.row](#) , [Array.column](#) , [Array.ArrayRestrictions\(\)](#) , [Array.ReduceRestrictionCounter\(\)](#) , [Array.ProcessArrayParametersCompile\(bool\)](#) , [Array.ProcessArrayParametersExecute\(bool\)](#) , [Array.SetIntArray\(int, int, int\)](#) , [Array.SetRealArray\(double, int, int\)](#) , [Array.GetIntArray\(int, int\)](#) , [Array.GetRealArray\(int, int\)](#) , [Array.Rows](#) , [Array.Columns](#) , [Evaluation.expression](#) , [Evaluation.evaluatedExpression](#) , [Evaluation.varName](#) , [Evaluation.value](#) , [Evaluation.ProcessExpression\(string\)](#) , [Evaluation.Expression](#) , [Evaluation.VarName](#) , [Evaluation.Value](#) , [Evaluation.Local](#) , [Command.program](#) , [Command.parameterList](#) , [Command.parameters](#) , [Command.paramsint](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.ToString\(\)](#) , [Command.Program](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Paramsint](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Peek()

```
public Peek()
```

Methods

CheckParameters(string[])

```
public override void CheckParameters(string[] Parameters)
```

Parameters

Parameters [string](#)[]

Compile()

```
public override void Compile()
```

Execute()

```
public override void Execute()
```

Class PenColour

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class PenColour : CommandThreeParameters, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [CanvasCommand](#) ← [CommandOneParameter](#) ← [CommandTwoParameters](#) ← [CommandThreeParameters](#) ← PenColour

Implements

[ICommand](#)

Inherited Members

[CommandThreeParameters.param3](#) , [CommandThreeParameters.param3unprocessed](#) , [CommandThreeParameters.CheckParameters\(string\[\]\)](#) , [CommandTwoParameters.param2](#) , [CommandTwoParameters.param2unprocessed](#) , [CommandOneParameter.param1](#) , [CommandOneParameter.param1unprocessed](#) , [CanvasCommand.yPos](#) , [CanvasCommand.xPos](#) , [CanvasCommand.canvas](#) , [CanvasCommand.Canvas](#) , [Command.program](#) , [Command.parameterList](#) , [Command.parameters](#) , [Command.paramsint](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.Compile\(\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.ToString\(\)](#) , [Command.Program](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Paramsint](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

PenColour()

```
public PenColour()
```

Methods

Execute()

```
public override void Execute()
```

Class Poke

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class Poke : Array, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [Evaluation](#) ← [Array](#) ← Poke

Implements

[ICommand](#)

Inherited Members

[Array.PEEK](#) , [Array.POKE](#) , [Array.type](#) , [Array.rows](#) , [Array.columns](#) , [Array.valueInt](#) , [Array.valueReal](#) , [Array.intArray](#) , [Array.realArray](#) , [Array.pokeValue](#) , [Array.peekVar](#) , [Array.rowS](#) , [Array.columnS](#) , [Array.row](#) , [Array.column](#) , [Array.ArrayRestrictions\(\)](#) , [Array.ReduceRestrictionCounter\(\)](#) , [Array.ProcessArrayParametersCompile\(bool\)](#) , [Array.ProcessArrayParametersExecute\(bool\)](#) , [Array.SetIntArray\(int, int, int\)](#) , [Array.SetRealArray\(double, int, int\)](#) , [Array.GetIntArray\(int, int\)](#) , [Array.GetRealArray\(int, int\)](#) , [Array.Rows](#) , [Array.Columns](#) , [Evaluation.expression](#) , [Evaluation.evaluatedExpression](#) , [Evaluation.varName](#) , [Evaluation.value](#) , [Evaluation.ProcessExpression\(string\)](#) , [Evaluation.Expression](#) , [Evaluation.VarName](#) , [Evaluation.Value](#) , [Evaluation.Local](#) , [Command.program](#) , [Command.parameterList](#) , [Command.parameters](#) , [Command.paramsint](#) , [Command.ProcessParameters\(string\)](#) , [Command.ToString\(\)](#) , [Command.Program](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Paramsint](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Poke()

```
public Poke()
```

Methods

CheckParameters(string[])

```
public override void CheckParameters(string[] parameter)
```

Parameters

parameter [string](#) 

Compile()

```
public override void Compile()
```

Execute()

```
public override void Execute()
```

Set(StoredProgram, string)

```
public override void Set(StoredProgram Program, string Params)
```

Parameters

Program [StoredProgram](#)

Params [string](#) 

Class Real

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class Real : Evaluation, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [Evaluation](#) ← Real

Implements

[ICommand](#)

Inherited Members

[Evaluation.expression](#) , [Evaluation.evaluatedExpression](#) , [Evaluation.varName](#) , [Evaluation.value](#) , [Evaluation.CheckParameters\(string\[\]\)](#) , [Evaluation.ProcessExpression\(string\)](#) , [Evaluation.Expression](#) , [Evaluation.VarName](#) , [Evaluation.Local](#) , [Command.program](#) , [Command.parameterList](#) , [Command.parameters](#) , [Command.paramsint](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.ToString\(\)](#) , [Command.Program](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Paramsint](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Real()

```
public Real()
```

Properties

Value

```
public double Value { get; set; }
```


Property Value

[double](#)

Methods

Compile()

```
public override void Compile()
```

Execute()

```
public override void Execute()
```

Restrictions()

```
public virtual void Restrictions()
```

Class Rect

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class Rect : CommandOneParameter, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [CanvasCommand](#) ← [CommandOneParameter](#) ← Rect

Implements

[ICommand](#)

Inherited Members

[CommandOneParameter.param1](#) , [CommandOneParameter.param1unprocessed](#) , [CanvasCommand.yPos](#) , [CanvasCommand.xPos](#) , [CanvasCommand.canvas](#) , [CanvasCommand.Canvas](#) , [Command.program](#) , [Command.parameterList](#) , [Command.parameters](#) , [Command.paramsint](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.Compile\(\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.ToString\(\)](#) , [Command.Program](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Paramsint](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Rect()

```
public Rect()
```

Rect(Canvas, int, int)

```
public Rect(Canvas c, int width, int height)
```

Parameters

c [Canvas](#)

width [int](#)

height [int](#)

Methods

CheckParameters(string[])

```
public override void CheckParameters(string[] parameterList)
```

Parameters

parameterList [string](#)[]

Execute()

```
public override void Execute()
```

Class RestrictionException

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class RestrictionException : BOOSEException, ISerializable
```

Inheritance

[object](#)  ← [Exception](#)  ← [BOOSEException](#) ← RestrictionException

Implements

[ISerializable](#) 

Inherited Members

[Exception.GetBaseException\(\)](#)  , [Exception.ToString\(\)](#)  , [Exception.GetType\(\)](#)  ,
[Exception.TargetSite](#)  , [Exception.Message](#)  , [Exception.Data](#)  , [Exception.InnerException](#)  ,
[Exception.HelpLink](#)  , [Exception.Source](#)  , [Exception.HResult](#)  , [Exception.StackTrace](#)  ,
[Exception.SerializeObjectState](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

RestrictionException(string)

```
public RestrictionException(string msg)
```

Parameters

msg [string](#) 

Class StoredProgram

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class StoredProgram : ArrayList, IList, ICollection, IEnumerable,
    ICloneable, IStoredProgram
```

Inheritance

[object](#) ← [ArrayList](#) ← StoredProgram

Implements

[IList](#), [ICollection](#), [IEnumerable](#), [ICloneable](#), [IStoredProgram](#)

Inherited Members

[ArrayList.Adapter\(IList\)](#), [ArrayList.Add\(object\)](#), [ArrayList.AddRange\(ICollection\)](#), [ArrayList.BinarySearch\(int, int, object, IComparer\)](#), [ArrayList.BinarySearch\(object\)](#), [ArrayList.BinarySearch\(object, IComparer\)](#), [ArrayList.Clear\(\)](#), [ArrayList.Clone\(\)](#), [ArrayList.Contains\(object\)](#), [ArrayList.CopyTo\(Array\)](#), [ArrayList.CopyTo\(Array, int\)](#), [ArrayList.CopyTo\(int, Array, int, int\)](#), [ArrayList.FixedSize\(IList\)](#), [ArrayList.FixedSize\(ArrayList\)](#), [ArrayList.GetEnumerator\(\)](#), [ArrayList.GetEnumerator\(int, int\)](#), [ArrayList.IndexOf\(object\)](#), [ArrayList.IndexOf\(object, int\)](#), [ArrayList.IndexOf\(object, int, int\)](#), [ArrayList.Insert\(int, object\)](#), [ArrayList.InsertRange\(int, ICollection\)](#), [ArrayList.LastIndexOf\(object\)](#), [ArrayList.LastIndexOf\(object, int\)](#), [ArrayList.LastIndexOf\(object, int, int\)](#), [ArrayList.ReadOnly\(IList\)](#), [ArrayList.ReadOnly\(ArrayList\)](#), [ArrayList.Remove\(object\)](#), [ArrayList.RemoveAt\(int\)](#), [ArrayList.RemoveRange\(int, int\)](#), [ArrayList.Repeat\(object, int\)](#), [ArrayList.Reverse\(\)](#), [ArrayList.Reverse\(int, int\)](#), [ArrayList.SetRange\(int, ICollection\)](#), [ArrayList.GetRange\(int, int\)](#), [ArrayList.Sort\(\)](#), [ArrayList.Sort\(IComparer\)](#), [ArrayList.Sort\(int, int, IComparer\)](#), [ArrayList.Synchronized\(IList\)](#), [ArrayList.Synchronized\(ArrayList\)](#), [ArrayList.ToArray\(\)](#), [ArrayList.ToArray\(Type\)](#), [ArrayList.TrimToSize\(\)](#), [ArrayList.Capacity](#), [ArrayList.Count](#), [ArrayList.IsFixedSize](#), [ArrayList.IsReadOnly](#), [ArrayList.IsSynchronized](#), [ArrayList.SyncRoot](#), [ArrayList.this\[int\]](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ToString\(\)](#), [object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.ReferenceEquals\(object, object\)](#), [object.GetHashCode\(\)](#)

Constructors

StoredProgram(ICanvas)

```
public StoredProgram(ICanvas canvas)
```

Parameters

canvas [ICanvas](#)

Fields

SyntaxOk

```
public bool SyntaxOk
```

Field Value

[bool](#)

Properties

PC

```
public virtual int PC { get; set; }
```

Property Value

[int](#)

Methods

Add(Command)

```
public virtual int Add(Command C)
```

Parameters

C [Command](#)

Returns

[int](#)

AddMethod(Method)

```
public virtual void AddMethod(Method M)
```

Parameters

M [Method](#)

AddVariable(Evaluation)

```
public virtual void AddVariable(Evaluation Variable)
```

Parameters

Variable [Evaluation](#)

Commandsleft()

```
public virtual bool Commandsleft()
```

Returns

[bool](#)

DeleteVariable(string)

```
public virtual void DeleteVariable(string varName)
```

Parameters

varName [string](#)

EvaluateExpression(string)

```
public virtual string EvaluateExpression(string Exp)
```

Parameters

Exp [string](#)

Returns

[string](#)

EvaluateExpressionWithString(string)

```
public virtual string EvaluateExpressionWithString(string expression)
```

Parameters

expression [string](#)

Returns

[string](#)

FindVariable(Evaluation)

```
public virtual int FindVariable(Evaluation Variable)
```

Parameters

Variable [Evaluation](#)

Returns

[int](#)

FindVariable(string)

```
public virtual int FindVariable(string varName)
```

Parameters

varName [string](#)

Returns

[int](#)

GetMethod(string)

```
public virtual Method GetMethod(string MethodName)
```

Parameters

MethodName [string](#)

Returns

[Method](#)

GetVarValue(string)

```
public virtual string GetVarValue(string varName)
```

Parameters

varName [string](#)

Returns

[string](#)

GetVariable(int)

```
public virtual Evaluation GetVariable(int index)
```

Parameters

index [int](#)

Returns

[Evaluation](#)

GetVariable(string)

```
public virtual Evaluation GetVariable(string VarName)
```

Parameters

VarName [string](#)

Returns

[Evaluation](#)

IsExpression(string)

```
public virtual bool IsExpression(string expression)
```

Parameters

expression [string](#)

Returns

[bool](#)

NextCommand()

```
public virtual object NextCommand()
```

Returns

[object](#)

Pop()

```
public virtual ConditionalCommand Pop()
```

Returns

[ConditionalCommand](#)

Push(ConditionalCommand)

```
public virtual void Push(ConditionalCommand Com)
```

Parameters

Com [ConditionalCommand](#)

ResetProgram()

```
public virtual void ResetProgram()
```

Run()

```
public virtual void Run()
```

UpdateVariable(string, bool)

```
public virtual void UpdateVariable(string varName, bool value)
```

Parameters

varName [string](#)

value [bool](#)

UpdateVariable(string, double)

```
public virtual void UpdateVariable(string varName, double value)
```

Parameters

varName [string](#)

value [double](#)

UpdateVariable(string, int)

```
public virtual void UpdateVariable(string varName, int value)
```

Parameters

varName [string](#)

value [int](#)

VariableExists(string)

```
public virtual bool VariableExists(string varName)
```

Parameters

varName [string](#)

Returns

[bool](#)

Class StoredProgramException

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class StoredProgramException : BOOSEException, ISerializable
```

Inheritance

[object](#)  ← [Exception](#)  ← [BOOSEException](#) ← StoredProgramException

Implements

[ISerializable](#) 

Inherited Members

[Exception.GetBaseException\(\)](#)  , [Exception.ToString\(\)](#)  , [Exception.GetType\(\)](#)  ,
[Exception.TargetSite](#)  , [Exception.Message](#)  , [Exception.Data](#)  , [Exception.InnerException](#)  ,
[Exception.HelpLink](#)  , [Exception.Source](#)  , [Exception.HResult](#)  , [Exception.StackTrace](#)  ,
[Exception.SerializeObjectState](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

StoredProgramException(string)

```
public StoredProgramException(string msg)
```

Parameters

msg [string](#) 

Class VarException

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class VarException : BOOSEException, ISerializable
```

Inheritance

[object](#)  ← [Exception](#)  ← [BOOSEException](#) ← VarException

Implements

[ISerializable](#) 

Inherited Members

[Exception.GetBaseException\(\)](#)  , [Exception.ToString\(\)](#)  , [Exception.GetType\(\)](#)  ,
[Exception.TargetSite](#)  , [Exception.Message](#)  , [Exception.Data](#)  , [Exception.InnerException](#)  ,
[Exception.HelpLink](#)  , [Exception.Source](#)  , [Exception.HResult](#)  , [Exception.StackTrace](#)  ,
[Exception.SerializeObjectState](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

VarException(string)

```
public VarException(string msg)
```

Parameters

msg [string](#) 

Class While

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class While : CompoundCommand, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [Evaluation](#) ← [Boolean](#) ← [ConditionalCommand](#) ← [CompoundCommand](#) ←
While

Implements

[ICommand](#)

Inherited Members

[CompoundCommand.ReduceRestrictions\(\)](#), [CompoundCommand.CheckParameters\(string\[\]\)](#),
[CompoundCommand.Compile\(\)](#), [CompoundCommand.CorrespondingCommand](#),
[ConditionalCommand.EndLineNumber](#), [ConditionalCommand.Execute\(\)](#),
[ConditionalCommand.EndLineNumber](#), [ConditionalCommand.Condition](#),
[ConditionalCommand.LineNumber](#), [ConditionalCommand.CondType](#),
[ConditionalCommand.ReturnLineNumber](#), [Boolean.Restrictions\(\)](#), [Boolean.BoolValue](#),
[Evaluation.expression](#), [Evaluation.evaluatedExpression](#), [Evaluation.varName](#), [Evaluation.value](#),
[Evaluation.ProcessExpression\(string\)](#), [Evaluation.Expression](#), [Evaluation.VarName](#),
[Evaluation.Value](#), [Evaluation.Local](#), [Command.program](#), [Command.parameterList](#),
[Command.parameters](#), [Command.paramsint](#), [Command.Set\(StoredProgram, string\)](#),
[Command.ProcessParameters\(string\)](#), [Command.ToString\(\)](#), [Command.Program](#),
[Command.Name](#), [Command.ParameterList](#), [Command.Parameters](#), [Command.Paramsint](#),
[object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.Equals\(object\)](#) ,
[object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

While()

```
public While()
```


Class Write

Namespace: [BOOSE](#)

Assembly: BOOSE.dll

```
public class Write : Evaluation, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [Evaluation](#) ← Write

Implements

[ICommand](#)

Inherited Members

[Evaluation.expression](#) , [Evaluation.evaluatedExpression](#) , [Evaluation.varName](#) , [Evaluation.value](#) , [Evaluation.Compile\(\)](#) , [Evaluation.ProcessExpression\(string\)](#) , [Evaluation.Expression](#) , [Evaluation.VarName](#) , [Evaluation.Value](#) , [Evaluation.Local](#) , [Command.program](#) , [Command.parameterList](#) , [Command.parameters](#) , [Command.paramsint](#) , [Command.Set\(StoredProgram, string\)](#) , [Command.ProcessParameters\(string\)](#) , [Command.ToString\(\)](#) , [Command.Program](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) , [Command.Paramsint](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

Write()

```
public Write()
```

Methods

CheckParameters(string[])

```
public override void CheckParameters(string[] parameter)
```

Parameters

parameter [string](#)  []

Execute()

```
public override void Execute()
```

Namespace Blazor.Extensions

Classes

[BECanvasComponent](#)

[CanvasContextExtensions](#)

[RenderingContext](#)

Class BECanvasComponent

Namespace: [Blazor.Extensions](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public class BECanvasComponent : ComponentBase, IComponent,
    IHandleEvent, IHandleAfterRender
```

Inheritance

[object](#)  ← [ComponentBase](#)  ← BECanvasComponent

Implements

[IComponent](#) , [IHandleEvent](#) , [IHandleAfterRender](#) 

Derived

[BECanvas](#)

Inherited Members

[ComponentBase.BuildRenderTree\(RenderTreeBuilder\)](#) , [ComponentBase.OnInitialized\(\)](#) , [ComponentBase.OnInitializedAsync\(\)](#) , [ComponentBase.OnParametersSet\(\)](#) , [ComponentBase.OnParametersSetAsync\(\)](#) , [ComponentBase.StateHasChanged\(\)](#) , [ComponentBase.ShouldRender\(\)](#) , [ComponentBase.OnAfterRender\(bool\)](#) , [ComponentBase.OnAfterRenderAsync\(bool\)](#) , [ComponentBase.InvokeAsync\(Action\)](#) , [ComponentBase.InvokeAsync\(Func<Task>\)](#) , [ComponentBase.DispatchExceptionAsync\(Exception\)](#) , [ComponentBase.SetParametersAsync\(ParameterView\)](#) , [ComponentBase.RendererInfo](#) , [ComponentBase.Assets](#) , [ComponentBase.AssignedRenderMode](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ToString\(\)](#) , [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Extension Methods

[CanvasContextExtensions.CreateCanvas2D\(BECanvasComponent\)](#) , [CanvasContextExtensions.CreateCanvas2DAsync\(BECanvasComponent\)](#) , [CanvasContextExtensions.CreateWebGL\(BECanvasComponent\)](#) , [CanvasContextExtensions.CreateWebGL\(BECanvasComponent, WebGLContextAttributes\)](#) , [CanvasContextExtensions.CreateWebGLAsync\(BECanvasComponent\)](#) , [CanvasContextExtensions.CreateWebGLAsync\(BECanvasComponent, WebGLContextAttributes\)](#) 

Constructors

BECanvasComponent()

```
public BECanvasComponent()
```

Fields

Id

```
protected readonly string Id
```

Field Value

[string](#)

_canvasRef

```
protected ElementReference _canvasRef
```

Field Value

[ElementReference](#)

Properties

CanvasReference

```
public ElementReference CanvasReference { get; }
```

Property Value

[ElementReference](#)

Height

[Parameter]

```
public long Height { get; set; }
```

Property Value

[long](#) 

Width

[Parameter]

```
public long Width { get; set; }
```

Property Value

[long](#) 

Class CanvasContextExtensions

Namespace: [Blazor.Extensions](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public static class CanvasContextExtensions
```

Inheritance

[object](#)  ← CanvasContextExtensions

Inherited Members

[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Methods

CreateCanvas2D(BECanvasComponent)

```
public static Canvas2DContext CreateCanvas2D(this BECanvasComponent canvas)
```

Parameters

canvas [BECanvasComponent](#)

Returns

[Canvas2DContext](#)

CreateCanvas2DAsync(BECanvasComponent)

```
public static Task<Canvas2DContext> CreateCanvas2DAsync(this  
BECanvasComponent canvas)
```

Parameters

canvas [BECanvasComponent](#)

Returns

[Task](#) [↗](#) <[Canvas2DContext](#)>

CreateWebGL(BECanvasComponent)

```
public static WebGLContext CreateWebGL(this BECanvasComponent canvas)
```

Parameters

canvas [BECanvasComponent](#)

Returns

[WebGLContext](#)

CreateWebGL(BECanvasComponent, WebGLContext Attributes)

```
public static WebGLContext CreateWebGL(this BECanvasComponent canvas,  
WebGLContextAttributes attributes)
```

Parameters

canvas [BECanvasComponent](#)

attributes [WebGLContextAttributes](#)

Returns

[WebGLContext](#)

CreateWebGLAsync(BECanvasComponent)


```
public static Task<WebGLContext> CreateWebGLAsync(this BECanvasComponent canvas)
```

Parameters

canvas [BECanvasComponent](#)

Returns

[Task](#)  <[WebGLContext](#)>

CreateWebGLAsync(BECanvasComponent, WebGLContext Attributes)

```
public static Task<WebGLContext> CreateWebGLAsync(this BECanvasComponent canvas,  
WebGLContextAttributes attributes)
```

Parameters

canvas [BECanvasComponent](#)

attributes [WebGLContextAttributes](#)

Returns

[Task](#)  <[WebGLContext](#)>

Class RenderingContext

Namespace: [Blazor.Extensions](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public abstract class RenderingContext : IDisposable
```

Inheritance

[object](#)  ← RenderingContext

Implements

[IDisposable](#) 

Derived

[Canvas2DContext](#), [WebGLContext](#)

Inherited Members

[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Properties

Canvas

```
public ElementReference Canvas { get; }
```

Property Value

[ElementReference](#) 

Methods

BatchCallAsync(string, bool, params object[])

```
protected Task BatchCallAsync(string name, bool isMethodCall, params object[] value)
```

Parameters

name [string](#)

isMethodCall [bool](#)

value [object](#)[]

Returns

[Task](#)

BeginBatchAsync()

```
public Task BeginBatchAsync()
```

Returns

[Task](#)

CallMethodAsync<T>(string)

```
protected Task<T> CallMethodAsync<T>(string method)
```

Parameters

method [string](#)

Returns

[Task](#)<T>

Type Parameters

T

CallMethodAsync<T>(string, params object[])

```
protected Task<T> CallMethodAsync<T>(string method, params object[] value)
```

Parameters

method [string](#)

value [object](#)[]

Returns

[Task](#)<T>

Type Parameters

T

CallMethod<T>(string)

```
protected T CallMethod<T>(string method)
```

Parameters

method [string](#)

Returns

T

Type Parameters

T

CallMethod<T>(string, params object[])

```
protected T CallMethod<T>(string method, params object[] value)
```

Parameters

method [string](#)[↗]

value [object](#)[↗][]

Returns

T

Type Parameters

T

Dispose()

```
public void Dispose()
```

EndBatchAsync()

```
public Task EndBatchAsync()
```

Returns

[Task](#)[↗]

ExtendedInitializeAsync()

```
protected virtual Task ExtendedInitializeAsync()
```

Returns

[Task](#)[↗]

GetPropertyAsync<T>(string)

```
protected Task<T> GetPropertyAsync<T>(string property)
```

Parameters

property [string](#)

Returns

[Task](#)<T>

Type Parameters

T

Namespace Blazor.Extensions.Canvas

Classes

[BECanvas](#)

Class BECanvas

Namespace: [Blazor.Extensions.Canvas](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public class BECanvas : BECanvasComponent, IComponent,
    IHandleEvent, IHandleAfterRender
```

Inheritance

[object](#)  ← [ComponentBase](#)  ← [BECanvasComponent](#)  ← BECanvas

Implements

[IComponent](#) , [IHandleEvent](#) , [IHandleAfterRender](#) 

Inherited Members

[BECanvasComponent.Id](#), [BECanvasComponent._canvasRef](#), [BECanvasComponent.Height](#),
[BECanvasComponent.Width](#), [BECanvasComponent.CanvasReference](#),
[ComponentBase.OnInitialized\(\)](#) , [ComponentBase.OnInitializedAsync\(\)](#) ,
[ComponentBase.OnParametersSet\(\)](#) , [ComponentBase.OnParametersSetAsync\(\)](#) ,
[ComponentBase.StateHasChanged\(\)](#) , [ComponentBase.ShouldRender\(\)](#) ,
[ComponentBase.OnAfterRender\(bool\)](#) , [ComponentBase.OnAfterRenderAsync\(bool\)](#) ,
[ComponentBase.InvokeAsync\(Action\)](#) , [ComponentBase.InvokeAsync\(Func<Task>\)](#) ,
[ComponentBase.DispatchExceptionAsync\(Exception\)](#) ,
[ComponentBase.SetParametersAsync\(ParameterView\)](#) , [ComponentBase.RendererInfo](#) ,
[ComponentBase.Assets](#) , [ComponentBase.AssignedRenderMode](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ToString\(\)](#) , [object.Equals\(object\)](#) ,
[object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Extension Methods

[CanvasContextExtensions.CreateCanvas2D\(BECanvasComponent\)](#),
[CanvasContextExtensions.CreateCanvas2DAsync\(BECanvasComponent\)](#),
[CanvasContextExtensions.CreateWebGL\(BECanvasComponent\)](#),
[CanvasContextExtensions.CreateWebGL\(BECanvasComponent, WebGLContextAttributes\)](#),
[CanvasContextExtensions.CreateWebGLAsync\(BECanvasComponent\)](#),
[CanvasContextExtensions.CreateWebGLAsync\(BECanvasComponent, WebGLContextAttributes\)](#)

Constructors

BECanvas()

```
public BECanvas()
```

Methods

BuildRenderTree(RenderTreeBuilder)

```
protected override void BuildRenderTree(RenderTreeBuilder __builder)
```

Parameters

`__builder` [RenderTreeBuilder](#)

Namespace Blazor.Extensions.Canvas. Canvas2D

Classes

[Canvas2DContext](#)

Enums

[LineCap](#)

[LineJoin](#)

[RepeatPattern](#)

[TextAlign](#)

[TextBaseline](#)

[TextDirection](#)

Class Canvas2DContext

Namespace: [Blazor.Extensions.Canvas.Canvas2D](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public class Canvas2DContext : RenderingContext, IDisposable
```

Inheritance

[object](#)  ← [RenderingContext](#) ← Canvas2DContext

Implements

[IDisposable](#) 

Inherited Members

[RenderingContext.ExtendedInitializeAsync\(\)](#), [RenderingContext.BeginBatchAsync\(\)](#),
[RenderingContext.EndBatchAsync\(\)](#),
[RenderingContext.BatchCallAsync\(string, bool, params object\[\]\)](#),
[RenderingContext.GetPropertyAsync<T>\(string\)](#), [RenderingContext.CallMethod<T>\(string\)](#),
[RenderingContext.CallMethodAsync<T>\(string\)](#),
[RenderingContext.CallMethod<T>\(string, params object\[\]\)](#),
[RenderingContext.CallMethodAsync<T>\(string, params object\[\]\)](#), [RenderingContext.Dispose\(\)](#),
[RenderingContext.Canvas](#), [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ToString\(\)](#) ,
[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) ,
[object.GetHashCode\(\)](#) 

Properties

Direction

```
public TextDirection Direction { get; }
```

Property Value

[TextDirection](#)

FillStyle

```
public object FillStyle { get; }
```

Property Value

[object](#)

Font

```
public string Font { get; }
```

Property Value

[string](#)

GlobalAlpha

```
public float GlobalAlpha { get; }
```

Property Value

[float](#)

LineCap

```
public LineCap LineCap { get; }
```

Property Value

[LineCap](#)

LineDashOffset

```
public float LineDashOffset { get; }
```

Property Value

[float](#)

LineJoin

```
public LineJoin LineJoin { get; }
```

Property Value

[LineJoin](#)

LineWidth

```
public float LineWidth { get; }
```

Property Value

[float](#)

MiterLimit

```
public float MiterLimit { get; }
```

Property Value

[float](#)

ShadowBlur

```
public float ShadowBlur { get; }
```

Property Value

[float](#)

ShadowColor

```
public string ShadowColor { get; }
```

Property Value

[string](#)

ShadowOffsetX

```
public float ShadowOffsetX { get; }
```

Property Value

[float](#)

ShadowOffsetY

```
public float ShadowOffsetY { get; }
```

Property Value

[float](#)

StrokeStyle

```
public string StrokeStyle { get; }
```

Property Value

[string](#) 

TextAlign

```
public TextAlign TextAlign { get; }
```

Property Value

[TextAlign](#)

TextBaseline

```
public TextBaseline TextBaseline { get; }
```

Property Value

[TextBaseline](#)

Methods

Arc(double, double, double, double, double, bool?)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void Arc(double x, double y, double radius, double startAngle, double  
endAngle, bool? anticlockwise = null)
```

Parameters

x [double](#) 

y [double](#) 

radius [double](#)

startAngle [double](#)

endAngle [double](#)

anticlockwise [bool](#)?

ArcAsync(double, double, double, double, double, bool?)

```
public Task ArcAsync(double x, double y, double radius, double startAngle, double endAngle, bool? anticlockwise = null)
```

Parameters

x [double](#)

y [double](#)

radius [double](#)

startAngle [double](#)

endAngle [double](#)

anticlockwise [bool](#)?

Returns

[Task](#)

ArcTo(double, double, double, double, double)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void ArcTo(double x1, double y1, double x2, double y2, double radius)
```

Parameters

x1 [double](#)

y1 [double](#)

x2 [double](#)

y2 [double](#)

radius [double](#)

ArcToAsync(double, double, double, double, double)

```
public Task ArcToAsync(double x1, double y1, double x2, double y2, double radius)
```

Parameters

x1 [double](#)

y1 [double](#)

x2 [double](#)

y2 [double](#)

radius [double](#)

Returns

[Task](#)

BeginPath()

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void BeginPath()
```

BeginPathAsync()

```
public Task BeginPathAsync()
```

Returns

[Task](#)

BezierCurveTo(double, double, double, double, double, double)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void BezierCurveTo(double cp1X, double cp1Y, double cp2X, double cp2Y, double  
x, double y)
```

Parameters

cp1X [double](#)

cp1Y [double](#)

cp2X [double](#)

cp2Y [double](#)

x [double](#)

y [double](#)

BezierCurveToAsync(double, double, double, double, double, double)

```
public Task BezierCurveToAsync(double cp1X, double cp1Y, double cp2X, double cp2Y,  
double x, double y)
```

Parameters

cp1X [double](#)

cp1Y [double](#)

cp2X [double](#)

cp2Y [double](#)

x [double](#)

y [double](#)

Returns

[Task](#)

ClearRect(double, double, double, double)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void ClearRect(double x, double y, double width, double height)
```

Parameters

x [double](#)

y [double](#)

width [double](#)

height [double](#)

ClearRectAsync(double, double, double, double)

```
public Task ClearRectAsync(double x, double y, double width, double height)
```

Parameters

x [double](#)

y [double](#)

width [double](#)

height [double](#)

Returns

[Task](#)

Clip()

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void Clip()
```

ClipAsync()

```
public Task ClipAsync()
```

Returns

[Task](#)

ClosePath()

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void ClosePath()
```

ClosePathAsync()

```
public Task ClosePathAsync()
```

Returns

[Task](#)

CreatePatternAsync(ElementReference, RepeatPattern)

```
public Task<object> CreatePatternAsync(ElementReference image, RepeatPattern repeat)
```

Parameters

image [ElementReference](#)

repeat [RepeatPattern](#)

Returns

[Task](#)<[object](#)>

DrawFocusIfNeeded(ElementReference)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void DrawFocusIfNeeded(ElementReference elementReference)
```

Parameters

elementReference [ElementReference](#)

DrawFocusIfNeededAsync(ElementReference)

```
public Task DrawFocusIfNeededAsync(ElementReference elementReference)
```

Parameters

elementReference [ElementReference](#)

Returns

[Task](#)

DrawImageAsync(ElementReference, double, double)

```
public Task DrawImageAsync(ElementReference elementReference, double dx, double dy)
```

Parameters

elementReference [ElementReference](#)

dx [double](#)

dy [double](#)

Returns

[Task](#)

DrawImageAsync(ElementReference, double, double, double, double)

```
public Task DrawImageAsync(ElementReference elementReference, double dx, double dy, double dWidth, double dHeight)
```

Parameters

elementReference [ElementReference](#)

dx [double](#)

dy [double](#)

dWidth [double](#)

dHeight [double](#)

Returns

[Task](#)

DrawImageAsync(ElementReference, double, double, double, double)

double, double, double, double, double, double)

```
public Task DrawImageAsync(ElementReference elementReference, double sx, double sy,  
double sWidth, double sHeight, double dx, double dy, double dWidth, double dHeight)
```

Parameters

elementReference [ElementReference](#)

sx [double](#)

sy [double](#)

sWidth [double](#)

sHeight [double](#)

dx [double](#)

dy [double](#)

dWidth [double](#)

dHeight [double](#)

Returns

[Task](#)

Fill()

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void Fill()
```

FillAsync()

```
public Task FillAsync()
```

Returns

[Task](#)

FillRect(double, double, double, double)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void FillRect(double x, double y, double width, double height)
```

Parameters

x [double](#)

y [double](#)

width [double](#)

height [double](#)

FillRectAsync(double, double, double, double)

```
public Task FillRectAsync(double x, double y, double width, double height)
```

Parameters

x [double](#)

y [double](#)

width [double](#)

height [double](#)

Returns

[Task](#)

FillText(string, double, double, double?)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void FillText(string text, double x, double y, double? maxWidth = null)
```

Parameters

text [string](#)

x [double](#)

y [double](#)

maxWidth [double](#)?

FillTextAsync(string, double, double, double?)

```
public Task FillTextAsync(string text, double x, double y, double? maxWidth = null)
```

Parameters

text [string](#)

x [double](#)

y [double](#)

maxWidth [double](#)?

Returns

[Task](#)

GetLineDash()

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public float[] GetLineDash()
```

Returns

[float](#)  []

GetLineDashAsync()

```
public Task<float[]> GetLineDashAsync()
```

Returns

[Task](#)  <[float](#)  []>

IsPointInPath(double, double)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public bool IsPointInPath(double x, double y)
```

Parameters

x [double](#) 

y [double](#) 

Returns

[bool](#) 

IsPointInPathAsync(double, double)

```
public Task<bool> IsPointInPathAsync(double x, double y)
```

Parameters

x [double](#) 

y [double](#) 

Returns

[Task](#) <[bool](#)>

IsPointInStroke(double, double)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public bool IsPointInStroke(double x, double y)
```

Parameters

x [double](#)

y [double](#)

Returns

[bool](#)

IsPointInStrokeAsync(double, double)

```
public Task<bool> IsPointInStrokeAsync(double x, double y)
```

Parameters

x [double](#)

y [double](#)

Returns

[Task](#) <[bool](#)>

LineTo(double, double)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void LineTo(double x, double y)
```

Parameters

x [double](#)

y [double](#)

LineToAsync(double, double)

```
public Task LineToAsync(double x, double y)
```

Parameters

x [double](#)

y [double](#)

Returns

[Task](#)

MeasureText(string)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public TextMetrics MeasureText(string text)
```

Parameters

text [string](#)

Returns

[TextMetrics](#)

MeasureTextAsync(string)

```
public Task<TextMetrics> MeasureTextAsync(string text)
```

Parameters

text [string](#)

Returns

[Task](#) <[TextMetrics](#)>

MoveTo(double, double)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void MoveTo(double x, double y)
```

Parameters

x [double](#)

y [double](#)

MoveToAsync(double, double)

```
public Task MoveToAsync(double x, double y)
```

Parameters

x [double](#)

y [double](#)

Returns

[Task](#)

QuadraticCurveTo(double, double, double, double)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void QuadraticCurveTo(double cpx, double cpy, double x, double y)
```

Parameters

cpx [double](#)

cpy [double](#)

x [double](#)

y [double](#)

QuadraticCurveToAsync(double, double, double, double)

```
public Task QuadraticCurveToAsync(double cpx, double cpy, double x, double y)
```

Parameters

cpx [double](#)

cpy [double](#)

x [double](#)

y [double](#)

Returns

[Task](#)

Rect(double, double, double, double)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void Rect(double x, double y, double width, double height)
```

Parameters

x [double](#)

y [double](#)

width [double](#)

height [double](#)

RectAsync(double, double, double, double)

```
public Task RectAsync(double x, double y, double width, double height)
```

Parameters

x [double](#)

y [double](#)

width [double](#)

height [double](#)

Returns

[Task](#)

Restore()

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void Restore()
```

RestoreAsync()

```
public Task RestoreAsync()
```

Returns

[Task](#)

Rotate(float)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void Rotate(float angle)
```

Parameters

angle [float](#)

RotateAsync(float)

```
public Task RotateAsync(float angle)
```

Parameters

angle [float](#)

Returns

[Task](#)

Save()

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void Save()
```

SaveAsync()

```
public Task SaveAsync()
```


Returns

[Task](#)

Scale(double, double)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void Scale(double x, double y)
```

Parameters

x [double](#)

y [double](#)

ScaleAsync(double, double)

```
public Task ScaleAsync(double x, double y)
```

Parameters

x [double](#)

y [double](#)

Returns

[Task](#)

ScrollPathIntoView()

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void ScrollPathIntoView()
```

ScrollPathIntoViewAsync()

```
public Task ScrollPathIntoViewAsync()
```

Returns

[Task](#)

SetDirectionAsync(TextDirection)

```
public Task SetDirectionAsync(TextDirection value)
```

Parameters

value [TextDirection](#)

Returns

[Task](#)

SetFillStyleAsync(object)

```
public Task SetFillStyleAsync(object value)
```

Parameters

value [object](#)

Returns

[Task](#)

SetFontAsync(string)

```
public Task SetFontAsync(string value)
```

Parameters

value [string](#)[↗]

Returns

[Task](#)[↗]

SetGlobalAlphaAsync(float)

```
public Task SetGlobalAlphaAsync(float value)
```

Parameters

value [float](#)[↗]

Returns

[Task](#)[↗]

SetLineCapAsync(LineCap)

```
public Task SetLineCapAsync(LineCap value)
```

Parameters

value [LineCap](#)

Returns

[Task](#)[↗]

SetLineDash(float[])

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void SetLineDash(float[] segments)
```

Parameters

segments [float](#)[]

SetLineDashAsync(float[])

```
public Task SetLineDashAsync(float[] segments)
```

Parameters

segments [float](#)[]

Returns

[Task](#)

SetLineDashOffsetAsync(float)

```
public Task SetLineDashOffsetAsync(float value)
```

Parameters

value [float](#)

Returns

[Task](#)

SetLineJoinAsync(LineJoin)

```
public Task SetLineJoinAsync(LineJoin value)
```

Parameters

value [LineJoin](#)

Returns

[Task](#)

SetLineWidthAsync(float)

```
public Task SetLineWidthAsync(float value)
```

Parameters

value [float](#)

Returns

[Task](#)

SetMiterLimitAsync(float)

```
public Task SetMiterLimitAsync(float value)
```

Parameters

value [float](#)

Returns

[Task](#)

SetShadowBlurAsync(float)

```
public Task SetShadowBlurAsync(float value)
```

Parameters

value [float](#)

Returns

[Task](#)

SetShadowColorAsync(string)

```
public Task SetShadowColorAsync(string value)
```

Parameters

value [string](#)

Returns

[Task](#)

SetShadowOffsetXAsync(float)

```
public Task SetShadowOffsetXAsync(float value)
```

Parameters

value [float](#)

Returns

[Task](#)

SetShadowOffsetYAsync(float)

```
public Task SetShadowOffsetYAsync(float value)
```

Parameters

value [float](#)

Returns

[Task](#)

SetStrokeStyleAsync(string)

```
public Task SetStrokeStyleAsync(string value)
```

Parameters

value [string](#)

Returns

[Task](#)

SetTextAlignAsync(TextAlign)

```
public Task SetTextAlignAsync(TextAlign value)
```

Parameters

value [TextAlign](#)

Returns

[Task](#)

SetTextBaselineAsync(TextBaseline)

```
public Task SetTextBaselineAsync(TextBaseline value)
```

Parameters

value [TextBaseline](#)

Returns

[Task](#)

SetTransform(double, double, double, double, double, double)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void SetTransform(double m11, double m12, double m21, double m22, double dx,  
double dy)
```

Parameters

m11 [double](#)

m12 [double](#)

m21 [double](#)

m22 [double](#)

dx [double](#)

dy [double](#)

SetTransformAsync(double, double, double, double, double, double)

```
public Task SetTransformAsync(double m11, double m12, double m21, double m22, double  
dx, double dy)
```

Parameters

m11 [double](#)

m12 [double](#)

m21 [double](#)

m22 [double](#)

dx [double](#)

dy [double](#)

Returns

[Task](#)

Stroke()

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void Stroke()
```

StrokeAsync()

```
public Task StrokeAsync()
```

Returns

[Task](#)

StrokeRect(double, double, double, double)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void StrokeRect(double x, double y, double width, double height)
```

Parameters

x [double](#)

y [double](#)

width [double](#)

height [double](#)

StrokeRectAsync(double, double, double, double)

```
public Task StrokeRectAsync(double x, double y, double width, double height)
```

Parameters

x [double](#)

y [double](#)

width [double](#)

height [double](#)

Returns

[Task](#)

StrokeText(string, double, double, double?)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void StrokeText(string text, double x, double y, double? maxWidth = null)
```

Parameters

text [string](#)

x [double](#)

y [double](#)

maxWidth [double](#)?

StrokeTextAsync(string, double, double, double?)

```
public Task StrokeTextAsync(string text, double x, double y, double? maxWidth  
= null)
```

Parameters

text [string](#)

x [double](#)

y [double](#)

maxWidth [double](#)?

Returns

[Task](#)

Transform(double, double, double, double, double, double)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void Transform(double m11, double m12, double m21, double m22, double dx,  
double dy)
```

Parameters

m11 [double](#)

m12 [double](#)

m21 [double](#)

m22 [double](#)

dx [double](#)

dy [double](#)

TransformAsync(double, double, double, double, double,

double)

```
public Task TransformAsync(double m11, double m12, double m21, double m22, double dx, double dy)
```

Parameters

m11 [double](#)

m12 [double](#)

m21 [double](#)

m22 [double](#)

dx [double](#)

dy [double](#)

Returns

[Task](#)

Translate(double, double)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void Translate(double x, double y)
```

Parameters

x [double](#)

y [double](#)

TranslateAsync(double, double)

```
public Task TranslateAsync(double x, double y)
```

Parameters

x [double](#)

y [double](#)

Returns

[Task](#)

Enum LineCap

Namespace: [Blazor.Extensions.Canvas.Canvas2D](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum LineCap
```

Fields

Butt = 0

Round = 1

Square = 2

Enum LineJoin

Namespace: [Blazor.Extensions.Canvas.Canvas2D](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum LineJoin
```

Fields

```
Bevel = 2
```

```
Miter = 0
```

```
Round = 1
```

Enum RepeatPattern

Namespace: [Blazor.Extensions.Canvas.Canvas2D](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum RepeatPattern
```

Fields

NoRepeat = 3

Repeat = 0

RepeatX = 1

RepeatY = 2

Enum TextAlign

Namespace: [Blazor.Extensions.Canvas.Canvas2D](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum TextAlign
```

Fields

Center = 4

End = 1

Left = 2

Right = 3

Start = 0

Enum TextBaseline

Namespace: [Blazor.Extensions.Canvas.Canvas2D](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum TextBaseline
```

Fields

Alphabetic = 0

Bottom = 5

Hanging = 2

Ideographic = 4

Middle = 3

Top = 1

Enum TextDirection

Namespace: [Blazor.Extensions.Canvas.Canvas2D](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum TextDirection
```

Fields

```
Inherit = 0
```

```
LTR = 1
```

```
RTL = 2
```

Namespace Blazor.Extensions.Canvas.Model

Classes

[TextMetrics](#)

Class TextMetrics

Namespace: [Blazor.Extensions.Canvas.Model](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public class TextMetrics
```

Inheritance

[object](#)  ← TextMetrics

Inherited Members

[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

TextMetrics()

```
public TextMetrics()
```

Properties

ActualBoundingBoxAscent

```
public double ActualBoundingBoxAscent { get; set; }
```

Property Value

[double](#) 

ActualBoundingBoxDescent

```
public double ActualBoundingBoxDescent { get; set; }
```

Property Value

[double](#)

ActualBoundingBoxLeft

```
public double ActualBoundingBoxLeft { get; set; }
```

Property Value

[double](#)

ActualBoundingBoxRight

```
public double ActualBoundingBoxRight { get; set; }
```

Property Value

[double](#)

AlphabeticBaseline

```
public double AlphabeticBaseline { get; set; }
```

Property Value

[double](#)

EmHeightAscent

```
public double EmHeightAscent { get; set; }
```

Property Value

[double](#)

EmHeightDescent

```
public double EmHeightDescent { get; set; }
```

Property Value

[double](#)

FontBoundingBoxAscent

```
public double FontBoundingBoxAscent { get; set; }
```

Property Value

[double](#)

FontBoundingBoxDescent

```
public double FontBoundingBoxDescent { get; set; }
```

Property Value

[double](#)

HangingBaseline

```
public double HangingBaseline { get; set; }
```

Property Value

[double](#)

IdeographicBaseline

```
public double IdeographicBaseline { get; set; }
```

Property Value

[double](#)

Width

```
public double Width { get; set; }
```

Property Value

[double](#)

Namespace Blazor.Extensions.Canvas.WebGL

Classes

[WebGLActiveInfo](#)

[WebGLBuffer](#)

[WebGLContext](#)

[WebGLContextAttributes](#)

[WebGLFramebuffer](#)

[WebGLObject](#)

[WebGLProgram](#)

[WebGLRenderbuffer](#)

[WebGLShader](#)

[WebGLShaderPrecisionFormat](#)

[WebGLTexture](#)

[WebGLUniformLocation](#)

Enums

[BlendingEquation](#)

[BlendingMode](#)

[BufferBits](#)

[BufferParameter](#)

[BufferType](#)

[BufferUsageHint](#)

[CompareFunction](#)

[DataType](#)

[EnableCap](#)

[Error](#)

[Face](#)

[FramebufferAttachment](#)

[FramebufferAttachmentParameter](#)

[FramebufferStatus](#)

[FramebufferType](#)

[FrontFaceDirection](#)

[HintMode](#)

[HintTarget](#)

[Parameter](#)

[PixelFormat](#)

[PixelStorageMode](#)

[PixelType](#)

[Primitive](#)

[ProgramParameter](#)

[RenderbufferFormat](#)

[RenderbufferParameter](#)

[RenderbufferType](#)

[ShaderParameter](#)

[ShaderPrecision](#)

[ShaderType](#)

[StencilFunction](#)

[Texture](#)

[Texture2DType](#)

[TextureParameter](#)

[TextureParameterValue](#)

[TextureType](#)

[UniformType](#)

[VertexAttribute](#)

[VertexAttributePointer](#)

Enum BlendingEquation

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum BlendingEquation
```

Fields

```
FUNC_ADD = 32774
```

```
FUNC_REVERSE_SUBTRACT = 32779
```

```
FUNC_SUBTRACT = 32778
```

Enum BlendingMode

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum BlendingMode
```

Fields

CONSTANT_ALPHA = 32771

CONSTANT_COLOR = 32769

DST_ALPHA = 772

DST_COLOR = 774

ONE = 1

ONE_MINUS_CONSTANT_ALPHA = 32772

ONE_MINUS_CONSTANT_COLOR = 32770

ONE_MINUS_DST_ALPHA = 773

ONE_MINUS_DST_COLOR = 775

ONE_MINUS_SRC_ALPHA = 771

ONE_MINUS_SRC_COLOR = 769

SRC_ALPHA = 770

SRC_ALPHA_SATURATE = 776

SRC_COLOR = 768

ZERO = 0

Enum BufferBits

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum BufferBits
```

Fields

```
COLOR_BUFFER_BIT = 16384
```

```
DEPTH_BUFFER_BIT = 256
```

```
STENCIL_BUFFER_BIT = 1024
```

Enum BufferParameter

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum BufferParameter
```

Fields

```
BUFFER_SIZE = 34660
```

```
BUFFER_USAGE = 34661
```

Enum BufferType

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum BufferType
```

Fields

```
ARRAY_BUFFER = 34962
```

```
ELEMENT_ARRAY_BUFFER = 34963
```


Enum BufferUsageHint

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum BufferUsageHint
```

Fields

```
DYNAMIC_DRAW = 35048
```

```
STATIC_DRAW = 35044
```

```
STREAM_DRAW = 35040
```

Enum CompareFunction

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum CompareFunction
```

Fields

ALWAYS = 519

EQUAL = 514

GEQUAL = 518

GREATER = 516

LEQUAL = 515

LESS = 513

NEVER = 512

NOTEQUAL = 517

Enum DataType

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum DataType
```

Fields

BYTE = 5120

FLOAT = 5126

INT = 5124

SHORT = 5122

UNSIGNED_BYTE = 5121

UNSIGNED_INT = 5125

UNSIGNED_SHORT = 5123

Enum EnableCap

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum EnableCap
```

Fields

BLEND = 3042

CULL_FACE = 2884

DEPTH_TEST = 2929

DITHER = 3024

POLYGON_OFFSET_FILL = 32823

SAMPLE_ALPHA_TO_COVERAGE = 32926

SAMPLE_COVERAGE = 32928

SCISSOR_TEST = 3089

STENCIL_TEST = 2960

Enum Error

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum Error
```

Fields

```
CONTEXT_LOST_WEBGL = 37442
```

```
INVALID_ENUM = 1280
```

```
INVALID_OPERATION = 1282
```

```
INVALID_VALUE = 1281
```

```
NO_ERROR = 0
```

```
OUT_OF_MEMORY = 1285
```

Enum Face

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum Face
```

Fields

```
BACK = 1029
```

```
FRONT = 1028
```

```
FRONT_AND_BACK = 1032
```

Enum FramebufferAttachment

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum FramebufferAttachment
```

Fields

```
COLOR_ATTACHMENT0 = 36064
```

```
DEPTH_ATTACHMENT = 36096
```

```
DEPTH_STENCIL_ATTACHMENT = 33306
```

```
STENCIL_ATTACHMENT = 36128
```

Enum FramebufferAttachmentParameter

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum FramebufferAttachmentParameter
```

Fields

```
FRAMEBUFFER_ATTACHMENT_OBJECT_NAME = 36049
```

```
FRAMEBUFFER_ATTACHMENT_OBJECT_TYPE = 36048
```

```
FRAMEBUFFER_ATTACHMENT_TEXTURE_CUBE_MAP_FACE = 36051
```

```
FRAMEBUFFER_ATTACHMENT_TEXTURE_LEVEL = 36050
```


Enum FramebufferStatus

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum FramebufferStatus
```

Fields

```
FRAMEBUFFER_COMPLETE = 36053
```

```
FRAMEBUFFER_INCOMPLETE_ATTACHMENT = 36054
```

```
FRAMEBUFFER_INCOMPLETE_DIMENSIONS = 36057
```

```
FRAMEBUFFER_INCOMPLETE_MISSING_ATTACHMENT = 36055
```

```
FRAMEBUFFER_UNSUPPORTED = 36061
```

```
NONE = 0
```

Enum FramebufferType

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum FramebufferType
```

Fields

```
FRAMEBUFFER = 36160
```

Enum FrontFaceDirection

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum FrontFaceDirection
```

Fields

CCW = 2305

CW = 2304

Enum HintMode

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum HintMode
```

Fields

DONT_CARE = 4352

FASTEST = 4353

NICEST = 4354

Enum HintTarget

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum HintTarget
```

Fields

```
GENERATE_MIPMAP_HINT = 33170
```

Enum Parameter

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum Parameter
```

Fields

ACTIVE_TEXTURE = 34016

ALIASED_LINE_WIDTH_RANGE = 33902

ALIASED_POINT_SIZE_RANGE = 33901

ALPHA_BITS = 3413

ARRAY_BUFFER_BINDING = 34964

BLEND_COLOR = 32773

BLEND_DST_ALPHA = 32970

BLEND_DST_RGB = 32968

BLEND_EQUATION = 32777

BLEND_EQUATION_ALPHA = 34877

BLEND_EQUATION_RGB = 32777

BLEND_SRC_ALPHA = 32971

BLEND_SRC_RGB = 32969

BLUE_BITS = 3412

BROWSER_DEFAULT_WEBGL = 37444

COLOR_CLEAR_VALUE = 3106

COLOR_WRITEMASK = 3107

COMPRESSED_TEXTURE_FORMATS = 34467

CULL_FACE_MODE = 2885

CURRENT_PROGRAM = 35725

DEPTH_BITS = 3414

DEPTH_CLEAR_VALUE = 2931

DEPTH_FUNC = 2932

DEPTH_RANGE = 2928

DEPTH_WRITEMASK = 2930

ELEMENT_ARRAY_BUFFER_BINDING = 34965

FRAMEBUFFER_BINDING = 36006

FRONT_FACE = 2886

GREEN_BITS = 3411

IMPLEMENTATION_COLOR_READ_FORMAT = 35739

IMPLEMENTATION_COLOR_READ_TYPE = 35738

LINE_WIDTH = 2849

MAX_COMBINED_TEXTURE_IMAGE_UNITS = 35661

MAX_FRAGMENT_UNIFORM_VECTORS = 36349

MAX_RENDERBUFFER_SIZE = 34024

MAX_TEXTURE_IMAGE_UNITS = 34930

MAX_TEXTURE_SIZE = 3379

MAX_VARYING_VECTORS = 36348

MAX_VERTEX_ATTRIBS = 34921

MAX_VERTEX_TEXTURE_IMAGE_UNITS = 35660

MAX_VERTEX_UNIFORM_VECTORS = 36347

MAX_VIEWPORT_DIMS = 3386

PACK_ALIGNMENT = 3333

POLYGON_OFFSET_FACTOR = 32824

POLYGON_OFFSET_UNITS = 10752

RED_BITS = 3410

RENDERBUFFER_BINDING = 36007

RENDERER = 7937

SAMPLES = 32937

SAMPLE_BUFFERS = 32936

SAMPLE_COVERAGE_INVERT = 32939

SAMPLE_COVERAGE_VALUE = 32938

SCISSOR_BOX = 3088

SHADING_LANGUAGE_VERSION = 35724

STENCIL_BACK_FAIL = 34817

STENCIL_BACK_FUNC = 34816

STENCIL_BACK_PASS_DEPTH_FAIL = 34818

STENCIL_BACK_PASS_DEPTH_PASS = 34819

STENCIL_BACK_REF = 36003

STENCIL_BACK_VALUE_MASK = 36004

STENCIL_BACK_WRITEMASK = 36005

STENCIL_BITS = 3415

STENCIL_CLEAR_VALUE = 2961

STENCIL_FAIL = 2964

STENCIL_FUNC = 2962

STENCIL_PASS_DEPTH_FAIL = 2965

STENCIL_PASS_DEPTH_PASS = 2966

STENCIL_REF = 2967

STENCIL_VALUE_MASK = 2963

STENCIL_WRITEMASK = 2968

SUBPIXEL_BITS = 3408

TEXTURE_BINDING_2D = 32873

UNPACK_ALIGNMENT = 3317

VENDOR = 7936

VERSION = 7938

VIEWPORT = 2978

Enum PixelFormat

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum PixelFormat
```

Fields

ALPHA = 6406

LUMINANCE = 6409

LUMINANCE_ALPHA = 6410

RGB = 6407

RGBA = 6408

Enum PixelStorageMode

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum PixelStorageMode
```

Fields

```
UNPACK_COLORSPACE_CONVERSION_WEBGL = 37443
```

```
UNPACK_FLIP_Y_WEBGL = 37440
```

```
UNPACK_PREMULTIPLY_ALPHA_WEBGL = 37441
```

Enum PixelType

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum PixelType
```

Fields

```
FLOAT = 5126
```

```
UNSIGNED_BYTE = 5121
```

```
UNSIGNED_SHORT_4_4_4_4 = 32819
```

```
UNSIGNED_SHORT_5_5_5_1 = 32820
```

```
UNSIGNED_SHORT_5_6_5 = 33635
```

Enum Primitive

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum Primitive
```

Fields

```
LINES = 1
```

```
LINE_LOOP = 2
```

```
LINE_STRIP = 3
```

```
POINTS = 0
```

```
TRIANGLES = 4
```

```
TRIANGLE_FAN = 6
```

```
TRIANGLE_STRIP = 5
```

Enum ProgramParameter

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum ProgramParameter
```

Fields

ACTIVE_ATTRIBUTES = 35721

ACTIVE_UNIFORMS = 35718

ATTACHED_SHADERS = 35717

DELETE_STATUS = 35712

LINK_STATUS = 35714

VALIDATE_STATUS = 35715

Enum RenderbufferFormat

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum RenderbufferFormat
```

Fields

DEPTH_COMPONENT16 = 33189

DEPTH_STENCIL = 34041

RGB565 = 36194

RGB5_A1 = 32855

RGBA4 = 32854

STENCIL_INDEX8 = 36168

Enum RenderbufferParameter

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum RenderbufferParameter
```

Fields

```
RENDERBUFFER_ALPHA_SIZE = 36179
```

```
RENDERBUFFER_BLUE_SIZE = 36178
```

```
RENDERBUFFER_DEPTH_SIZE = 36180
```

```
RENDERBUFFER_GREEN_SIZE = 36177
```

```
RENDERBUFFER_HEIGHT = 36163
```

```
RENDERBUFFER_INTERNAL_FORMAT = 36164
```

```
RENDERBUFFER_RED_SIZE = 36176
```

```
RENDERBUFFER_STENCIL_SIZE = 36181
```

```
RENDERBUFFER_WIDTH = 36162
```


Enum RenderbufferType

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum RenderbufferType
```

Fields

```
RENDERBUFFER = 36161
```

Enum ShaderParameter

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum ShaderParameter
```

Fields

```
COMPILE_STATUS = 35713
```

```
DELETE_STATUS = 35712
```

```
SHADER_TYPE = 35663
```

Enum ShaderPrecision

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum ShaderPrecision
```

Fields

```
HIGH_FLOAT = 36338
```

```
HIGH_INT = 36341
```

```
LOW_FLOAT = 36336
```

```
LOW_INT = 36339
```

```
MEDIUM_FLOAT = 36337
```

```
MEDIUM_INT = 36340
```

Enum ShaderType

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum ShaderType
```

Fields

```
FRAGMENT_SHADER = 35632
```

```
VERTEX_SHADER = 35633
```

Enum StencilFunction

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum StencilFunction
```

Fields

DECR = 7683

DECR_WRAP = 34056

INCR = 7682

INCR_WRAP = 34055

INVERT = 5386

KEEP = 7680

REPLACE = 7681

Enum Texture

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum Texture
```

Fields

TEXTURE0 = 33984

TEXTURE1 = 33985

TEXTURE10 = 33994

TEXTURE11 = 33995

TEXTURE12 = 33996

TEXTURE13 = 33997

TEXTURE14 = 33998

TEXTURE15 = 33999

TEXTURE16 = 34000

TEXTURE17 = 34001

TEXTURE18 = 34002

TEXTURE19 = 34003

TEXTURE2 = 33986

TEXTURE20 = 34004

TEXTURE21 = 34005

TEXTURE22 = 34006

TEXTURE23 = 34007

TEXTURE24 = 34008

TEXTURE25 = 34009

TEXTURE26 = 34010

TEXTURE27 = 34011

TEXTURE28 = 34012

TEXTURE29 = 34013

TEXTURE3 = 33987

TEXTURE30 = 34014

TEXTURE31 = 34015

TEXTURE4 = 33988

TEXTURE5 = 33989

TEXTURE6 = 33990

TEXTURE7 = 33991

TEXTURE8 = 33992

TEXTURE9 = 33993

Enum Texture2DType

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum Texture2DType
```

Fields

```
TEXTURE_2D = 3553
```

```
TEXTURE_CUBE_MAP_NEGATIVE_X = 34070
```

```
TEXTURE_CUBE_MAP_NEGATIVE_Y = 34072
```

```
TEXTURE_CUBE_MAP_NEGATIVE_Z = 34074
```

```
TEXTURE_CUBE_MAP_POSITIVE_X = 34069
```

```
TEXTURE_CUBE_MAP_POSITIVE_Y = 34071
```

```
TEXTURE_CUBE_MAP_POSITIVE_Z = 34073
```


Enum TextureParameter

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum TextureParameter
```

Fields

```
TEXTURE_MAG_FILTER = 10240
```

```
TEXTURE_MIN_FILTER = 10241
```

```
TEXTURE_WRAP_S = 10242
```

```
TEXTURE_WRAP_T = 10243
```

Enum TextureParameterValue

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum TextureParameterValue
```

Fields

```
CLAMP_TO_EDGE = 33071
```

```
LINEAR = 9729
```

```
LINEAR_MIPMAP_LINEAR = 9987
```

```
LINEAR_MIPMAP_NEAREST = 9985
```

```
MIRRORED_REPEAT = 33648
```

```
NEAREST = 9728
```

```
NEAREST_MIPMAP_LINEAR = 9986
```

```
NEAREST_MIPMAP_NEAREST = 9984
```

```
REPEAT = 10497
```

Enum TextureType

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum TextureType
```

Fields

```
TEXTURE_2D = 3553
```

```
TEXTURE_CUBE_MAP = 34067
```

Enum UniformType

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum UniformType
```

Fields

BOOL = 35670

BOOL_VEC2 = 35671

BOOL_VEC3 = 35672

BOOL_VEC4 = 35673

FLOAT_MAT2 = 35674

FLOAT_MAT3 = 35675

FLOAT_MAT4 = 35676

FLOAT_VEC2 = 35664

FLOAT_VEC3 = 35665

FLOAT_VEC4 = 35666

INT_VEC2 = 35667

INT_VEC3 = 35668

INT_VEC4 = 35669

SAMPLER_2D = 35678

SAMPLER_CUBE = 35680

Enum VertexAttribute

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum VertexAttribute
```

Fields

```
CURRENT_VERTEX_ATTRIB = 34342
```

```
VERTEX_ATTRIB_ARRAY_BUFFER_BINDING = 34975
```

```
VERTEX_ATTRIB_ARRAY_ENABLED = 34338
```

```
VERTEX_ATTRIB_ARRAY_NORMALIZED = 34922
```

```
VERTEX_ATTRIB_ARRAY_SIZE = 34339
```

```
VERTEX_ATTRIB_ARRAY_STRIDE = 34340
```

```
VERTEX_ATTRIB_ARRAY_TYPE = 34341
```

Enum VertexAttribPointer

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public enum VertexAttribPointer
```

Fields

```
VERTEX_ATTRIB_ARRAY_POINTER = 34373
```

Class WebGLActiveInfo

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public class WebGLActiveInfo
```

Inheritance

[object](#)  ← WebGLActiveInfo

Inherited Members

[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

WebGLActiveInfo()

```
public WebGLActiveInfo()
```

Properties

Name

```
public string Name { get; set; }
```

Property Value

[string](#) 

Size

```
public int Size { get; set; }
```

Property Value

[int](#)

Type

```
public UniformType Type { get; set; }
```

Property Value

[UniformType](#)

Class WebGLBuffer

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public class WebGLBuffer : WebGLObject
```

Inheritance

[object](#)  ← [WebGLObject](#) ← WebGLBuffer

Inherited Members

[WebGLObject.WebGLType](#) , [WebGLObject.Id](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  ,
[object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  ,
[object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

WebGLBuffer()

```
public WebGLBuffer()
```

Class WebGLContext

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public class WebGLContext : RenderingContext, IDisposable
```

Inheritance

[object](#)  ← [RenderingContext](#) ← WebGLContext

Implements

[IDisposable](#) 

Inherited Members

[RenderingContext.BeginBatchAsync\(\)](#), [RenderingContext.EndBatchAsync\(\)](#),
[RenderingContext.BatchCallAsync\(string, bool, params object\[\]\)](#),
[RenderingContext.GetPropertyAsync<T>\(string\)](#), [RenderingContext.CallMethod<T>\(string\)](#),
[RenderingContext.CallMethodAsync<T>\(string\)](#),
[RenderingContext.CallMethod<T>\(string, params object\[\]\)](#),
[RenderingContext.CallMethodAsync<T>\(string, params object\[\]\)](#), [RenderingContext.Dispose\(\)](#),
[RenderingContext.Canvas](#), [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ToString\(\)](#) ,
[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) ,
[object.GetHashCode\(\)](#) 

Properties

DrawingBufferHeight

```
public int DrawingBufferHeight { get; }
```

Property Value

[int](#) 

DrawingBufferWidth

```
public int DrawingBufferWidth { get; }
```

Property Value

[int](#)

Methods

ActiveTexture(Texture)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void ActiveTexture(Texture texture)
```

Parameters

texture [Texture](#)

ActiveTextureAsync(Texture)

```
public Task ActiveTextureAsync(Texture texture)
```

Parameters

texture [Texture](#)

Returns

[Task](#)

AttachShader(WebGLProgram, WebGLShader)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void AttachShader(WebGLProgram program, WebGLShader shader)
```

Parameters

program [WebGLProgram](#)

shader [WebGLShader](#)

AttachShaderAsync(WebGLProgram, WebGLShader)

```
public Task AttachShaderAsync(WebGLProgram program, WebGLShader shader)
```

Parameters

program [WebGLProgram](#)

shader [WebGLShader](#)

Returns

[Task](#)

BindAttribLocation(WebGLProgram, uint, string)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void BindAttribLocation(WebGLProgram program, uint index, string name)
```

Parameters

program [WebGLProgram](#)

index [uint](#)

name [string](#)

BindAttribLocationAsync(WebGLProgram, uint, string)

```
public Task BindAttribLocationAsync(WebGLProgram program, uint index, string name)
```

Parameters

program [WebGLProgram](#)

index [uint](#)

name [string](#)

Returns

[Task](#)

BindBuffer(BufferType, WebGLBuffer)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void BindBuffer(BufferType target, WebGLBuffer buffer)
```

Parameters

target [BufferType](#)

buffer [WebGLBuffer](#)

BindBufferAsync(BufferType, WebGLBuffer)

```
public Task BindBufferAsync(BufferType target, WebGLBuffer buffer)
```

Parameters

target [BufferType](#)

buffer [WebGLBuffer](#)

Returns

[Task](#)

BindFramebuffer(FramebufferType, WebGLFramebuffer)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void BindFramebuffer(FramebufferType target, WebGLFramebuffer framebuffer)
```

Parameters

target [FramebufferType](#)

framebuffer [WebGLFramebuffer](#)

BindFramebufferAsync(FramebufferType, WebGLFramebuffer)

```
public Task BindFramebufferAsync(FramebufferType target,  
WebGLFramebuffer framebuffer)
```

Parameters

target [FramebufferType](#)

framebuffer [WebGLFramebuffer](#)

Returns

[Task](#)

BindRenderbuffer(RenderbufferType, WebGLRenderbuffer)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void BindRenderbuffer(RenderbufferType target, WebGLRenderbuffer  
renderbuffer)
```

Parameters

target [RenderbufferType](#)

renderbuffer [WebGLRenderbuffer](#)

BindRenderbufferAsync(RenderbufferType, WebGLRenderbuffer)

```
public Task BindRenderbufferAsync(RenderbufferType target,
    WebGLRenderbuffer renderbuffer)
```

Parameters

target [RenderbufferType](#)

renderbuffer [WebGLRenderbuffer](#)

Returns

[Task](#)

BindTexture(TextureType, WebGLTexture)

```
[Obsolete("Use the async version instead, which is already called internally.")]
public void BindTexture(TextureType type, WebGLTexture texture)
```

Parameters

type [TextureType](#)

texture [WebGLTexture](#)

BindTextureAsync(TextureType, WebGLTexture)

```
public Task BindTextureAsync(TextureType type, WebGLTexture texture)
```

Parameters

type [TextureType](#)

texture [WebGLTexture](#)

Returns

[Task](#)

BlendColor(float, float, float, float)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void BlendColor(float red, float green, float blue, float alpha)
```

Parameters

red [float](#)

green [float](#)

blue [float](#)

alpha [float](#)

BlendColorAsync(float, float, float, float)

```
public Task BlendColorAsync(float red, float green, float blue, float alpha)
```

Parameters

red [float](#)

green [float](#)

blue [float](#)

alpha [float](#)

Returns

[Task](#)

BlendEquation(BlendingEquation)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void BlendEquation(BlendingEquation equation)
```

Parameters

equation [BlendingEquation](#)

BlendEquationAsync(BlendingEquation)

```
public Task BlendEquationAsync(BlendingEquation equation)
```

Parameters

equation [BlendingEquation](#)

Returns

[Task](#)

BlendEquationSeparate(BlendingEquation, BlendingEquation)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void BlendEquationSeparate(BlendingEquation modeRGB,  
BlendingEquation modeAlpha)
```

Parameters

modeRGB [BlendingEquation](#)

modeAlpha [BlendingEquation](#)

BlendEquationSeparateAsync(BlendingEquation, BlendingEquation)

```
public Task BlendEquationSeparateAsync(BlendingEquation modeRGB,  
BlendingEquation modeAlpha)
```

Parameters

modeRGB [BlendingEquation](#)

modeAlpha [BlendingEquation](#)

Returns

[Task](#)

BlendFunc(BlendingMode, BlendingMode)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void BlendFunc(BlendingMode sfactor, BlendingMode dfactor)
```

Parameters

sfactor [BlendingMode](#)

dfactor [BlendingMode](#)

BlendFuncAsync(BlendingMode, BlendingMode)

```
public Task BlendFuncAsync(BlendingMode sfactor, BlendingMode dfactor)
```

Parameters

sfactor [BlendingMode](#)

dfactor [BlendingMode](#)

Returns

[Task](#) 

BlendFuncSeparate(BlendingMode, BlendingMode, BlendingMode, BlendingMode)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void BlendFuncSeparate(BlendingMode srcRGB, BlendingMode dstRGB, BlendingMode  
srcAlpha, BlendingMode dstAlpha)
```

Parameters

srcRGB [BlendingMode](#)

dstRGB [BlendingMode](#)

srcAlpha [BlendingMode](#)

dstAlpha [BlendingMode](#)

BlendFuncSeparateAsync(BlendingMode, BlendingMode, BlendingMode, BlendingMode)

```
public Task BlendFuncSeparateAsync(BlendingMode srcRGB, BlendingMode dstRGB,  
BlendingMode srcAlpha, BlendingMode dstAlpha)
```

Parameters

srcRGB [BlendingMode](#)

dstRGB [BlendingMode](#)

srcAlpha [BlendingMode](#)

dstAlpha [BlendingMode](#)

Returns

[Task](#)

BufferData(BufferType, int, BufferUsageHint)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void BufferData(BufferType target, int size, BufferUsageHint usage)
```

Parameters

target [BufferType](#)

size [int](#)

usage [BufferUsageHint](#)

BufferDataAsync(BufferType, int, BufferUsageHint)

```
public Task BufferDataAsync(BufferType target, int size, BufferUsageHint usage)
```

Parameters

target [BufferType](#)

size [int](#)

usage [BufferUsageHint](#)

Returns

[Task](#)

BufferDataAsync<T>(BufferType, T[], BufferUsageHint)

```
public Task BufferDataAsync<T>(BufferType target, T[] data, BufferUsageHint usage)
```

Parameters

target [BufferType](#)

data T[]

usage [BufferUsageHint](#)

Returns

[Task](#)

Type Parameters

T

BufferData<T>(BufferType, T[], BufferUsageHint)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void BufferData<T>(BufferType target, T[] data, BufferUsageHint usage)
```

Parameters

target [BufferType](#)

data T[]

usage [BufferUsageHint](#)

Type Parameters

T

BufferSubDataAsync<T>(BufferType, uint, T[])

```
public Task BufferSubDataAsync<T>(BufferType target, uint offset, T[] data)
```

Parameters

target [BufferType](#)

offset [uint](#)

data T[]

Returns

[Task](#)

Type Parameters

T

BufferSubData<T>(BufferType, uint, T[])

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void BufferSubData<T>(BufferType target, uint offset, T[] data)
```

Parameters

target [BufferType](#)

offset [uint](#)

data T[]

Type Parameters

T

CheckFramebufferStatus(FramebufferType)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public FramebufferStatus CheckFramebufferStatus(FramebufferType target)
```

Parameters

target [FramebufferType](#)

Returns

[FramebufferStatus](#)

CheckFramebufferStatusAsync(FramebufferType)

```
public Task<FramebufferStatus> CheckFramebufferStatusAsync(FramebufferType target)
```

Parameters

target [FramebufferType](#)

Returns

[Task](#) [<FramebufferStatus>](#)

Clear(BufferBits)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void Clear(BufferBits mask)
```

Parameters

mask [BufferBits](#)

ClearAsync(BufferBits)

```
public Task ClearAsync(BufferBits mask)
```

Parameters

mask [BufferBits](#)

Returns

[Task](#)

ClearColor(float, float, float, float)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void ClearColor(float red, float green, float blue, float alpha)
```

Parameters

red [float](#)

green [float](#)

blue [float](#)

alpha [float](#)

ClearColorAsync(float, float, float, float)

```
public Task ClearColorAsync(float red, float green, float blue, float alpha)
```

Parameters

red [float](#)

green [float](#)

blue [float](#)

alpha [float](#)

Returns

[Task](#)

ClearDepth(float)


```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void ClearDepth(float depth)
```

Parameters

depth [float](#)

ClearDepthAsync(float)

```
public Task ClearDepthAsync(float depth)
```

Parameters

depth [float](#)

Returns

[Task](#)

ClearStencil(int)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void ClearStencil(int stencil)
```

Parameters

stencil [int](#)

ClearStencilAsync(int)

```
public Task ClearStencilAsync(int stencil)
```

Parameters

stencil [int](#)

Returns

[Task](#)

ColorMask(bool, bool, bool, bool)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void ColorMask(bool red, bool green, bool blue, bool alpha)
```

Parameters

red [bool](#)

green [bool](#)

blue [bool](#)

alpha [bool](#)

ColorMaskAsync(bool, bool, bool, bool)

```
public Task ColorMaskAsync(bool red, bool green, bool blue, bool alpha)
```

Parameters

red [bool](#)

green [bool](#)

blue [bool](#)

alpha [bool](#)

Returns

[Task](#)

CompileShader(WebGLShader)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void CompileShader(WebGLShader shader)
```

Parameters

shader [WebGLShader](#)

CompileShaderAsync(WebGLShader)

```
public Task CompileShaderAsync(WebGLShader shader)
```

Parameters

shader [WebGLShader](#)

Returns

[Task](#)

CopyTexImage2D(Texture2DType, int, PixelFormat, int, int, int, int, int)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void CopyTexImage2D(Texture2DType target, int level, PixelFormat format, int  
x, int y, int width, int height, int border)
```

Parameters

target [Texture2DType](#)

level [int](#)

format [PixelFormat](#)

x [int](#)

y [int](#)

width [int](#)

height [int](#)

border [int](#)

CopyTexImage2DAsync(Texture2DType, int, PixelFormat, int, int, int, int, int)

```
public Task CopyTexImage2DAsync(Texture2DType target, int level, PixelFormat format,
int x, int y, int width, int height, int border)
```

Parameters

target [Texture2DType](#)

level [int](#)

format [PixelFormat](#)

x [int](#)

y [int](#)

width [int](#)

height [int](#)

border [int](#)

Returns

[Task](#)

CopyTexSubImage2D(Texture2DType, int, int, int, int, int, int, int, int)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void CopyTexSubImage2D(Texture2DType target, int level, int xoffset, int  
yoffset, int x, int y, int width, int height)
```

Parameters

target [Texture2DType](#)

level [int](#)

xoffset [int](#)

yoffset [int](#)

x [int](#)

y [int](#)

width [int](#)

height [int](#)

CopyTexSubImage2DAsync(Texture2DType, int, int, int, int, int, int, int)

```
public Task CopyTexSubImage2DAsync(Texture2DType target, int level, int xoffset, int  
yoffset, int x, int y, int width, int height)
```

Parameters

target [Texture2DType](#)

level [int](#)

xoffset [int](#)

yoffset [int](#)

x [int](#)

y [int](#)

width [int](#)

height [int](#)

Returns

[Task](#)

CreateBuffer()

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public WebGLBuffer CreateBuffer()
```

Returns

[WebGLBuffer](#)

CreateBufferAsync()

```
public Task<WebGLBuffer> CreateBufferAsync()
```

Returns

[Task](#) <[WebGLBuffer](#)>

CreateFramebuffer()

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public WebGLFramebuffer CreateFramebuffer()
```

Returns

[WebGLFramebuffer](#)

CreateFramebufferAsync()

```
public Task<WebGLFramebuffer> CreateFramebufferAsync()
```

Returns

[Task](#) [<WebGLFramebuffer>](#)

CreateProgram()

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public WebGLProgram CreateProgram()
```

Returns

[WebGLProgram](#)

CreateProgramAsync()

```
public Task<WebGLProgram> CreateProgramAsync()
```

Returns

[Task](#) [<WebGLProgram>](#)

CreateRenderbuffer()

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public WebGLRenderbuffer CreateRenderbuffer()
```

Returns

[WebGLRenderbuffer](#)

CreateRenderbufferAsync()

```
public Task<WebGLRenderbuffer> CreateRenderbufferAsync()
```

Returns

[Task](#) [<WebGLRenderbuffer>](#)

CreateShader(ShaderType)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public WebGLShader CreateShader(ShaderType type)
```

Parameters

type [ShaderType](#)

Returns

[WebGLShader](#)

CreateShaderAsync(ShaderType)

```
public Task<WebGLShader> CreateShaderAsync(ShaderType type)
```

Parameters

type [ShaderType](#)

Returns

[Task](#) [<WebGLShader>](#)

CreateTexture()


```
[Obsolete("Use the async version instead, which is already called internally.")]  
public WebGLTexture CreateTexture()
```

Returns

[WebGLTexture](#)

CreateTextureAsync()

```
public Task<WebGLTexture> CreateTextureAsync()
```

Returns

[Task](#) [WebGLTexture](#)

CullFace(Face)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void CullFace(Face mode)
```

Parameters

mode [Face](#)

CullFaceAsync(Face)

```
public Task CullFaceAsync(Face mode)
```

Parameters

mode [Face](#)

Returns

[Task](#)

DeleteBuffer(WebGLBuffer)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void DeleteBuffer(WebGLBuffer buffer)
```

Parameters

buffer [WebGLBuffer](#)

DeleteBufferAsync(WebGLBuffer)

```
public Task DeleteBufferAsync(WebGLBuffer buffer)
```

Parameters

buffer [WebGLBuffer](#)

Returns

[Task](#)

DeleteFramebuffer(WebGLFramebuffer)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void DeleteFramebuffer(WebGLFramebuffer buffer)
```

Parameters

buffer [WebGLFramebuffer](#)

DeleteFramebufferAsync(WebGLFramebuffer)

```
public Task DeleteFramebufferAsync(WebGLFramebuffer buffer)
```

Parameters

buffer [WebGLFramebuffer](#)

Returns

[Task](#) 

DeleteProgram(WebGLProgram)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void DeleteProgram(WebGLProgram program)
```

Parameters

program [WebGLProgram](#)

DeleteProgramAsync(WebGLProgram)

```
public Task DeleteProgramAsync(WebGLProgram program)
```

Parameters

program [WebGLProgram](#)

Returns

[Task](#) 

DeleteRenderbuffer(WebGLRenderbuffer)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void DeleteRenderbuffer(WebGLRenderbuffer buffer)
```

Parameters

buffer [WebGLRenderbuffer](#)

DeleteRenderbufferAsync(WebGLRenderbuffer)

```
public Task DeleteRenderbufferAsync(WebGLRenderbuffer buffer)
```

Parameters

buffer [WebGLRenderbuffer](#)

Returns

[Task](#) 

DeleteShader(WebGLShader)

```
[Obsolete("Use the async version instead, which is already called internally.")]
```

```
public void DeleteShader(WebGLShader shader)
```

Parameters

shader [WebGLShader](#)

DeleteShaderAsync(WebGLShader)

```
public Task DeleteShaderAsync(WebGLShader shader)
```

Parameters

shader [WebGLShader](#)

Returns

[Task](#) 

DeleteTexture(WebGLTexture)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void DeleteTexture(WebGLTexture texture)
```

Parameters

texture [WebGLTexture](#)

DeleteTextureAsync(WebGLTexture)

```
public Task DeleteTextureAsync(WebGLTexture texture)
```

Parameters

texture [WebGLTexture](#)

Returns

[Task](#)

DepthFunc(CompareFunction)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void DepthFunc(CompareFunction func)
```

Parameters

func [CompareFunction](#)

DepthFuncAsync(CompareFunction)

```
public Task DepthFuncAsync(CompareFunction func)
```

Parameters

func [CompareFunction](#)

Returns

[Task](#)

DepthMask(bool)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void DepthMask(bool flag)
```

Parameters

flag [bool](#)

DepthMaskAsync(bool)

```
public Task DepthMaskAsync(bool flag)
```

Parameters

flag [bool](#)

Returns

[Task](#)

DepthRange(float, float)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void DepthRange(float zNear, float zFar)
```

Parameters

zNear [float](#)

zFar [float](#)

DepthRangeAsync(float, float)

```
public Task DepthRangeAsync(float zNear, float zFar)
```

Parameters

zNear [float](#)

zFar [float](#)

Returns

[Task](#)

DetachShader(WebGLProgram, WebGLShader)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void DetachShader(WebGLProgram program, WebGLShader shader)
```

Parameters

program [WebGLProgram](#)

shader [WebGLShader](#)

DetachShaderAsync(WebGLProgram, WebGLShader)

```
public Task DetachShaderAsync(WebGLProgram program, WebGLShader shader)
```

Parameters

program [WebGLProgram](#)

shader [WebGLShader](#)

Returns

[Task](#)

Disable(EnableCap)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void Disable(EnableCap cap)
```

Parameters

cap [EnableCap](#)

DisableAsync(EnableCap)

```
public Task DisableAsync(EnableCap cap)
```

Parameters

cap [EnableCap](#)

Returns

[Task](#)

DisableVertexAttribArray(uint)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void DisableVertexAttribArray(uint index)
```

Parameters

index [uint](#)

DisableVertexArrayAsync(uint)

```
public Task DisableVertexArrayAsync(uint index)
```

Parameters

index [uint](#)

Returns

[Task](#)

DrawArrays(Primitive, int, int)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void DrawArrays(Primitive mode, int first, int count)
```

Parameters

mode [Primitive](#)

first [int](#)

count [int](#)

DrawArraysAsync(Primitive, int, int)

```
public Task DrawArraysAsync(Primitive mode, int first, int count)
```

Parameters

mode [Primitive](#)

first [int](#)

count [int](#)

Returns

[Task](#)

DrawElements(Primitive, int, DataType, long)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void DrawElements(Primitive mode, int count, DataType type, long offset)
```

Parameters

mode [Primitive](#)

count [int](#)

type [DataType](#)

offset [long](#)

DrawElementsAsync(Primitive, int, DataType, long)

```
public Task DrawElementsAsync(Primitive mode, int count, DataType type, long offset)
```

Parameters

mode [Primitive](#)

count [int](#)

type [DataType](#)

offset [long](#)

Returns

[Task](#)

Enable(EnableCap)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void Enable(EnableCap cap)
```

Parameters

cap [EnableCap](#)

EnableAsync(EnableCap)

```
public Task EnableAsync(EnableCap cap)
```

Parameters

cap [EnableCap](#)

Returns

[Task](#)[↗](#)

EnableVertexAttribArray(uint)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void EnableVertexAttribArray(uint index)
```

Parameters

index [uint](#)[↗](#)

EnableVertexAttribArrayAsync(uint)

```
public Task EnableVertexAttribArrayAsync(uint index)
```

Parameters

index [uint](#)

Returns

[Task](#)

ExtendedInitializeAsync()

```
protected override Task ExtendedInitializeAsync()
```

Returns

[Task](#)

Finish()

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void Finish()
```

FinishAsync()

```
public Task FinishAsync()
```

Returns

[Task](#)

Flush()

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void Flush()
```

FlushAsync()

```
public Task FlushAsync()
```

Returns

[Task](#)

FramebufferRenderbuffer(FramebufferType, FramebufferAttachment, RenderbufferType, WebGLRenderbuffer)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void FramebufferRenderbuffer(FramebufferType target, FramebufferAttachment  
attachment, RenderbufferType renderbuffertarget, WebGLRenderbuffer renderbuffer)
```

Parameters

target [FramebufferType](#)

attachment [FramebufferAttachment](#)

renderbuffertarget [RenderbufferType](#)

renderbuffer [WebGLRenderbuffer](#)

FramebufferRenderbufferAsync(FramebufferType, FramebufferAttachment, RenderbufferType, WebGLRenderbuffer)

```
public Task FramebufferRenderbufferAsync(FramebufferType target,  
FramebufferAttachment attachment, RenderbufferType renderbuffertarget,  
WebGLRenderbuffer renderbuffer)
```

Parameters

target [FramebufferType](#)

attachment [FramebufferAttachment](#)

renderbuffertarget [RenderbufferType](#)

renderbuffer [WebGLRenderbuffer](#)

Returns

[Task](#) 

FramebufferTexture2D(FramebufferType, FramebufferAttachment, Texture2DType, WebGLTexture, int)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void FramebufferTexture2D(FramebufferType target, FramebufferAttachment  
attachment, Texture2DType textarget, WebGLTexture texture, int level)
```

Parameters

target [FramebufferType](#)

attachment [FramebufferAttachment](#)

textarget [Texture2DType](#)

texture [WebGLTexture](#)

level [int](#) 

FramebufferTexture2DAsync(FramebufferType, FramebufferAttachment, Texture2DType, WebGLTexture, int)

```
public Task FramebufferTexture2DAsync(FramebufferType target, FramebufferAttachment  
attachment, Texture2DType textarget, WebGLTexture texture, int level)
```

Parameters

target [FramebufferType](#)

attachment [FramebufferAttachment](#)

textarget [Texture2DType](#)

texture [WebGLTexture](#)

level [int](#)

Returns

[Task](#)

FrontFace(FrontFaceDirection)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void FrontFace(FrontFaceDirection mode)
```

Parameters

mode [FrontFaceDirection](#)

FrontFaceAsync(FrontFaceDirection)

```
public Task FrontFaceAsync(FrontFaceDirection mode)
```

Parameters

mode [FrontFaceDirection](#)

Returns

[Task](#)

GenerateMipmap(TextureType)

```
[Obsolete("Use the async version instead, which is already called internally.")]
```

```
public void GenerateMipmap(TextureType target)
```

Parameters

target [TextureType](#)

GenerateMipmapAsync(TextureType)

```
public Task GenerateMipmapAsync(TextureType target)
```

Parameters

target [TextureType](#)

Returns

[Task](#)

GetActiveAttrib(WebGLProgram, uint)

```
[Obsolete("Use the async version instead, which is already called internally.")]
```

```
public WebGLActiveInfo GetActiveAttrib(WebGLProgram program, uint index)
```

Parameters

program [WebGLProgram](#)

index [uint](#)

Returns

[WebGLActiveInfo](#)

GetActiveAttribAsync(WebGLProgram, uint)


```
public Task<WebGLActiveInfo> GetActiveAttribAsync(WebGLProgram program, uint index)
```

Parameters

program [WebGLProgram](#)

index [uint](#)

Returns

[Task](#) <[WebGLActiveInfo](#)>

GetActiveUniform(WebGLProgram, uint)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public WebGLActiveInfo GetActiveUniform(WebGLProgram program, uint index)
```

Parameters

program [WebGLProgram](#)

index [uint](#)

Returns

[WebGLActiveInfo](#)

GetActiveUniformAsync(WebGLProgram, uint)

```
public Task<WebGLActiveInfo> GetActiveUniformAsync(WebGLProgram program, uint index)
```

Parameters

program [WebGLProgram](#)

index [uint](#)

Returns

[Task](#)  [WebGLActiveInfo](#)>

GetAttachedShaders(WebGLProgram)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public WebGLShader[] GetAttachedShaders(WebGLProgram program)
```

Parameters

program [WebGLProgram](#)

Returns

[WebGLShader](#)[]

GetAttachedShadersAsync(WebGLProgram)

```
public Task<WebGLShader[]> GetAttachedShadersAsync(WebGLProgram program)
```

Parameters

program [WebGLProgram](#)

Returns

[Task](#)  [WebGLShader](#)[]>

GetAttribLocation(WebGLProgram, string)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public int GetAttribLocation(WebGLProgram program, string name)
```

Parameters

program [WebGLProgram](#)

name [string](#)

Returns

[int](#)

GetAttribLocationAsync(WebGLProgram, string)

```
public Task<int> GetAttribLocationAsync(WebGLProgram program, string name)
```

Parameters

program [WebGLProgram](#)

name [string](#)

Returns

[Task](#)<[int](#)>

GetBufferParameterAsync<T>(BufferType, BufferParameter)

```
public Task<T> GetBufferParameterAsync<T>(BufferType target, BufferParameter pname)
```

Parameters

target [BufferType](#)

pname [BufferParameter](#)

Returns

[Task](#)<T>

Type Parameters

T

GetBufferParameter<T>(BufferType, BufferParameter)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public T GetBufferParameter<T>(BufferType target, BufferParameter pname)
```

Parameters

target [BufferType](#)

pname [BufferParameter](#)

Returns

T

Type Parameters

T

GetContextAttributes()

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public WebGLContextAttributes GetContextAttributes()
```

Returns

[WebGLContextAttributes](#)

GetContextAttributesAsync()

```
public Task<WebGLContextAttributes> GetContextAttributesAsync()
```

Returns

[Task](#) <[WebGLContextAttributes](#)>

GetError()

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public Error GetError()
```

Returns

[Error](#)

GetErrorAsync()

```
public Task<Error> GetErrorAsync()
```

Returns

[Task](#) <[Error](#)>

GetFramebufferAttachmentParameterAsync<T> (FramebufferType, FramebufferAttachment, FramebufferAttachmentParameter)

```
public Task<T> GetFramebufferAttachmentParameterAsync<T>(FramebufferType target,  
FramebufferAttachment attachment, FramebufferAttachmentParameter pname)
```

Parameters

target [FramebufferType](#)

attachment [FramebufferAttachment](#)

pname [FramebufferAttachmentParameter](#)

Returns

[Task](#) <T>

Type Parameters

T

GetFramebufferAttachmentParameter<T> (FramebufferType, FramebufferAttachment, FramebufferAttachmentParameter)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public T GetFramebufferAttachmentParameter<T>(FramebufferType target,  
FramebufferAttachment attachment, FramebufferAttachmentParameter pname)
```

Parameters

target [FramebufferType](#)

attachment [FramebufferAttachment](#)

pname [FramebufferAttachmentParameter](#)

Returns

T

Type Parameters

T

GetParameterAsync<T>(Parameter)

```
public Task<T> GetParameterAsync<T>(Parameter parameter)
```

Parameters

parameter [Parameter](#)

Returns

[Task](#)  <T>

Type Parameters

T

GetParameter<T>(Parameter)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public T GetParameter<T>(Parameter parameter)
```

Parameters

parameter [Parameter](#)

Returns

T

Type Parameters

T

GetProgramInfoLog(WebGLProgram)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public string GetProgramInfoLog(WebGLProgram program)
```

Parameters

program [WebGLProgram](#)

Returns

[string](#) 

GetProgramInfoLogAsync(WebGLProgram)

```
public Task<string> GetProgramInfoLogAsync(WebGLProgram program)
```

Parameters

program [WebGLProgram](#)

Returns

[Task](#)<[string](#)>

GetProgramParameterAsync<T>(WebGLProgram, ProgramParameter)

```
public Task<T> GetProgramParameterAsync<T>(WebGLProgram program,  
ProgramParameter pname)
```

Parameters

program [WebGLProgram](#)

pname [ProgramParameter](#)

Returns

[Task](#)<T>

Type Parameters

T

GetProgramParameter<T>(WebGLProgram, ProgramParameter)

```
[Obsolete("Use the async version instead, which is already called internally.")]
```



```
public T GetProgramParameter<T>(WebGLProgram program, ProgramParameter pname)
```

Parameters

program [WebGLProgram](#)

pname [ProgramParameter](#)

Returns

T

Type Parameters

T

GetRenderbufferParameterAsync<T>(RenderbufferType, RenderbufferParameter)

```
public Task<T> GetRenderbufferParameterAsync<T>(RenderbufferType target,  
RenderbufferParameter pname)
```

Parameters

target [RenderbufferType](#)

pname [RenderbufferParameter](#)

Returns

[Task](#) <T>

Type Parameters

T

GetRenderbufferParameter<T>(RenderbufferType, RenderbufferParameter)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public T GetRenderbufferParameter<T>(RenderbufferType target,  
RenderbufferParameter pname)
```

Parameters

target [RenderbufferType](#)

pname [RenderbufferParameter](#)

Returns

T

Type Parameters

T

GetShaderInfoLog(WebGLShader)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public string GetShaderInfoLog(WebGLShader shader)
```

Parameters

shader [WebGLShader](#)

Returns

[string](#) 

GetShaderInfoLogAsync(WebGLShader)

```
public Task<string> GetShaderInfoLogAsync(WebGLShader shader)
```

Parameters

shader [WebGLShader](#)

Returns

[Task](#) [<string>](#)

GetShaderParameterAsync<T>(WebGLShader, ShaderParameter)

```
public Task<T> GetShaderParameterAsync<T>(WebGLShader shader, ShaderParameter pname)
```

Parameters

shader [WebGLShader](#)

pname [ShaderParameter](#)

Returns

[Task](#) [<T>](#)

Type Parameters

T

GetShaderParameter<T>(WebGLShader, ShaderParameter)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public T GetShaderParameter<T>(WebGLShader shader, ShaderParameter pname)
```

Parameters

shader [WebGLShader](#)

pname [ShaderParameter](#)

Returns

T

Type Parameters

T

GetShaderPrecisionFormat(ShaderType, ShaderPrecision)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public WebGLShaderPrecisionFormat GetShaderPrecisionFormat(ShaderType shaderType,  
    ShaderPrecision precisionType)
```

Parameters

shaderType [ShaderType](#)

precisionType [ShaderPrecision](#)

Returns

[WebGLShaderPrecisionFormat](#)

GetShaderPrecisionFormatAsync(ShaderType, Shader Precision)

```
public Task<WebGLShaderPrecisionFormat> GetShaderPrecisionFormatAsync(ShaderType  
    shaderType, ShaderPrecision precisionType)
```

Parameters

shaderType [ShaderType](#)

precisionType [ShaderPrecision](#)

Returns

[Task](#) <[WebGLShaderPrecisionFormat](#)>

GetShaderSource(WebGLShader)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public string GetShaderSource(WebGLShader shader)
```

Parameters

shader [WebGLShader](#)

Returns

[string](#)

GetShaderSourceAsync(WebGLShader)

```
public Task<string> GetShaderSourceAsync(WebGLShader shader)
```

Parameters

shader [WebGLShader](#)

Returns

[Task](#) <[string](#)>

GetTexParameterAsync<T>(TextureType, TextureParameter)

```
public Task<T> GetTexParameterAsync<T>(TextureType target, TextureParameter pname)
```

Parameters

target [TextureType](#)

pname [TextureParameter](#)

Returns

[Task](#)<T>

Type Parameters

T

GetTexParameter<T>(TextureType, TextureParameter)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public T GetTexParameter<T>(TextureType target, TextureParameter pname)
```

Parameters

target [TextureType](#)

pname [TextureParameter](#)

Returns

T

Type Parameters

T

GetUniformAsync<T>(WebGLProgram, WebGLUniformLocation)

```
public Task<T> GetUniformAsync<T>(WebGLProgram program, WebGLUniformLocation  
location)
```

Parameters

program [WebGLProgram](#)

location [WebGLUniformLocation](#)

Returns

[Task](#)  <T>

Type Parameters

T

GetUniformLocation(WebGLProgram, string)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public WebGLUniformLocation GetUniformLocation(WebGLProgram program, string name)
```

Parameters

program [WebGLProgram](#)

name [string](#) 

Returns

[WebGLUniformLocation](#)

GetUniformLocationAsync(WebGLProgram, string)

```
public Task<WebGLUniformLocation> GetUniformLocationAsync(WebGLProgram program,  
string name)
```

Parameters

program [WebGLProgram](#)

name [string](#) 

Returns

[Task](#) <[WebGLUniformLocation](#)>

GetUniform<T>(WebGLProgram, WebGLUniformLocation)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public T GetUniform<T>(WebGLProgram program, WebGLUniformLocation location)
```

Parameters

program [WebGLProgram](#)

location [WebGLUniformLocation](#)

Returns

T

Type Parameters

T

GetVertexAttribAsync<T>(uint, VertexAttribute)

```
public Task<T> GetVertexAttribAsync<T>(uint index, VertexAttribute pname)
```

Parameters

index [uint](#)

pname [VertexAttribute](#)

Returns

[Task](#) <T>

Type Parameters

T

GetVertexAttribOffset(uint, VertexAttributePointer)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public long GetVertexAttribOffset(uint index, VertexAttributePointer pname)
```

Parameters

index [uint](#)

pname [VertexAttributePointer](#)

Returns

[long](#)

GetVertexAttribOffsetAsync(uint, VertexAttributePointer)

```
public Task<long> GetVertexAttribOffsetAsync(uint index, VertexAttributePointer  
pname)
```

Parameters

index [uint](#)

pname [VertexAttributePointer](#)

Returns

[Task](#)<[long](#)>

GetVertexAttrib<T>(uint, VertexAttribute)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public T GetVertexAttrib<T>(uint index, VertexAttribute pname)
```

Parameters

index [uint](#)

pname [VertexAttribute](#)

Returns

T

Type Parameters

T

Hint(HintTarget, HintMode)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void Hint(HintTarget target, HintMode mode)
```

Parameters

target [HintTarget](#)

mode [HintMode](#)

HintAsync(HintTarget, HintMode)

```
public Task HintAsync(HintTarget target, HintMode mode)
```

Parameters

target [HintTarget](#)

mode [HintMode](#)

Returns

[Task](#)

IsBuffer(WebGLBuffer)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public bool IsBuffer(WebGLBuffer buffer)
```

Parameters

buffer [WebGLBuffer](#)

Returns

[bool](#)

IsBufferAsync(WebGLBuffer)

```
public Task<bool> IsBufferAsync(WebGLBuffer buffer)
```

Parameters

buffer [WebGLBuffer](#)

Returns

[Task](#) <[bool](#)>

IsContextLost()

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public bool IsContextLost()
```

Returns

[bool](#)

IsContextLostAsync()

```
public Task<bool> IsContextLostAsync()
```

Returns

[Task](#) [<bool>](#)

IsEnabled(EnableCap)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public bool IsEnabled(EnableCap cap)
```

Parameters

cap [EnableCap](#)

Returns

[bool](#)

IsEnabledAsync(EnableCap)

```
public Task<bool> IsEnabledAsync(EnableCap cap)
```

Parameters

cap [EnableCap](#)

Returns

[Task](#) [<bool>](#)

IsFramebuffer(WebGLFramebuffer)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public bool IsFramebuffer(WebGLFramebuffer framebuffer)
```

Parameters

framebuffer [WebGLFramebuffer](#)

Returns

[bool](#) 

IsFramebufferAsync(WebGLFramebuffer)

```
public Task<bool> IsFramebufferAsync(WebGLFramebuffer framebuffer)
```

Parameters

framebuffer [WebGLFramebuffer](#)

Returns

[Task](#)  [bool](#) 

IsProgram(WebGLProgram)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public bool IsProgram(WebGLProgram program)
```

Parameters

program [WebGLProgram](#)

Returns

[bool](#) 

IsProgramAsync(WebGLProgram)

```
public Task<bool> IsProgramAsync(WebGLProgram program)
```

Parameters

program [WebGLProgram](#)

Returns

[Task](#) <[bool](#)>

IsRenderbuffer(WebGLRenderbuffer)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public bool IsRenderbuffer(WebGLRenderbuffer renderbuffer)
```

Parameters

renderbuffer [WebGLRenderbuffer](#)

Returns

[bool](#)

IsRenderbufferAsync(WebGLRenderbuffer)

```
public Task<bool> IsRenderbufferAsync(WebGLRenderbuffer renderbuffer)
```

Parameters

renderbuffer [WebGLRenderbuffer](#)

Returns

[Task](#) <[bool](#)>

IsShader(WebGLShader)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public bool IsShader(WebGLShader shader)
```

Parameters

shader [WebGLShader](#)

Returns

[bool](#)

IsShaderAsync(WebGLShader)

```
public Task<bool> IsShaderAsync(WebGLShader shader)
```

Parameters

shader [WebGLShader](#)

Returns

[Task](#) <[bool](#)>

IsTexture(WebGLTexture)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public bool IsTexture(WebGLTexture texture)
```

Parameters

texture [WebGLTexture](#)

Returns

[bool](#)

IsTextureAsync(WebGLTexture)

```
public Task<bool> IsTextureAsync(WebGLTexture texture)
```

Parameters

texture [WebGLTexture](#)

Returns

[Task](#) <[bool](#)>

LineWidth(float)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void LineWidth(float width)
```

Parameters

width [float](#)

LineWidthAsync(float)

```
public Task LineWidthAsync(float width)
```

Parameters

width [float](#)

Returns

[Task](#)

LinkProgram(WebGLProgram)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void LinkProgram(WebGLProgram program)
```

Parameters

program [WebGLProgram](#)

LinkProgramAsync(WebGLProgram)

```
public Task LinkProgramAsync(WebGLProgram program)
```

Parameters

program [WebGLProgram](#)

Returns

[Task](#)

PixelStoreI(PixelStorageMode, int)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public bool PixelStoreI(PixelStorageMode pname, int param)
```

Parameters

pname [PixelStorageMode](#)

param [int](#)

Returns

[bool](#)

PixelStoreIAsync(PixelStorageMode, int)

```
public Task<bool> PixelStoreIAsync(PixelStorageMode pname, int param)
```

Parameters

pname [PixelStorageMode](#)

param [int](#)

Returns

[Task](#) <[bool](#)>

PolygonOffset(float, float)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void PolygonOffset(float factor, float units)
```

Parameters

factor [float](#)

units [float](#)

PolygonOffsetAsync(float, float)

```
public Task PolygonOffsetAsync(float factor, float units)
```

Parameters

factor [float](#)

units [float](#)

Returns

ReadPixels(int, int, int, int, PixelFormat, PixelType, byte[])

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void ReadPixels(int x, int y, int width, int height, PixelFormat format,  
    PixelType type, byte[] pixels)
```

Parameters

x [int](#)

y [int](#)

width [int](#)

height [int](#)

format [PixelFormat](#)

type [PixelType](#)

pixels [byte](#)[]

ReadPixelsAsync(int, int, int, int, PixelFormat, PixelType, byte[])

```
public Task ReadPixelsAsync(int x, int y, int width, int height, PixelFormat format,  
    PixelType type, byte[] pixels)
```

Parameters

x [int](#)

y [int](#)

width [int](#)

height [int](#)

format [PixelFormat](#)

type [PixelFormat](#)

pixels [byte](#)[]

Returns

[Task](#)

RenderbufferStorage(RenderbufferType, RenderbufferFormat, int, int)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void RenderbufferStorage(RenderbufferType type, RenderbufferFormat  
internalFormat, int width, int height)
```

Parameters

type [RenderbufferType](#)

internalFormat [RenderbufferFormat](#)

width [int](#)

height [int](#)

RenderbufferStorageAsync(RenderbufferType, RenderbufferFormat, int, int)

```
public Task RenderbufferStorageAsync(RenderbufferType type, RenderbufferFormat  
internalFormat, int width, int height)
```

Parameters

type [RenderbufferType](#)

internalFormat [RenderbufferFormat](#)

width [int](#)

height [int](#)

Returns

[Task](#)

SampleCoverage(float, bool)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void SampleCoverage(float value, bool invert)
```

Parameters

value [float](#)

invert [bool](#)

SampleCoverageAsync(float, bool)

```
public Task SampleCoverageAsync(float value, bool invert)
```

Parameters

value [float](#)

invert [bool](#)

Returns

[Task](#)

Scissor(int, int, int, int)

```
[Obsolete("Use the async version instead, which is already called internally.")]
```

```
public void Scissor(int x, int y, int width, int height)
```

Parameters

x [int](#)

y [int](#)

width [int](#)

height [int](#)

ScissorAsync(int, int, int, int)

```
public Task ScissorAsync(int x, int y, int width, int height)
```

Parameters

x [int](#)

y [int](#)

width [int](#)

height [int](#)

Returns

[Task](#)

ShaderSource(WebGLShader, string)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void ShaderSource(WebGLShader shader, string source)
```

Parameters

shader [WebGLShader](#)

source [string](#)

ShaderSourceAsync(WebGLShader, string)

```
public Task ShaderSourceAsync(WebGLShader shader, string source)
```

Parameters

shader [WebGLShader](#)

source [string](#)

Returns

[Task](#)

StencilFunc(CompareFunction, int, uint)

[Obsolete("Use the async version instead, which is already called internally.")]

```
public void StencilFunc(CompareFunction func, int reference, uint mask)
```

Parameters

func [CompareFunction](#)

reference [int](#)

mask [uint](#)

StencilFuncAsync(CompareFunction, int, uint)

```
public Task StencilFuncAsync(CompareFunction func, int reference, uint mask)
```

Parameters

func [CompareFunction](#)

reference [int](#)

mask [uint](#)

Returns

[Task](#)

StencilFuncSeparate(Face, CompareFunction, int, uint)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void StencilFuncSeparate(Face face, CompareFunction func, int reference,  
uint mask)
```

Parameters

face [Face](#)

func [CompareFunction](#)

reference [int](#)

mask [uint](#)

StencilFuncSeparateAsync(Face, CompareFunction, int, uint)

```
public Task StencilFuncSeparateAsync(Face face, CompareFunction func, int reference,  
uint mask)
```

Parameters

face [Face](#)

func [CompareFunction](#)

reference [int](#)

mask [uint](#)

Returns

[Task](#)

StencilMask(uint)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void StencilMask(uint mask)
```

Parameters

mask [uint](#)

StencilMaskAsync(uint)

```
public Task StencilMaskAsync(uint mask)
```

Parameters

mask [uint](#)

Returns

[Task](#)

StencilMaskSeparate(Face, uint)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void StencilMaskSeparate(Face face, uint mask)
```

Parameters

face [Face](#)

mask [uint](#)

StencilMaskSeparateAsync(Face, uint)

```
public Task StencilMaskSeparateAsync(Face face, uint mask)
```

Parameters

face [Face](#)

mask [uint](#)

Returns

[Task](#)

StencilOp(StencilFunction, StencilFunction, StencilFunction)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void StencilOp(StencilFunction fail, StencilFunction zfail,  
    StencilFunction zpass)
```

Parameters

fail [StencilFunction](#)

zfail [StencilFunction](#)

zpass [StencilFunction](#)

StencilOpAsync(StencilFunction, StencilFunction, StencilFunction)

```
public Task StencilOpAsync(StencilFunction fail, StencilFunction zfail,  
    StencilFunction zpass)
```

Parameters

fail [StencilFunction](#)

zfail [StencilFunction](#)

zpass [StencilFunction](#)

Returns

[Task](#)

StencilOpSeparate(Face, StencilFunction, StencilFunction, StencilFunction)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void StencilOpSeparate(Face face, StencilFunction fail, StencilFunction  
zfail, StencilFunction zpass)
```

Parameters

face [Face](#)

fail [StencilFunction](#)

zfail [StencilFunction](#)

zpass [StencilFunction](#)

StencilOpSeparateAsync(Face, StencilFunction, StencilFunction, StencilFunction)

```
public Task StencilOpSeparateAsync(Face face, StencilFunction fail, StencilFunction  
zfail, StencilFunction zpass)
```

Parameters

face [Face](#)

fail [StencilFunction](#)

zfail [StencilFunction](#)

zpass [StencilFunction](#)

Returns

[Task](#)[↗]

TexImage2DAsync<T>(Texture2DType, int, PixelFormat, int, int, PixelFormat, PixelType, T[])

```
public Task TexImage2DAsync<T>(Texture2DType target, int level, PixelFormat
internalFormat, int width, int height, PixelFormat format, PixelType type, T[]
pixels) where T : struct
```

Parameters

target [Texture2DType](#)

level [int](#)[↗]

internalFormat [PixelFormat](#)

width [int](#)[↗]

height [int](#)[↗]

format [PixelFormat](#)

type [PixelType](#)

pixels T[]

Returns

[Task](#)[↗]

Type Parameters

T

TexImage2D<T>(Texture2DType, int, PixelFormat, int, int, PixelFormat, PixelType, T[])

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void TexImage2D<T>(Texture2DType target, int level, PixelFormat  
internalFormat, int width, int height, PixelFormat format, PixelType type, T[]  
pixels) where T : struct
```

Parameters

target [Texture2DType](#)

level [int](#)

internalFormat [PixelFormat](#)

width [int](#)

height [int](#)

format [PixelFormat](#)

type [PixelType](#)

pixels T[]

Type Parameters

T

TexParameter(TextureType, TextureParameter, int)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void TexParameter(TextureType target, TextureParameter pname, int param)
```

Parameters

target [TextureType](#)

pname [TextureParameter](#)

param [int](#)

TexParameter(TextureType, TextureParameter, float)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void TexParameter(TextureType target, TextureParameter pname, float param)
```

Parameters

target [TextureType](#)

pname [TextureParameter](#)

param [float](#)

TexParameterAsync(TextureType, TextureParameter, int)

```
public Task TexParameterAsync(TextureType target, TextureParameter pname, int param)
```

Parameters

target [TextureType](#)

pname [TextureParameter](#)

param [int](#)

Returns

[Task](#)

TexParameterAsync(TextureType, TextureParameter, float)

```
public Task TexParameterAsync(TextureType target, TextureParameter pname,  
float param)
```

Parameters

target [TextureType](#)

pname [TextureParameter](#)

param [float](#)

Returns

[Task](#)

TexSubImage2DAsync<T>(Texture2DType, int, int, int, int, int, PixelFormat, PixelType, T[])

```
public Task TexSubImage2DAsync<T>(Texture2DType target, int level, int xoffset, int yoffset, int width, int height, PixelFormat format, PixelType type, T[] pixels)
where T : struct
```

Parameters

target [Texture2DType](#)

level [int](#)

xoffset [int](#)

yoffset [int](#)

width [int](#)

height [int](#)

format [PixelFormat](#)

type [PixelType](#)

pixels T[]

Returns

[Task](#)

Type Parameters

T

TexSubImage2D<T>(Texture2DType, int, int, int, int, int, PixelFormat, PixelType, T[])

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void TexSubImage2D<T>(Texture2DType target, int level, int xoffset, int  
yoffset, int width, int height, PixelFormat format, PixelType type, T[] pixels)  
where T : struct
```

Parameters

target [Texture2DType](#)

level [int](#)

xoffset [int](#)

yoffset [int](#)

width [int](#)

height [int](#)

format [PixelFormat](#)

type [PixelType](#)

pixels T[]

Type Parameters

T

Uniform(WebGLUniformLocation, params int[])


```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void Uniform(WebGLUniformLocation location, params int[] value)
```

Parameters

location [WebGLUniformLocation](#)

value [int](#)[]

Uniform(WebGLUniformLocation, params float[])

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void Uniform(WebGLUniformLocation location, params float[] value)
```

Parameters

location [WebGLUniformLocation](#)

value [float](#)[]

UniformAsync(WebGLUniformLocation, params int[])

```
public Task UniformAsync(WebGLUniformLocation location, params int[] value)
```

Parameters

location [WebGLUniformLocation](#)

value [int](#)[]

Returns

[Task](#)

UniformAsync(WebGLUniformLocation, params float[])

```
public Task UniformAsync(WebGLUniformLocation location, params float[] value)
```

Parameters

location [WebGLUniformLocation](#)

value [float](#)[]

Returns

[Task](#)

UniformMatrix(WebGLUniformLocation, bool, float[])

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void UniformMatrix(WebGLUniformLocation location, bool transpose,  
float[] value)
```

Parameters

location [WebGLUniformLocation](#)

transpose [bool](#)

value [float](#)[]

UniformMatrixAsync(WebGLUniformLocation, bool, float[])

```
public Task UniformMatrixAsync(WebGLUniformLocation location, bool transpose,  
float[] value)
```

Parameters

location [WebGLUniformLocation](#)

transpose [bool](#)

value [float](#)[]

Returns

[Task](#) 

UseProgram(WebGLProgram)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void UseProgram(WebGLProgram program)
```

Parameters

program [WebGLProgram](#)

UseProgramAsync(WebGLProgram)

```
public Task UseProgramAsync(WebGLProgram program)
```

Parameters

program [WebGLProgram](#)

Returns

[Task](#) 

ValidateProgram(WebGLProgram)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void ValidateProgram(WebGLProgram program)
```

Parameters

program [WebGLProgram](#)

ValidateProgramAsync(WebGLProgram)

```
public Task ValidateProgramAsync(WebGLProgram program)
```

Parameters

program [WebGLProgram](#)

Returns

[Task](#)

VertexAttrib(uint, params float[])

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void VertexAttrib(uint index, params float[] value)
```

Parameters

index [uint](#)

value [float](#)[]

VertexAttribAsync(uint, params float[])

```
public Task VertexAttribAsync(uint index, params float[] value)
```

Parameters

index [uint](#)

value [float](#)[]

Returns

[Task](#)

VertexAttribPointer(uint, int, DataType, bool, int, long)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void VertexAttribPointer(uint index, int size, DataType type, bool  
normalized, int stride, long offset)
```

Parameters

index [uint](#)

size [int](#)

type [DataType](#)

normalized [bool](#)

stride [int](#)

offset [long](#)

VertexAttribPointerAsync(uint, int, DataType, bool, int, long)

```
public Task VertexAttribPointerAsync(uint index, int size, DataType type, bool  
normalized, int stride, long offset)
```

Parameters

index [uint](#)

size [int](#)

type [DataType](#)

normalized [bool](#)

stride [int](#)

offset [long](#)

Returns

[Task](#)

Viewport(int, int, int, int)

```
[Obsolete("Use the async version instead, which is already called internally.")]  
public void Viewport(int x, int y, int width, int height)
```

Parameters

x [int](#)

y [int](#)

width [int](#)

height [int](#)

ViewportAsync(int, int, int, int)

```
public Task ViewportAsync(int x, int y, int width, int height)
```

Parameters

x [int](#)

y [int](#)

width [int](#)

height [int](#)

Returns

[Task](#)

Class WebGLContextAttributes

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public class WebGLContextAttributes
```

Inheritance

[object](#)  ← WebGLContextAttributes

Inherited Members

[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

WebGLContextAttributes()

```
public WebGLContextAttributes()
```

Fields

POWER_PREFERENCE_DEFAULT

```
public const string POWER_PREFERENCE_DEFAULT = "default"
```

Field Value

[string](#) 

POWER_PREFERENCE_HIGH_PERFORMANCE

```
public const string POWER_PREFERENCE_HIGH_PERFORMANCE = "high-performance"
```

Field Value

[string](#)↗

POWER_PREFERENCE_LOW_POWER

```
public const string POWER_PREFERENCE_LOW_POWER = "low-power"
```

Field Value

[string](#)↗

Properties

Alpha

```
public bool Alpha { get; set; }
```

Property Value

[bool](#)↗

Antialias

```
public bool Antialias { get; set; }
```

Property Value

[bool](#)↗

Depth

```
public bool Depth { get; set; }
```


Property Value

[bool](#)

FailIfMajorPerformanceCaveat

```
public bool FailIfMajorPerformanceCaveat { get; set; }
```

Property Value

[bool](#)

PowerPreference

```
public string PowerPreference { get; set; }
```

Property Value

[string](#)

PremultipliedAlpha

```
public bool PremultipliedAlpha { get; set; }
```

Property Value

[bool](#)

PreserveDrawingBuffer

```
public bool PreserveDrawingBuffer { get; set; }
```

Property Value

[bool](#) 

Stencil

```
public bool Stencil { get; set; }
```

Property Value

[bool](#) 

Class WebGLFramebuffer

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public class WebGLFramebuffer : WebGLObject
```

Inheritance

[object](#)  ← [WebGLObject](#) ← WebGLFramebuffer

Inherited Members

[WebGLObject.WebGLType](#) , [WebGLObject.Id](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  ,
[object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  ,
[object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

WebGLFramebuffer()

```
public WebGLFramebuffer()
```

Class WebGLObject

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public class WebGLObject
```

Inheritance

[object](#)  ← WebGLObject

Derived

[WebGLBuffer](#), [WebGLFramebuffer](#), [WebGLProgram](#), [WebGLRenderbuffer](#), [WebGLShader](#),
[WebGLTexture](#), [WebGLUniformLocation](#)

Inherited Members

[object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ToString\(\)](#) , [object.Equals\(object\)](#) ,
[object.Equals\(object, object\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.GetHashCode\(\)](#) 

Constructors

WebGLObject()

```
public WebGLObject()
```

Properties

Id

```
public int Id { get; set; }
```

Property Value

[int](#) 

WebGLType

```
public string WebGLType { get; set; }
```

Property Value

[string](#) 

Class WebGLProgram

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public class WebGLProgram : WebGLObject
```

Inheritance

[object](#)  ← [WebGLObject](#) ← WebGLProgram

Inherited Members

[WebGLObject.WebGLType](#) , [WebGLObject.Id](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  ,
[object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  ,
[object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

WebGLProgram()

```
public WebGLProgram()
```

Class WebGLRenderbuffer

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public class WebGLRenderbuffer : WebGLObject
```

Inheritance

[object](#)  ← [WebGLObject](#) ← WebGLRenderbuffer

Inherited Members

[WebGLObject.WebGLType](#) , [WebGLObject.Id](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  ,
[object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  ,
[object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

WebGLRenderbuffer()

```
public WebGLRenderbuffer()
```

Class WebGLShader

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public class WebGLShader : WebGLObject
```

Inheritance

[object](#)  ← [WebGLObject](#) ← WebGLShader

Inherited Members

[WebGLObject.WebGLType](#) , [WebGLObject.Id](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  ,
[object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  ,
[object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

WebGLShader()

```
public WebGLShader()
```


Class WebGLShaderPrecisionFormat

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public class WebGLShaderPrecisionFormat
```

Inheritance

[object](#)  ← WebGLShaderPrecisionFormat

Inherited Members

[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ToString\(\)](#)  , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

WebGLShaderPrecisionFormat()

```
public WebGLShaderPrecisionFormat()
```

Properties

Precision

```
public int Precision { get; set; }
```

Property Value

[int](#) 

RangeMax

```
public int RangeMax { get; set; }
```

Property Value

[int](#)

RangeMin

```
public int RangeMin { get; set; }
```

Property Value

[int](#)

Class WebGLTexture

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public class WebGLTexture : WebGLObject
```

Inheritance

[object](#)  ← [WebGLObject](#) ← WebGLTexture

Inherited Members

[WebGLObject.WebGLType](#) , [WebGLObject.Id](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  ,
[object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  ,
[object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

WebGLTexture()

```
public WebGLTexture()
```

Class WebGLUniformLocation

Namespace: [Blazor.Extensions.Canvas.WebGL](#)

Assembly: Blazor.Extensions.Canvas.dll

```
public class WebGLUniformLocation : WebGLObject
```

Inheritance

[object](#)  ← [WebGLObject](#) ← WebGLUniformLocation

Inherited Members

[WebGLObject.WebGLType](#) , [WebGLObject.Id](#) , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  ,
[object.ToString\(\)](#)  , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  ,
[object.ReferenceEquals\(object, object\)](#)  , [object.GetHashCode\(\)](#) 

Constructors

WebGLUniformLocation()

```
public WebGLUniformLocation()
```